

Table of Contents

Section / Chapter	Title	Page
Section 1	Introduction	1
1.1	Project Overview	1
1.2	Problem Statement	1
1.3	Project Objectives	2
1.4	System Scope	2
1.5	Technology Stack	3
1.6	System Architecture & Primary User Interface	3
1.7	Report Organization	4
Section 2	Background Research	5
2.1	Overview of Global E-commerce Platforms	5
2.2	Amazon	5
2.3	Noon	6
2.4	eBay	6
2.5	Technical Gap Analysis and Platform Comparison	7
2.6	Architectural Design Decisions	8
2.7	Competitive Advantages Summary	10
2.8	Limitations of the Current System	11
2.9	Conclusion	11
Section 3	Web Front-End — MVC Views, CSS, and Client-Side JavaScript	12
3.1	Design Token System (CSS Custom Properties)	12
3.2	Light Theme Override	13
3.3	Layout Architecture	14
3.4	Home Page (Views/Home/Index.cshtml)	16
3.5	Product Listing Page (Views/Products/Index.cshtml)	19
3.6	Cart Page (Views/Cart/Index.cshtml)	23
3.7	Checkout Page (Views/Checkout/Index.cshtml)	26
3.8	AI Assistant Modal	29
3.9	Theme Toggle JavaScript	32
3.10	Toast Notification System (wwwroot/js/site.js)	33
3.11	Wishlist Toggle (Optimistic UI Update)	35
3.12	Animation System	37
3.13	Form Controls & Semantic Colours	38

Section / Chapter	Title	Page
3.14	Admin Dashboard Styles	39
3.15	Grid System	40
3.16	Miscellaneous Components	40
Section 4	AI Integration — Google Gemini Product Assistant	42
4.1	Objective and Architectural Context	42
4.2	API Configuration and Secure Key Management	42
4.3	GeminiService Implementation	43
4.4	Controller Integration	47
4.5	Error Handling Strategy	50
4.6	Cost and Performance Considerations	51
Section 5	Backend Architecture — ASP.NET Core MVC E-Commerce Engine	52
5.1	Project Overview and Architectural Pattern	52
5.2	Data Access Layer	52
5.3	Entities and Database Schema	57
5.4	Business Logic — Product & Category Management (Admin)	64
5.5	Performance Optimization — In-Memory Caching	67
5.6	Cart Logic with Stock Validation	68
5.7	Checkout, Concurrency, and Payments	71
5.8	Security and Error Handling	78
5.9	Database Seeding	80
5.10	Email Service (Transactional Emails)	81
5.11	Analytics Service	82
5.12	Dependency Injection Summary (Program.cs)	83
Section 6	Agile User Stories & Requirements Traceability	84
6.1	Introduction	84
6.2	Epic 1: User Identity & Security	84
6.3	Epic 2: Product Browsing & Discovery	86
6.4	Epic 3: Shopping Cart & Wishlist	88
6.5	Epic 4: Checkout & Payments	90
6.6	Epic 5: AI Product Assistant (Gemini Integration)	92
6.7	Epic 6: Admin Dashboard & Management	94
6.8	Epic 7: Core Infrastructure & User Experience	96
6.9	Story Point Summary	98
6.10	Proposed System Screens and Interaction Walkthrough	99

Section / Chapter	Title	Page
Section 7	System Modeling & Architectural Diagrams	104
7.1	Entity-Relationship Diagram (ERD)	104
7.2	System Use Case Diagram	107
7.3	Checkout Sequence Diagram	109
7.4	Overall Program Flowchart	112
Section 8	Conclusion & References	115
8.1	Conclusion	115
8.2	References	116

1. Introduction

1.1 Project Overview

The E-commerce platform presented in this project is a full-stack web application built on ASP.NET Core 10.0 MVC. It provides a complete online shopping experience including product browsing, cart management, order processing, payment handling, and an administrative dashboard. The system also integrates an AI-powered product assistant using the Google Gemini API.

The project was designed with two primary goals. First, to create a production-ready E-commerce system that a small business could deploy and use to sell products online. Second, to serve as a comprehensive demonstration of modern ASP.NET Core development patterns including the repository pattern, unit of work, Entity Framework Core with SQL Server, ASP.NET Core Identity for authentication, dependency injection throughout, and third-party API integration for payments and AI.

The decision to build the platform from scratch rather than using a hosted solution like Shopify or WooCommerce was intentional. Building from scratch gives full control over the database schema, the business logic, the UI, and the hosting environment. It also avoids monthly subscription fees and platform-specific limitations such as restricted access to customer data or forced use of specific payment gateways.

The application is structured into three logical layers:

- **Controllers** handle HTTP requests and responses. They validate input, call service methods, and return views or JSON responses.
- **Services** contain the business logic. They orchestrate operations that span multiple repositories or interact with external APIs.
- **Repositories** abstract data access behind interfaces. They use Entity Framework Core to query SQL Server and return domain models.

This separation follows the single responsibility principle. Each layer has a distinct job, and changes to one layer (for example, switching from SQL Server to PostgreSQL) ideally affect only the repository layer.

1.2 Problem Statement

Small and medium businesses in Egypt and the broader Middle East face a specific set of challenges when trying to sell online. Understanding these challenges is important because they directly shaped the feature set of this project.

The first challenge is payment methods. Credit card penetration in Egypt is low. According to data from the Central Bank of Egypt, card-carrying adults represent less than 10% of the population. The dominant payment method for online shopping is cash on delivery (COD). Customers want to order online and pay in cash when the package arrives. Global platforms like Amazon Egypt offer COD for some items, but smaller merchants using self-hosted solutions often struggle to add COD support because it requires manual order management.

The second challenge is cost. Hosted E-commerce platforms charge recurring fees. Shopify starts at \$39 per month plus transaction fees. WooCommerce is free as a plugin but requires WordPress hosting, a domain name, and often paid extensions for features like Stripe integration or advanced shipping. For a small business testing the online market, these costs can be prohibitive.

The third challenge is technical control. Using a hosted platform means the business does not own the customer data in a directly accessible format. Exporting data is possible but requires platform-specific tools. Customizing the checkout flow or adding a new payment method requires the platform's permission and often developer fees.

The fourth challenge is localization. Many global platforms do not handle Arabic content well. The UI layout changes for right-to-left languages, currency formatting varies by country, and local tax rules (like Egypt's 14% VAT) are not always supported out of the box.

This project addresses these challenges by providing:

1. Cash on delivery as a built-in payment method alongside Stripe credit card processing
2. A self-hosted solution with no recurring platform fees
3. Full source code access and database control
4. Support for 14% VAT calculation built into the order pipeline
5. An admin panel designed for managing orders, products, and customers without third-party tools

1.3 Project Objectives

The project was built to meet the following objectives. Each objective maps to one or more modules in the final system.

Objective 1: Product Catalog with Filtering and Search

The system must allow customers to browse products by category, search by name or description, filter by price range, and sort by price or name. The product listing must support pagination and load quickly even with hundreds of products. Category-based browsing should work both from the homepage and through a dedicated category page.

Objective 2: Shopping Cart Management

Authenticated users must be able to add products to a cart, update quantities, remove items, and clear the cart. The cart must persist across sessions and show a running total including tax. Variant-based products (different sizes or colors) must be supported with separate stock tracking per variant.

Objective 3: Order Processing with Payment

The system must handle the full order lifecycle: checkout form with shipping details, promo code validation, order creation, payment processing, and order confirmation. Two payment methods must be supported: credit card via Stripe Checkout and cash on delivery. Orders must be stored with full line-item detail for future reference.

Objective 4: User Account Management

Users must be able to register, log in, reset their password via email, update their profile, change their password, and delete their account. Authentication must use ASP.NET Core Identity with role-based authorization separating customers from administrators.

Objective 5: Admin Dashboard

Administrators must have a dedicated panel for managing products, categories, orders, users, and promo codes. The dashboard must show key metrics: total revenue, order counts by status, top-selling products, recent orders, and sales charts.

Objective 6: AI-Powered Product Assistant

The system must integrate a generative AI assistant that customers can use to ask questions about products. The assistant must be context-aware, responding based on the product name and description, and must handle API errors gracefully including rate limiting.

Objective 7: Concurrency-Safe Stock Management

The system must prevent stock overselling when two customers place orders for the same product simultaneously. This must be handled at the database level using row versioning and optimistic concurrency with automatic retry logic.

1.4 System Scope

The application covers the following modules. Each module is a distinct functional area with its own controller, views, and data access logic.

Product Catalog

The catalog module handles product listing, detail views, filtering, and search. Products are organized into categories. Each product has a name, description, price, stock count, image, and optional variants. Featured products are highlighted on the homepage. The catalog supports server-side pagination with 12 products per page. AJAX filtering was added to improve the browsing experience by avoiding full page reloads when filters change.

Shopping Cart

The cart module manages the user's selected items. It supports adding products with a specific quantity, updating quantities, removing individual items, and clearing the entire cart. When a product has variants (size or color), the user selects the variant before adding to cart. Stock is checked at add time to prevent adding more items than available.

Checkout and Orders

The checkout module collects shipping information, validates promo codes, calculates totals with tax, and processes payment. For COD orders, the order is confirmed immediately. For credit card orders, the user is redirected to Stripe Checkout and the order is confirmed on successful return. Order confirmation emails are sent asynchronously.

User Accounts

Built on ASP.NET Core Identity, the accounts module handles registration, login, logout, password reset, profile updates, and account deletion. Role management separates Admin and Customer access. The profile page shows order history, wishlist count, and account statistics.

Admin Panel

The admin panel provides full CRUD for products, categories, and promo codes. It includes order management with status updates and user management with activate/deactivate controls. The analytics dashboard shows revenue metrics, order statistics, top products, top customers, and category performance with Chart.js visualizations.

AI Assistant

The AI assistant is accessible from any product detail page. Users click a button to open a modal, type a question about the product, and receive an AI-generated answer. The backend calls the Google Gemini API with a prompt built from the product name and description.

Wishlist

The wishlist module lets users save products for later. It supports adding and removing items via AJAX, checking if a product is in the wishlist, and displaying the wishlist count in the navigation bar.

Order History

Authenticated users can view all their past orders through the `OrdersController`, which provides a list view (`MyOrders`) and a detailed view (`Details`) for each individual order with its line items and payment status.

1.5 Technology Stack

The following table lists the key technologies used in this project along with the specific version and purpose.

Layer	Technology	Version	Purpose
Backend Framework	ASP.NET Core MVC	10.0	Web application framework
Object-Relational Mapping	Entity Framework Core	10.0	Database access and migration
Database	SQL Server	2022	Relational data storage
Authentication	ASP.NET Core Identity	10.0	User management and roles
Payments	Stripe SDK	Latest	Credit card payment processing

Layer	Technology	Version	Purpose
AI	Google Gemini API	v1beta	Product assistant
Frontend CSS	Bootstrap	5.3	Responsive layout and components
Frontend Icons	Bootstrap Icons	1.11	UI icon set
Charts	Chart.js	4.4	Admin analytics charts
Caching	In-Memory Cache	Built-in	Category and featured product caching
HTTP Client	HttpClientFactory	Built-in	Gemini API communication
Email	System.Net.Mail.SmtpClient	Built-in	Transactional email sending

The entry point of the application is `Program.cs`, which configures services, middleware, and the request pipeline.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.User.RequireUniqueEmail = true;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<IPaymentService, StripePaymentService>();
builder.Services.AddScoped<IAalyticsService, AnalyticsService>();
builder.Services.AddScoped<IImageService, ImageService>();
builder.Services.AddScoped<IGeminiService, GeminiService>();

builder.Services.AddMemoryCache();
builder.Services.AddResponseCaching();
builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseResponseCaching();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");
```

```
app.Run();
```

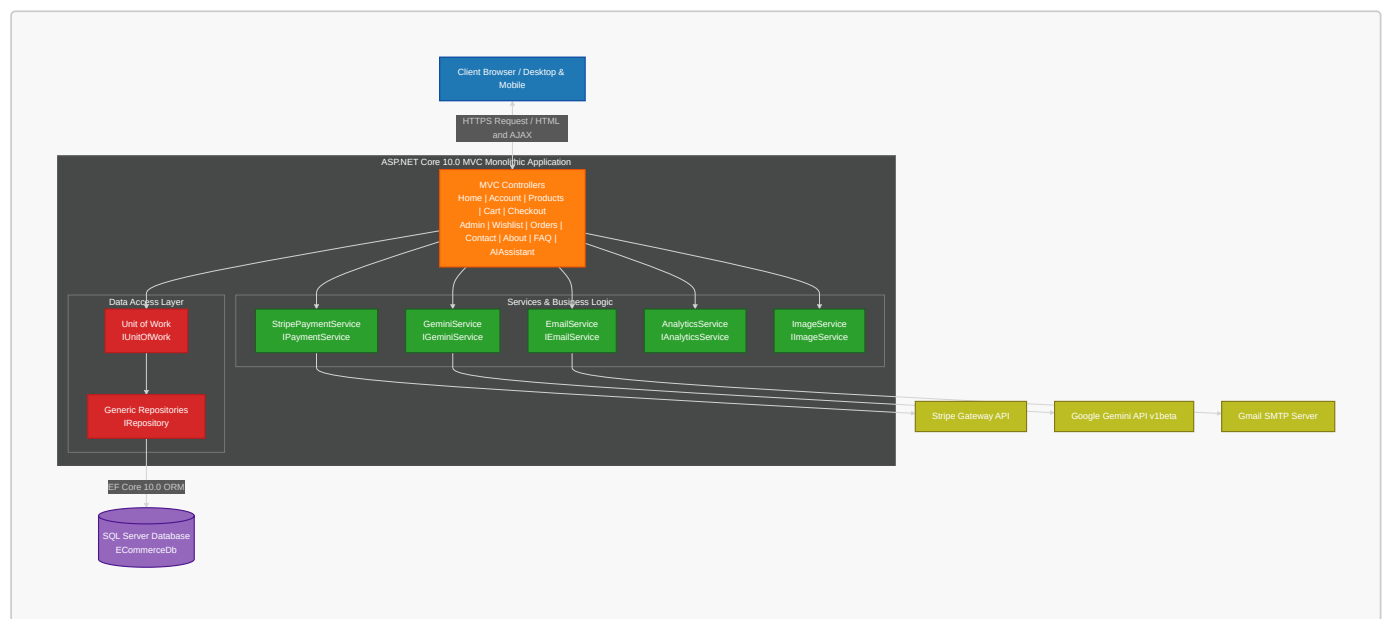
The **Program.cs** file is divided into three sections. The first section registers services with the dependency injection container. The second section configures the middleware pipeline. The third section seeds the database with initial data (admin user, sample categories, and products).

Service registration uses the scoped lifetime because **IUnitOfWork** and the services that depend on it should be created once per HTTP request and disposed when the request ends. **IMemoryCache** and **IHttpClientFactory** are singletons, shared across all requests.

The middleware pipeline order is important. **UseAuthentication** and **UseAuthorization** are placed between **UseRouting** and **UseMapControllerRoute**. This ensures that the authorization middleware can inspect the route before deciding whether to allow access. **UseResponseCaching** is placed before **UseRouting** so that cached responses can be served without executing the MVC pipeline.

1.6 System Architecture & Primary User Interface

The following system architecture flowchart illustrates the relationships between the client-side browser, MVC controllers, application business services, the repository/data access layer, SQL Server database, and external APIs (Google Gemini, Stripe, and SMTP).



To provide an immediate understanding of the visual style and design aesthetics of the developed application, the homepage design is presented below. It features the responsive glassmorphism header, the hero banner with a dynamic visual ring, and the auto-fill grids for categories and featured products:

Ataba

Home

Categories

About

Search products...

2

2

A

Premium Quality

Discover
Exceptional
Products

Curated collections of premium items for the discerning buyer. Experience luxury shopping redefined.

Shop Now

Explore

Free Shipping

On orders over \$50

Secure Payment

100% secure checkout

Easy Returns

30-day return policy

Shop by Category

Browse our curated collections

Accessories

Fashion & Style

Footwear

Lifestyle

Luxury Watches

Smart Electronics

Featured Products

Handpicked selections for you

FEATURED



Elegant Gold Watch

A stunning gold-plated timepiece with premium craftsmanship. Perf...

\$15,999.00

IN STOCK

View

FEATURED



Classic Silver Timepiece

Timeless silver watch with minimalist design. Swiss movement for ...

\$12,500.00

IN STOCK

View

FEATURED



Diamond Accent Watch

Luxurious watch featuring diamond accents and premium leather str...

\$22,500.00

IN STOCK

View

FEATURED



Premium Wool Blazer

Elegant wool blazer with perfect fit. Ideal for business meetings...

\$4,500.00

IN STOCK

View

FEATURED



Designer Outfit Set

Complete outfit set with coordinated pieces for a sophisticated L...

\$5,500.00

IN STOCK

View

FEATURED



Premium Wireless Earbuds

High-quality wireless earbuds with noise cancellation and premium...

\$2,800.00

IN STOCK

View

FEATURED



Leather Wallet Set

Premium leather wallet set with card holder and coin purse.

\$1,200.00

IN STOCK

View

FEATURED



Premium Yoga Mat

High-quality yoga mat with superior grip and comfort.

\$950.00

IN STOCK

View

View All Products →

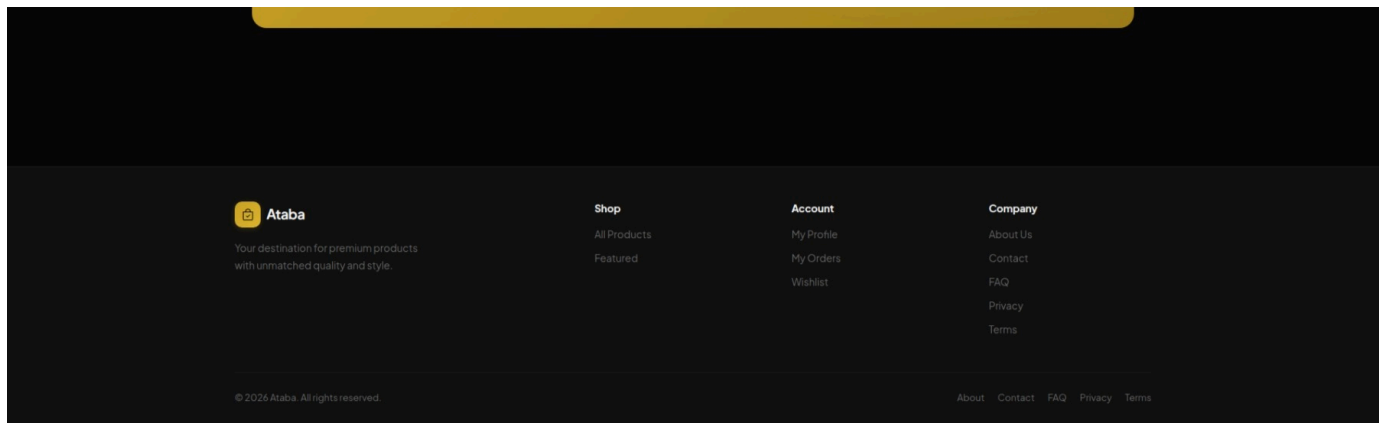
Stay in the Loop

Get exclusive offers and new arrivals straight to your inbox

Enter your email address

Subscribe

9 / 130



1.7 Report Organization

This book is divided into eight detailed sections:

- **Section 1** (this section) introduces the project, defines the problem it solves, lists the objectives, and describes the scope, technology stack, high-level architecture, and the primary user interface.
- **Section 2** presents background research on existing E-commerce platforms including Amazon, Noon, and eBay. It analyzes their business models, technical architectures, and limitations, compares them to our custom system, and highlights our competitive advantages.
- **Section 3** describes the frontend implementation. It covers the Razor view structure, Bootstrap 5 layout, AJAX filtering, responsive design, and JavaScript features including the cart, wishlist, and the AI assistant modal.
- **Section 4** focuses on the AI integration. It explains how the Google Gemini API is called from ASP.NET Core, how the prompt is constructed, how rate limiting and errors are handled, and the security measures around the API.
- **Section 5** covers the backend architecture in detail. It includes the database schema with all entity models, the repository and unit of work patterns, controller logic for each module, authentication and authorization configuration, payment processing with Stripe and COD, concurrency control with row versioning, and core support services.
- **Section 6** details the Agile development methodology, including 38 user stories across 7 epics (covering all implemented controller actions from authentication and checkout through AI assistant, order history, and account deletion), and presents a complete visual walkthrough of the designed system's screens and step-by-step user interaction steps.
- **Section 7** presents system modeling and diagrams, including the Entity-Relationship Diagram (ERD), System Use Case Diagram, Checkout Sequence Diagram, and the overall System Program Flowchart.
- **Section 8** concludes the book, summarizes achievements and future developmental scopes, and outlines technical academic references.

2. Background Research

2.1 Overview of Global E-commerce Platforms

Before designing this system, it was important to understand the existing landscape of E-commerce platforms. The three largest platforms — Amazon, Noon, and eBay — each operate with different business models and technical architectures. Studying their strengths and weaknesses helped identify the features that a custom ASP.NET Core platform should prioritize.

2.2 Amazon

Amazon is the largest online retailer globally by revenue and market capitalization. It operates a hybrid model: Amazon sells its own inventory (first-party) and also allows third-party merchants to list products on the platform (third-party marketplace).

2.2.1 Business Model

Amazon's third-party marketplace allows merchants to create product listings, set prices, and fulfill orders either through Amazon's Fulfillment by Amazon (FBA) service or through their own shipping. Amazon charges referral fees that vary by category, typically around 15% of the sale price. Merchants also pay additional fees for FBA storage, shipping, advertising, and subscription fees for professional selling accounts.

The platform's revenue comes from multiple sources: referral fees, subscription fees (Amazon Prime), advertising (sponsored products), and cloud services (AWS). This diversification allows Amazon to operate profitably even if the marketplace itself runs on thin margins.

2.2.2 Technical Architecture

Amazon's technical infrastructure is proprietary and not publicly documented in detail. However, several architectural characteristics are known from published papers and talks:

- **Microservices:** Amazon migrated from a monolithic architecture to microservices in the early 2000s. Each service owns its data and communicates through defined APIs.
- **Distributed databases:** Amazon uses DynamoDB (its own NoSQL database) for the shopping cart and session management, and a relational database layer for order processing and inventory.
- **Recommendation engine:** The product recommendation system processes user behavior data (views, purchases, searches) to generate personalized suggestions. This is one of Amazon's most valuable technical assets and is credited with driving 30% of total sales.
- **Search indexing:** Product search uses a custom search index with faceted filtering, spell correction, and ranked results based on relevance and sales velocity.

2.2.3 Limitations for Small Merchants

For a small business in Egypt, Amazon has several disadvantages:

- Referral fees of 15% significantly reduce profit margins, especially for low-cost items
- The merchant has no direct access to customer email addresses or purchase history — Amazon owns the customer relationship
- Customizing the storefront beyond basic branding options is not possible
- COD support on Amazon Egypt is limited to certain products and regions
- The seller dashboard provides standard reports but cannot be extended with custom analytics

2.3 Noon

Noon is an E-commerce platform founded in 2017 by Mohamed Alabbar (Emaar Properties) with investment from the Public Investment Fund of Saudi Arabia. It operates primarily in Saudi Arabia, the United Arab Emirates, and Egypt.

2.3.1 Business Model

Noon follows a similar model to Amazon: it sells first-party inventory and hosts third-party merchants. Noon differentiates itself through:

- Same-day delivery in major Gulf cities
- COD as a standard payment option across all markets
- Arabic-first user interface with full right-to-left support
- Local customer service centers in each market

Noon does not disclose its fee structure publicly, but estimates suggest referral fees of 10-20% depending on the category.

2.3.2 Technical Architecture

Noon is built on a combination of open-source and commercial technologies. The frontend is built with React and the backend uses Java-based microservices. The platform runs on cloud infrastructure and uses Elasticsearch for product search.

2.3.3 Relevance to This Project

Noon's success in the Middle East validates several design decisions made in this project:

- COD is not a niche payment method — it is a requirement for the Middle Eastern market
- Arabic support and local currency formatting matter for user adoption
- Fast delivery is a competitive advantage, which means the admin panel needs efficient order management to help merchants process orders quickly

2.4 eBay

eBay operates a different model from Amazon and Noon. It is primarily a peer-to-peer marketplace for both new and used goods, supporting both fixed-price listings and auctions.

2.4.1 Business Model

eBay charges insertion fees for listings (up to a certain number per month are free) and final value fees when an item sells, typically 10-15% of the sale price. Unlike Amazon, eBay does not hold inventory or fulfill orders — all transactions are between the buyer and seller directly.

2.4.2 Technical Architecture

eBay's architecture has evolved from a single monolithic Perl application in the 1990s to a Java-based microservices platform. Known technical details include:

- A search infrastructure that indexes billions of listings in near real-time
- A feedback and reputation system that tracks buyer and seller trustworthiness
- API access for third-party developers to create listing tools and analytics dashboards

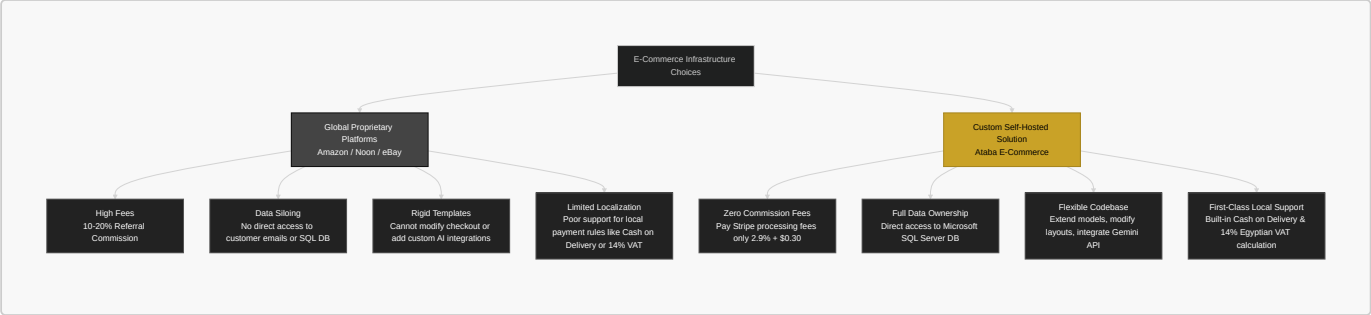
2.4.3 Limitations

eBay's auction model is not suitable for a fixed-price retail store. The platform is also known for buyer protection policies that can favor buyers over sellers in disputes. For a small business wanting to build a brand, eBay provides limited branding control and no direct customer relationship.

2.5 Technical Gap Analysis and Platform Comparison

Comparing the three major platforms to a custom ASP.NET Core solution highlights how a custom self-hosted system bridges key operational and financial gaps for local merchants.

The flowchart below categorizes the trade-offs between utilizing proprietary global platforms (Amazon, Noon, eBay) and deploying our custom, self-hosted system (Ataba):

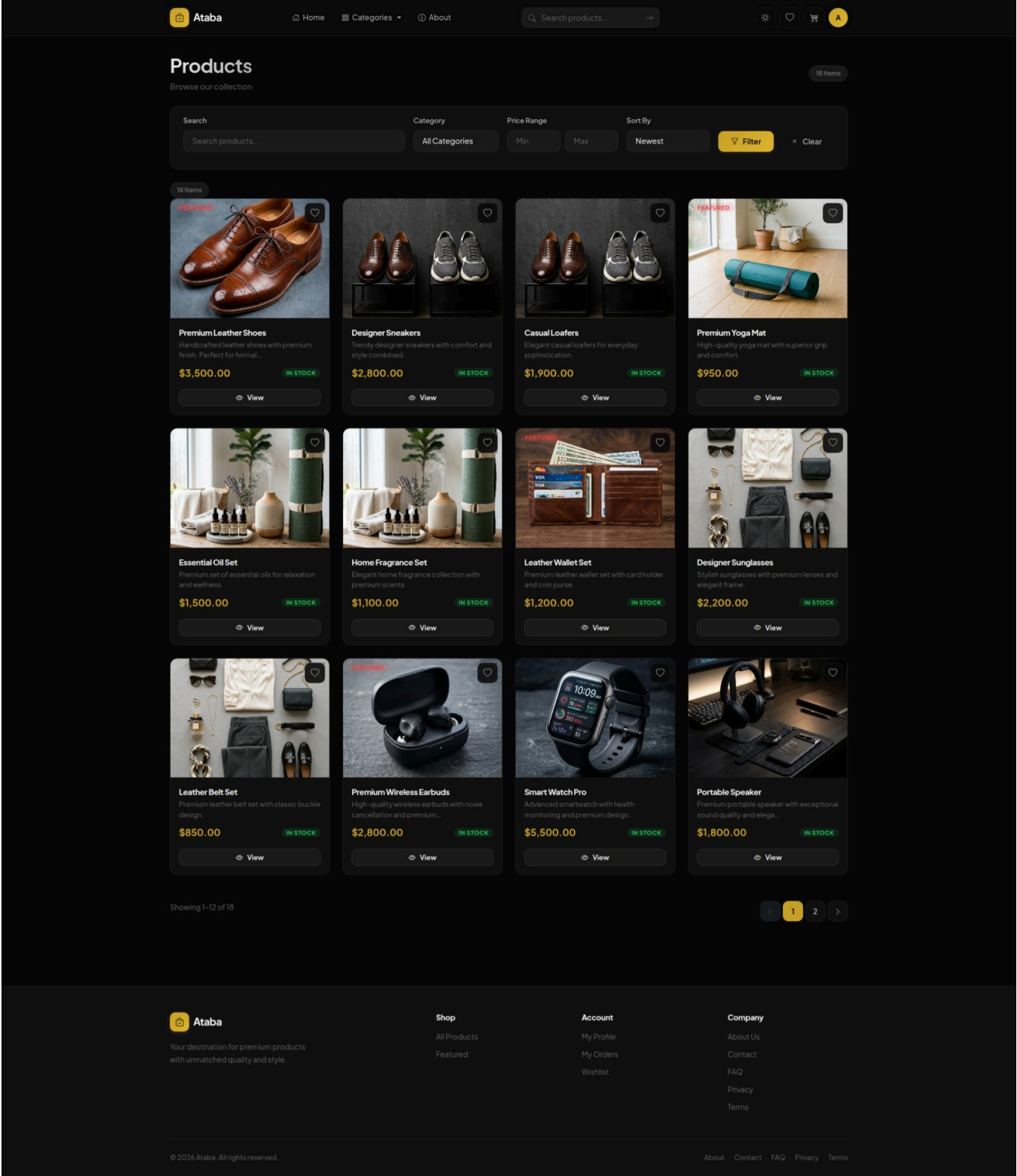


Below is the comparative table detailing the specific technical and financial metrics of each platform versus our proposed system:

Feature	Amazon	Noon	eBay	This Project (Ataba)
Monthly subscription fee	Professional account \$39.99/month	Commission-based	Store subscription \$4.95-\$27.95/month	None
Per-transaction fee	15% average referral fee	10-20% estimated	10-15% final value fee	Stripe processing fee only (2.9% + \$0.30)
COD support	Limited regions	Yes	No	Yes (First-class built-in)
Source code access	No	No	No	Full (MIT license)
Database access	No	No	No	Full SQL Server access
Custom checkout flow	No	No	No	Full control
Promo/coupon engine	Yes (seller-funded)	Yes	No	Built-in promo code system
AI assistant	Alexa shopping	No	No	Gemini product Q&A (Integrated)
Data ownership	Limited	Limited	Limited	Full ownership
Self-hosted	No	No	No	Yes
Technology stack	Proprietary	Proprietary	Proprietary	.NET 10 + SQL Server

The key differentiator is data ownership and customization. When a business uses Amazon, it does not own the customer relationship — Amazon does. When a business uses this platform, all customer data, order history, and analytics are stored in its own SQL Server database. The business can run custom reports, segment customers based on purchase history, and integrate the data with its own accounting or CRM system.

To illustrate the product listing and filtering experience that our custom system offers, the interface below shows the AJAX-driven product catalog with the sidebar filters and sorting controls:



2.6 Architectural Design Decisions

Several architectural decisions were made after reviewing how the major platforms handle common E-commerce problems.

2.6.1 Repository Pattern and Unit of Work

The repository pattern is widely used in enterprise .NET applications to abstract data access. Instead of writing Entity Framework queries directly in controllers, the application defines repository interfaces and implementations for each entity. The controllers depend on the interfaces, not the implementation. This has three benefits:

1. **Testability:** Repository interfaces can be mocked in unit tests, allowing controller logic to be tested without connecting to a real database.
2. **Consistency:** All data access goes through the same pattern, making it easier to audit for performance issues like N+1 queries.
3. **Flexibility:** The implementation can be changed (for example, from Entity Framework to Dapper) without changing the controllers.

The `IRepository<T>` interface defines the standard data operations:

```
public interface IRepository<T> where T : class
{
    IQueryable<T> GetQueryable(bool asNoTracking);
    Task<IEnumerable<T>> GetAllAsync();
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter);
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter,
        params Expression<Func<T, object>>[] includes);
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter,
        bool asNoTracking, params Expression<Func<T, object>>[] includes);
    Task<T?> GetByIdAsync(int id);
    Task<T?> GetByIdAsync(int id, bool asNoTracking,
        params Expression<Func<T, object>>[] includes);
    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter);
    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter,
        bool asNoTracking, params Expression<Func<T, object>>[] includes);
    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
    void DeleteRange(IEnumerable<T> entities);
    Task<bool> AnyAsync(Expression<Func<T, bool>> filter);
    Task<int> CountAsync(Expression<Func<T, bool>>? filter = null);
}
```

Each method has overloads for common variations. The `asNoTracking` parameter controls whether Entity Framework tracks the returned entities. Read-only queries that do not modify data should use `asNoTracking: true` to avoid the overhead of change tracking. The `includes` parameter accepts expressions that cause Entity Framework to eagerly load related data, preventing the N+1 query problem.

The implementation `Repository<T>` uses Entity Framework Core internally:

```
public class Repository<T> : IRepository<T> where T : class
{
    private readonly ApplicationDbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repository(ApplicationDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public IQueryable<T> GetQueryable(bool asNoTracking)
    {
        return asNoTracking ? _dbSet.AsNoTracking() : _dbSet;
    }

    public async Task<IEnumerable<T>> GetAsync(
        Expression<Func<T, bool>> filter,
```

```

        bool asNoTracking,
        params Expression<Func<T, object>>[] includes)
    {
        var query = GetQueryable(asNoTracking).Where(filter);
        query = ApplyIncludes(query, includes);
        return await query.ToListAsync();
    }

    public async Task<T?> GetByIdAsync(int id)
    {
        return await _dbSet.FindAsync(id);
    }

    public async Task AddAsync(T entity)
    {
        await _dbSet.AddAsync(entity);
    }

    public void Update(T entity)
    {
        _dbSet.Update(entity);
    }

    public void Delete(T entity)
    {
        _dbSet.Remove(entity);
    }
}

```

The `IUnitOfWork` interface groups all repositories and provides a shared `SaveAsync` method and transaction support:

```

public interface IUnitOfWork : IDisposable
{
    IRepository<Category> Categories { get; }
    IRepository<Product> Products { get; }
    IRepository<ProductVariant> ProductVariants { get; }
    IRepository<Order> Orders { get; }
    IRepository<OrderItem> OrderItems { get; }
    IRepository<ShoppingCart> ShoppingCarts { get; }
    IRepository<Payment> Payments { get; }
    IRepository<ApplicationUser> Users { get; }
    IRepository<ProductReview> ProductReviews { get; }
    IRepository<PromoCode> PromoCodes { get; }
    IRepository<Wishlist> Wishlists { get; }
    Task<int> SaveAsync();
    Task<IDbContextTransaction> BeginTransactionAsync();
}

```

The `UnitOfWork` implementation creates a single `ApplicationDbContext` instance and passes it to each repository. This means all repository operations within a single HTTP request share the same database context. When `SaveAsync` is called, all pending changes are sent to SQL Server in one transaction. When `BeginTransactionAsync` is called explicitly (for order processing), multiple `SaveAsync` calls are grouped into a single database transaction that can be committed or rolled back atomically.

2.6.2 Why Not a Microservices Architecture?

Amazon and Noon both use microservices. However, for this project, a monolithic MVC architecture was the correct choice. The reasons are:

- **Development speed:** A monolith is faster to build and debug, especially for a single developer
- **Deployment simplicity:** One application to deploy, one process to monitor
- **Transaction consistency:** Distributed transactions across microservices are complex and error-prone. A monolith with a single database makes it straightforward to maintain ACID guarantees during order processing
- **Scale requirements:** For a small to medium business handling hundreds or low thousands of orders per day, a single .NET application with SQL Server is more than sufficient
- **No complex orchestration needed:** Microservices add value when different teams need to deploy independently or when different services have different scaling requirements. Neither applies here

If the platform grows to handle millions of orders per day, the monolith can be split into services. The repository pattern and interface-based design make this refactoring possible without rewriting the entire codebase.

2.6.3 SQL Server Choice

SQL Server was chosen over other databases for several reasons:

- Common in Egyptian university curricula and local hosting environments
- Strong integration with Entity Framework Core through the `Microsoft.EntityFrameworkCore.SqlServer` provider
- Support for `ROWVERSION` for concurrency control (not available in SQLite)
- Mature tooling (SQL Server Management Studio, Azure Data Studio)
- Free Developer Edition for development and small-scale deployment

The connection is configured with a specific port number (1433) and `TrustServerCertificate=True` for development convenience:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost,1433;Database=ECommerceDb;User
Id=sa;Password=your_password;TrustServerCertificate=True"
  }
}
```

2.6.4 Concurrency Control with Row Versioning

One of the most technically interesting challenges in E-commerce is preventing stock overselling. Consider this race condition:

1. Customer A loads a product page that shows 5 units in stock
2. Customer B loads the same product page at the same time, also seeing 5 units
3. Customer A places an order for 5 units — stock is updated to 0
4. Customer B places an order for 3 units — but the stock was already 0

Without concurrency control, step 4 would succeed and the business would be unable to fulfill Customer B's order. Entity Framework provides `DbUpdateConcurrencyException` for exactly this scenario.

The `Product` entity includes a `RowVersion` property decorated with the `[Timestamp]` attribute:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
}
```

```
public decimal Price { get; set; }
public int Stock { get; set; }

[Timestamp]
public byte[] RowVersion { get; set; } = [];
}
```

When Entity Framework generates an UPDATE statement for a row with a `[Timestamp]` property, it includes the original version value in the WHERE clause:

```
UPDATE Products SET Stock = @Stock
WHERE Id = @Id AND RowVersion = @OriginalRowVersion;
```

If another transaction has already updated the row, the RowVersion in the database no longer matches. SQL Server reports zero rows affected, and Entity Framework throws `DbUpdateConcurrencyException`.

The checkout controller handles this exception with a retry loop:

```
const int maxRetries = 3;

for (int attempt = 1; attempt <= maxRetries; attempt++)
{
    await using var transaction = await _unitOfWork.BeginTransactionAsync();

    try
    {
        // Load cart items and products
        // Check stock for all items
        // Deduct stock and create order
        // Save to database
        await _unitOfWork.SaveAsync();
        await transaction.CommitAsync();

        return RedirectToAction("OrderConfirmation", new { orderId = order.Id });
    }
    catch (DbUpdateConcurrencyException) when (attempt < maxRetries)
    {
        await transaction.RollbackAsync();
        await Task.Delay(100 * attempt);
        continue;
    }
    catch (DbUpdateConcurrencyException) when (attempt >= maxRetries)
    {
        await transaction.RollbackAsync();
        TempData["ErrorMessage"] = "Some items were just purchased. Please review your cart.";
        return RedirectToAction("Index");
    }
}
```

The retry delay increases with each attempt (100ms, 200ms, 300ms). This is called exponential backoff. The delay gives the conflicting transaction time to complete before retrying. If all three attempts fail, the user is shown an error message and asked to review their cart.

This approach is called optimistic concurrency control. It assumes conflicts are rare (optimistic) and handles them when they occur rather than locking rows preventively (pessimistic). For an E-commerce application with moderate traffic, optimistic concurrency is the right choice because most of the time, customers are browsing different products and conflicts do not occur.

2.7 Competitive Advantages Summary

Based on the research above, the following competitive advantages were identified for this project:

1. **No transaction fees beyond payment processing:** Unlike Amazon's 15% referral fee, this platform only incurs the Stripe processing fee of 2.9% plus \$0.30 per transaction. For a product priced at 500 EGP, Amazon's fee would be approximately 75 EGP while Stripe's fee would be approximately 17 EGP.
2. **COD support as a first-class feature:** COD is implemented alongside credit card payments in the same checkout flow. The admin panel supports order management for COD orders, including status tracking and delivery confirmation.
3. **Full database ownership:** All customer, product, and order data is stored in the merchant's own SQL Server database. This enables custom reporting, data integration with other business systems, and complete data portability.
4. **Built-in promo code engine:** Percentage-based and fixed-amount discounts with configurable limits, expiration dates, minimum purchase amounts, and usage caps. The promo code validation is exposed via a real-time AJAX endpoint so users see the discount before submitting the order.
5. **AI assistant integration:** The Gemini-powered Q&A modal provides a feature that none of the major platforms offer as a built-in tool. For product categories where customers frequently ask questions (electronics, fashion), this can reduce support workload.
6. **Extensible architecture:** Because the codebase uses the repository pattern and dependency injection, adding new features follows a consistent pattern. A new entity requires creating a model class, adding a `DbSet` to the context, defining repository operations if the standard ones are insufficient, writing a controller, and creating the views. The scaffolding is well-defined.

2.8 Limitations of the Current System

The research also identified areas where the current system falls short of the major platforms. These are documented here as limitations and possible directions for future work.

- **Search indexing:** Product search uses SQL Server's `LIKE` operator, which does not scale to tens of thousands of products. Amazon uses Elasticsearch, which provides full-text search with relevance ranking, typo tolerance, and faceted filtering. Adding Elasticsearch or using SQL Server's full-text search indexes would significantly improve search quality.
- **Recommendation engine:** The platform does not have a recommendation system. Amazon attributes a large portion of its revenue to personalized product recommendations. Implementing a simple rule-based recommender (frequently bought together, customers who viewed this also viewed) would be a meaningful addition.
- **Mobile application:** The platform is web-only. Amazon, Noon, and eBay all have native mobile applications with push notifications, camera-based barcode scanning, and offline browsing. A React Native or Flutter wrapper around the existing API could address this.
- **Performance optimization:** The current application uses in-memory caching for categories and featured products. For production use at scale, a distributed cache like Redis would be more appropriate. The response caching middleware is useful but does not cache personalized content.
- **Email deliverability:** The `Smtplib` approach works but has deliverability issues compared to transactional email services (SendGrid, Mailgun, Amazon SES). These services provide better deliverability tracking, template management, and reputation monitoring.

- **No automated testing:** The project does not include unit or integration tests. For a production deployment, tests are essential to prevent regressions when adding features or fixing bugs. The repository pattern makes unit testing controllers straightforward because repository interfaces can be mocked.

2.9 Conclusion

The research phase confirmed that building a custom ASP.NET Core E-commerce platform is a viable alternative to using existing hosted solutions. The major platforms charge significant fees, restrict data access, and limit customization. For a small to medium business in Egypt, a self-hosted .NET solution with COD support, a promo code engine, and an AI assistant provides a competitive feature set at a fraction of the cost.

The architectural decisions — repository pattern, unit of work, optimistic concurrency control, and monolithic MVC — were made based on the scale requirements of the target market. Microservices, NoSQL databases, and distributed search are powerful technologies but would add unnecessary complexity for the current scope. The architecture is designed to evolve: the repository interfaces can be re-implemented for a different database, the monolithic controllers can be split into microservices, and a search index can be added without modifying the existing data access code.

3. Web Front-End — MVC Views, CSS, and Client-Side JavaScript

3.1 Design Token System (CSS Custom Properties)

The entire visual language is driven by CSS custom properties declared on `:root` and overridden for the light theme via `[data-bs-theme="light"]`. No hardcoded colours exist beyond the token values — every component references these variables, giving us a centralized design system that can be re-themed in one place.

```
:root {  
  /* Backgrounds */  
  --bg-primary: #080808;  
  --bg-secondary: #111111;  
  --bg-tertiary: #1a1a1a;  
  --bg-elevated: #222222;  
  --bg-hover: #2a2a2a;  
  
  /* Accent (gold) */  
  --accent: #c9a227;  
  --accent-hover: #e0b630;  
  --accent-dim: #a8871f;  
  --accent-muted: rgba(201, 162, 39, 0.12);  
  --accent-glow: rgba(201, 162, 39, 0.20);  
  
  /* Text */  
  --text-primary: #f5f5f5;  
  --text-secondary: #a8a8a8;  
  --text-muted: #666666;  
  
  /* Semantic */  
  --success: #22c55e;  
  --danger: #ef4444;  
  --warning: #f59e0b;  
  --info: #38bdf8;  
  
  /* Borders */  
  --border-subtle: rgba(255, 255, 255, 0.06);  
  --border-light: rgba(255, 255, 255, 0.11);  
  --border-medium: rgba(255, 255, 255, 0.18);  
  
  /* Shadows */  
  --shadow-sm: 0 2px 6px rgba(0,0,0,0.35);  
  --shadow-md: 0 4px 14px rgba(0,0,0,0.40);  
  --shadow-lg: 0 8px 28px rgba(0,0,0,0.50);  
  --shadow-lg: 0 8px 28px rgba(0,0,0,0.50);  
  --shadow-glow: 0 0 24px rgba(201, 162, 39, 0.18);  
  
  /* Glass */  
  --glass-bg: rgba(10, 10, 10, 0.88);  
  --glass-border: rgba(255, 255, 255, 0.07);  
  
  /* Radii */  
  --radius-sm: 6px;  
  --radius-md: 10px;  
  --radius-lg: 14px;  
  --radius-xl: 20px;  
  --radius-full: 9999px;
```

```

/* Motion */
--ease-out:    cubic-bezier(0.16, 1, 0.3, 1);
--ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
--transition:  0.22s var(--ease-out);

/* Font */
--font-sans: 'Plus Jakarta Sans', system-ui, -apple-system, sans-serif;
--header-h:  68px;
}

```

UI/UX rationale: A single gold accent (#c9a227) on a near-black background creates a luxury e-commerce feel. All interactive elements share the `--transition` timing function — a custom cubic-bezier (overshoot-free) that feels snappy but not jarring. The glass header uses `backdrop-filter: blur(12px)` with `--glass-bg` for a frosted effect that reveals page content scrolling underneath, giving a modern macOS-inspired depth hierarchy.

3.2 Light Theme Override

The same token system switches via attribute selector — every component automatically re-colours:

```

[data-bs-theme="light"] {
  --bg-primary:    #f7f7f8;
  --bg-secondary:  #ffffff;
  --bg-tertiary:   #f0f0f2;
  --text-primary:  #111111;
  --text-secondary: #4b4b4b;
  --text-muted:    #9a9a9a;
  --border-subtle: rgba(0, 0, 0, 0.06);
  --glass-bg:      rgba(255, 255, 255, 0.92);
  --glass-border:  rgba(0, 0, 0, 0.06);
  /* shadow-opacity reduced for light mode */
  --shadow-md:     0 4px 14px rgba(0,0,0,0.09);
}

```

FOUC Prevention: The theme cookie is read server-side in `_Layout.cshtml` and the `data-bs-theme` attribute is set *before* the HTML is streamed:

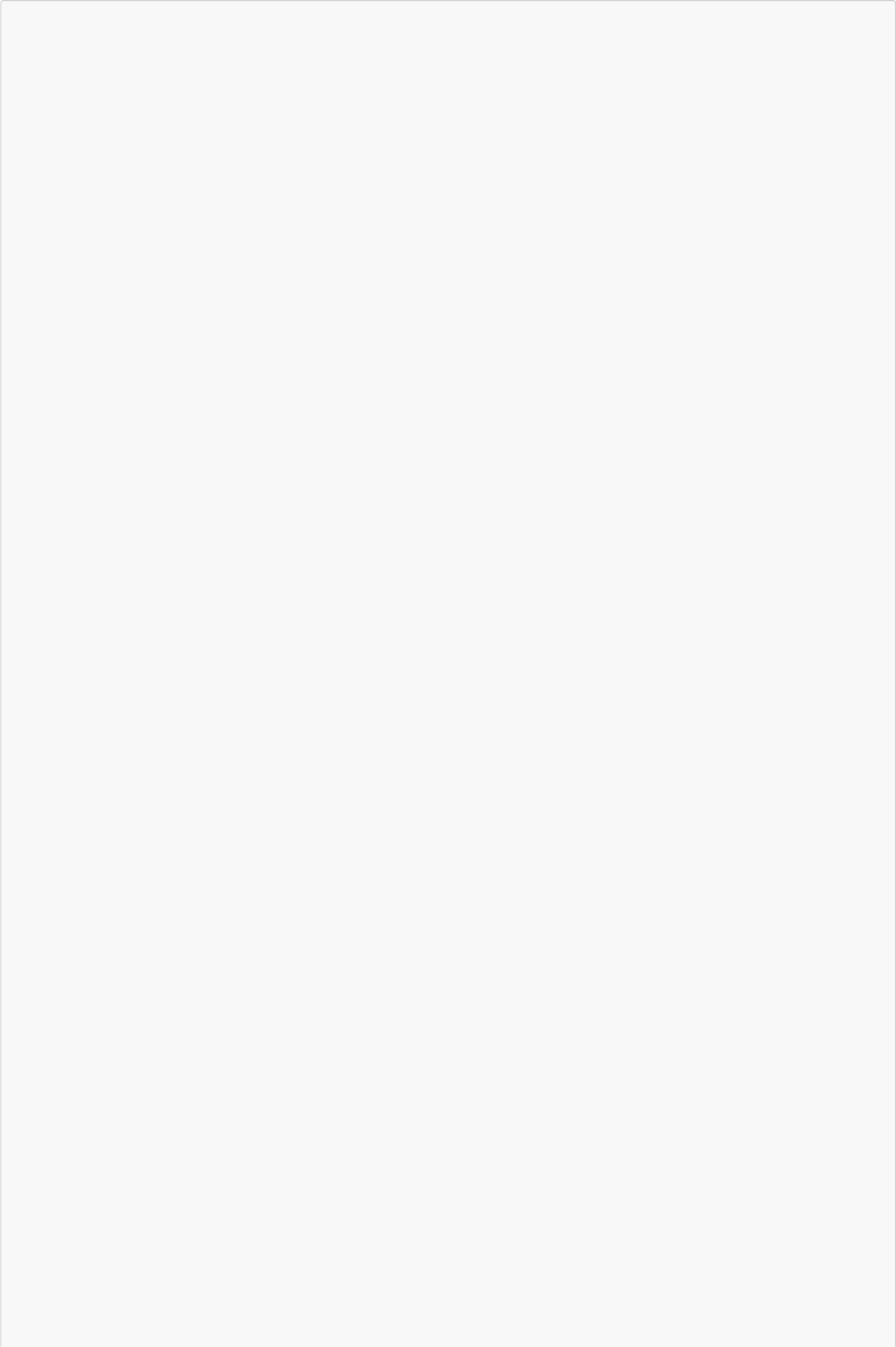
```

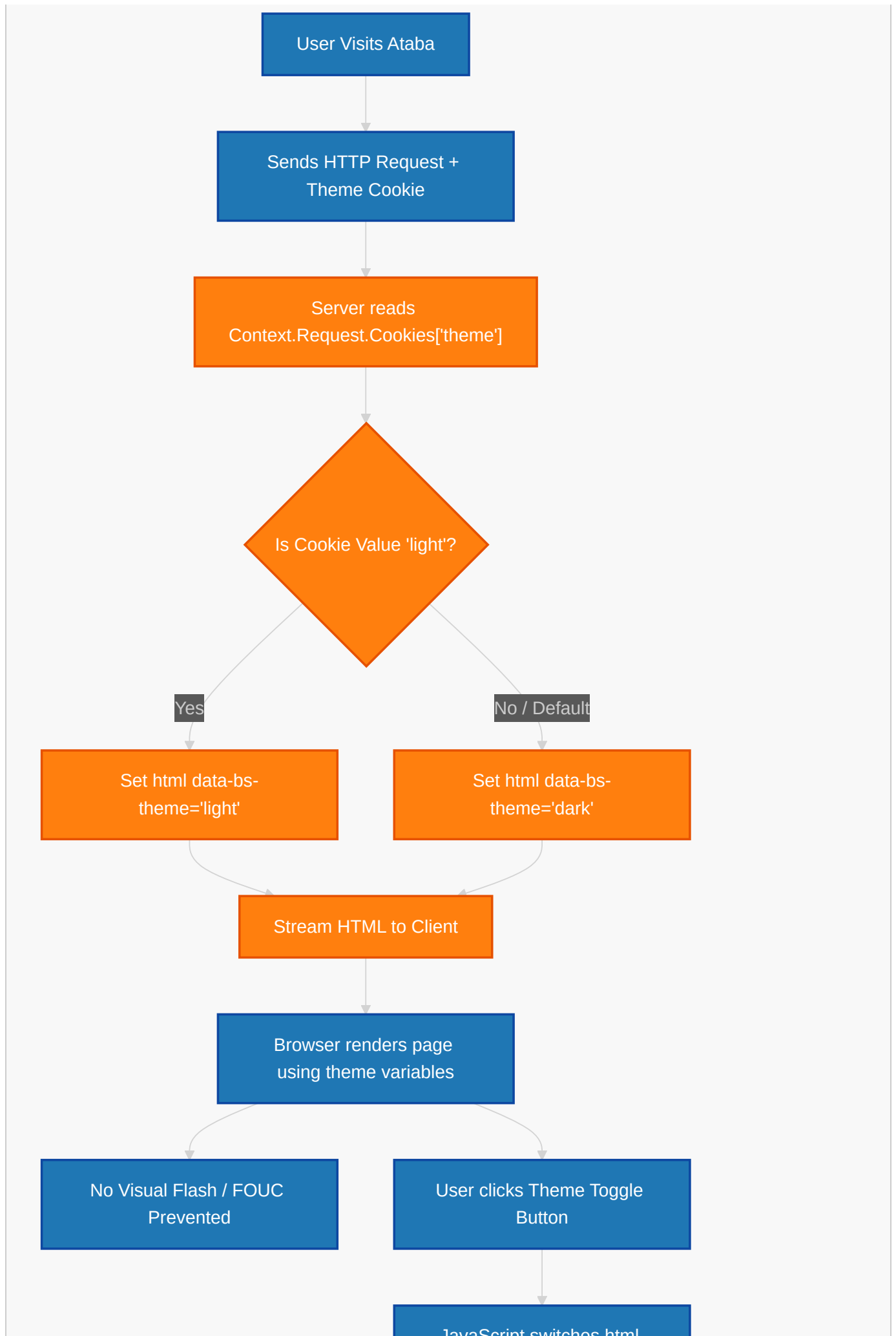
@{
  var themeCookie = Context.Request.Cookies["theme"];
  var activeTheme = (themeCookie == "light") ? "light" : "dark";
}
<html lang="en" data-bs-theme="@activeTheme">

```

This eliminates the flash of unstyled content that would occur if JavaScript set the theme after page load.

The flowchart below illustrates the detailed logic flow of the FOUC prevention and theme selection process:





JavaScript switches HTML
data-bs-theme and
updates theme Cookie

3.3 Layout Architecture

3.3.1 Global Wrapper

`_Layout.cshtml` defines the shell — header, main content area (with TempData alerts), footer, and the global AI Assistant modal. The `<main>` element uses `padding-top: calc(var(--header-h) + var(--space-xl))` to offset the fixed header:

```
main {
  min-height: calc(100vh - 200px);
  padding-top: calc(var(--header-h) + var(--space-xl));
  padding-bottom: var(--space-2xl);
}
```

3.3.2 Glass Header

Fixed-position, full-width, with backdrop blur for the frosted-glass effect:

```
.header {
  position: fixed;
  top: 0; left: 0; right: 0;
  z-index: 1000;
  background: var(--glass-bg);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border-bottom: 1px solid var(--glass-border);
  transition: background-color var(--transition), border-color var(--transition), box-shadow var(--transition);
}
.header.scrolled { box-shadow: var(--shadow-md); }
```

The `.scrolled` class is toggled by JS on `scroll > 8px`:

```
(function () {
  const header = document.querySelector('.header');
  if (!header) return;
  const onScroll = () => header.classList.toggle('scrolled', window.scrollY > 8);
  window.addEventListener('scroll', onScroll, { passive: true });
  onScroll();
})();
```

3.3.3 Navigation Layout

The header uses flexbox with three zones: logo (left), nav links + search (center), actions (right):

```
.header-inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 14px 0;
  gap: var(--space-lg);
}
```

Categories Dropdown — Pure CSS dropdown triggered by hover (`.dropdown:hover .dropdown-menu`):

```
.dropdown-menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  background: var(--bg-secondary);
  border: 1px solid var(--border-light);
  border-radius: var(--radius-md);
  box-shadow: var(--shadow-lg);
  padding: var(--space-xs);
  min-width: 200px;
  z-index: 1001;
  opacity: 0;
  visibility: hidden;
  transform: translate3d(0, -10px, 0);
  transition: opacity var(--transition), transform var(--transition), visibility 0.22s;
}
.dropdown:hover .dropdown-menu { /* open state */ }
```

Categories are cached server-side with `IMemoryCache` (5-minute absolute, 2-minute sliding) and rendered in both desktop dropdown and mobile nav:

```
@inject IMemoryCache MemoryCache
@inject IUnitOfWork UnitOfWork
@{
    var navCategories = await MemoryCache.GetOrCreateAsync("NavCategories", async entry
=>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
        entry.SlidingExpiration = TimeSpan.FromMinutes(2);
        return (await UnitOfWork.Categories.GetAsync(c => c.IsActive)).ToList();
    });
}
```

3.3.4 Search Bar

The search bar is a form that GET-submits to `ProductsController.Index`. It uses a pill-shaped input with an absolutely-positioned search icon and arrow button:

```
<form asp-controller="Products" asp-action="Index" method="get" class="nav-search d-
none d-lg-flex">
  <div class="search-wrapper">
    <i class="bi bi-search search-icon"></i>
    <input type="text" name="searchTerm" class="form-control search-input">
```

```

        placeholder="Search products..."
value="@Context.Request.Query["searchTerm"]">
    <button type="submit" class="search-btn" aria-label="Search">
        <i class="bi bi-arrow-right"></i>
    </button>
</div>
</form>

```

```

.search-input {
  padding: 0 38px 0 36px !important;
  height: 38px;
  font-size: 0.85rem;
  background: var(--bg-tertiary);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-full);
  color: var(--text-primary);
  line-height: 38px;
  transition: border-color var(--transition), box-shadow var(--transition), background-color var(--transition);
}
.search-input:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-muted);
  background: var(--bg-secondary);
}

```

3.3.5 Mobile Menu

Button toggles `.show` on the `.mobile-nav` element. Uses `max-height` animation for the expand/collapse (choreographed with opacity and visibility):

```

.mobile-nav {
  max-height: 0;
  opacity: 0;
  overflow: hidden;
  visibility: hidden;
  transition: max-height 0.38s var(--ease-out), opacity 0.25s var(--ease-out), visibility 0s 0.38s;
  border-top: 1px solid var(--border-subtle);
}
.mobile-nav.show {
  max-height: 520px;
  opacity: 1;
  visibility: visible;
  transition: max-height 0.38s var(--ease-out), opacity 0.25s var(--ease-out), visibility 0s;
}

```

The toggle JS swaps the hamburger/close icon:

```

function toggleMobileMenu() {
  const nav = document.getElementById('mobile-nav');
  const btn = document.getElementById('mobile-menu-btn');
}

```

```
const isOpen = nav.classList.toggle('show');
const icon = btn.querySelector('i');
icon.className = isOpen ? 'bi bi-x' : 'bi bi-list';
btn.setAttribute('aria-expanded', isOpen);
}
```

3.3.6 Footer

Four-column CSS grid with `2fr repeat(3, 1fr)` — brand description spans 2 fractions, then three link columns. Collapses to 2 columns at 1024px, single column at 480px:

```
.footer-grid {
  display: grid;
  grid-template-columns: 2fr repeat(3, 1fr);
  gap: var(--space-2xl);
  margin-bottom: var(--space-2xl);
}
@media (max-width: 1024px) { .footer-grid { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 768px) {
  .footer-grid { grid-template-columns: 1fr 1fr; }
  .footer-brand { grid-column: 1 / -1; }
  .footer-bottom { flex-direction: column; text-align: center; }
}
@media (max-width: 480px) { .footer-grid { grid-template-columns: 1fr; } }
```

3.4 Home Page (Views/Home/Index.cshtml)

3.4.1 Hero Section

Two-column grid (text | visual ring) that collapses to single column on mobile. The background uses layered radial gradients and a dot-grid overlay (pseudo-elements) for a subtle tech-luxury texture:

```
.hero-background {
  position: absolute;
  inset: 0;
  background: linear-gradient(150deg, var(--bg-primary) 0%, var(--bg-secondary) 55%,
var(--bg-tertiary) 100%);
  z-index: -1;
}
.hero-background::before {
  content: '';
  position: absolute;
  inset: 0;
  background:
    radial-gradient(ellipse 80% 60% at 20% 60%, rgba(201, 162, 39, 0.10) 0%,
transparent 60%),
    radial-gradient(ellipse 50% 40% at 80% 30%, rgba(201, 162, 39, 0.06) 0%,
transparent 50%);
}
.hero-background::after {
  content: '';
  position: absolute;
  inset: 0;
  background-image: radial-gradient(circle 1px at center, rgba(201,162,39,0.18) 1px,
transparent 0);
  background-size: 48px 48px;
}
```

```
    opacity: 0.25;
  }
```

The visual ring uses `pulse-glow` animation and a radial gradient to create a glowing orb:

```
.hero-visual-ring {
  width: clamp(200px, 22vw, 300px);
  height: clamp(200px, 22vw, 300px);
  border-radius: 50%;
  background: radial-gradient(circle, var(--accent-muted) 0%, transparent 70%);
  border: 1px solid rgba(201, 162, 39, 0.2);
  animation: pulse-glow 4s ease-in-out infinite;
}
```

The hero heading uses `clamp()` for fluid typography and a gradient text fill:

```
.hero h1 {
  font-size: clamp(2.5rem, 5vw, 3.75rem);
  font-weight: 800;
  line-height: 1.08;
  background: linear-gradient(135deg, var(--text-primary) 0%, var(--text-secondary) 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}
```

3.4.2 Feature Cards

Three-column grid of cards with icon, title, and description. The icon container uses `--accent-muted` background that transitions to full `--accent` on hover:

```
<div class="feature-card animate-fade-in-up stagger-1">
  <div class="feature-card-icon">
    <i class="bi bi-truck" aria-hidden="true"></i>
  </div>
  <h5>Free Shipping</h5>
  <p>On orders over $50</p>
</div>
```

```
.feature-card-icon {
  width: 56px; height: 56px;
  display: flex; align-items: center; justify-content: center;
  margin: 0 auto var(--space-md);
  background: var(--accent-muted);
  border-radius: var(--radius-lg);
  font-size: 1.5rem;
  color: var(--accent);
  transition: background-color var(--transition), transform var(--transition);
}
.feature-card:hover .feature-card-icon {
  background: var(--accent);
}
```

```

    color: #000;
    transform: scale(1.06);
}

```

3.4.3 Category Grid

Uses `grid-auto-200` (auto-fill with 160px minimum) with a cycled icon set:

```

@{
    string[] catIcons = { "bi-bag-heart", "bi-headphones", "bi-shoe", "bi-lamp", "bi-
controller", "bi-camera" };
}
...
@for (int i = 0; i < Math.Min(categories.Count, 6); i++)
{
    var icon = catIcons[i % catIcons.Length];
    <a asp-controller="Products" asp-action="ByCategory" asp-route-
id="@categories[i].Id"
    class="category-card animate-fade-in-up stagger-@(i + 1)"
    aria-label="Browse @categories[i].Name products">
        <div class="category-card-icon">
            <i class="bi @icon" aria-hidden="true"></i>
        </div>
        <span>@categories[i].Name</span>
    </a>
}

```

3.4.4 Featured Products Grid

Renders `product-card` articles in an `auto-fill` grid. Each card includes an AI assistant button that opens the global modal via `openAIModal()`:

```

<button class="ai-card-btn"
    onclick="event.preventDefault();
openAIModal('@Html.Raw(product.Name.Replace("'", "\"\"'))',
'@Html.Raw(product.Description?.Replace("'", "\"\"").Replace("\n", " ").Replace("\r",
""))')"
    title="Ask AI about this product"
    aria-label="Ask AI about @product.Name">
    <i class="bi bi-sparkles"></i>
</button>

```

The `Replace("'", "\"")` calls escape single quotes to prevent JS injection from product names containing apostrophes.

3.4.5 Newsletter Section

Gold gradient card with a CTA form:

```

.newsletter-card {
    background: linear-gradient(135deg, var(--accent) 0%, #9e7d1a 100%);
    border-radius: var(--radius-xl);
    position: relative;
}
.newsletter-card::before {

```

```

content: '';
position: absolute;
inset: 0;
background:
    radial-gradient(ellipse 60% 80% at 80% 50%, rgba(255,255,255,0.08) 0%, transparent
60%),
    radial-gradient(ellipse 40% 60% at 10% 20%, rgba(255,255,255,0.05) 0%, transparent
50%);
pointer-events: none;
}

```

3.5 Product Listing Page (Views/Products/Index.cshtml)

3.5.1 Filter Form

A card containing a horizontal filter form with search, category dropdown, price range (min/max), sort order, and action buttons:

```

<form asp-action="Index" method="get" class="filter-form" id="filterForm">
    <input type="hidden" name="page" id="pageInput" value="1" />

    <div class="form-group filter-search">
        <label class="form-label" for="searchInput">Search</label>
        <input type="text" id="searchInput" name="searchTerm" class="form-control"
            placeholder="Search products..." value="@searchTerm">
    </div>

    <div class="form-group filter-min-width">
        <label class="form-label" for="categorySelect">Category</label>
        <select id="categorySelect" name="categoryId" class="form-select">
            <option value="">All Categories</option>
            @foreach (var category in categories)
            {
                <option value="@category.Id" selected="@((selectedCategoryId ==
category.Id))">
                    @category.Name
                </option>
            }
        </select>
    </div>

    <div class="form-group">
        <label class="form-label">Price Range</label>
        <div class="filter-group-flex">
            <input type="number" name="minPrice" class="form-control"
                placeholder="Min" value="@minPrice" step="0.01" min="0">
            <input type="number" name="maxPrice" class="form-control"
                placeholder="Max" value="@maxPrice" step="0.01" min="0">
        </div>
    </div>

    <div class="form-group filter-min-width-sm">
        <label class="form-label" for="sortSelect">Sort By</label>
        <select id="sortSelect" name="sortBy" class="form-select">
            <option value="newest" selected="@((sortBy == "newest" || sortBy ==
null))">Newest</option>
            <option value="price_asc" selected="@((sortBy == "price_asc"))">Price: Low to
High</option>
            <option value="price_desc" selected="@((sortBy == "price_desc"))">Price: High

```

```

to Low</option>
    <option value="name_asc" selected="@ (sortBy == "name_asc")">Name: A to
Z</option>
    <option value="name_desc" selected="@ (sortBy == "name_desc")">Name: Z to
A</option>
    </select>
</div>

<div class="form-group flex gap-sm items-center">
    <button type="submit" class="btn btn-primary"><i class="bi bi-funnel" aria-
hidden="true"></i> Filter</button>
    <a asp-action="Index" class="btn btn-ghost"><i class="bi bi-x" aria-
hidden="true"></i> Clear</a>
</div>
</form>

```

3.5.2 AJAX Filtering & Pagination

The product grid is loaded asynchronously via `fetch()` to `/Products/Filter`. The JavaScript collects form data, builds query params, and fetches a partial HTML replacement. On success, it updates the container's innerHTML and pushes the new URL to `history.replaceState` for proper browser back-button support.

The flowchart below traces the complete AJAX request/response lifecycle:



```

async function loadProducts() {
    const container = document.getElementById('productGridContainer');
    const overlay = document.getElementById('loadingOverlay');
    overlay.style.display = 'flex';

    try {
        const params = getFilterParams();
        const response = await fetch('@Url.Action("Filter", "Products")?' + params, {
            headers: { 'X-Requested-With': 'XMLHttpRequest' }
        });
        const html = await response.text();
        container.innerHTML = html;
        history.replaceState(null, '', '@Url.Action("Index", "Products")?' + params);
    } catch {
        container.innerHTML = '<div class="empty-state">...</div>';
    } finally {
        overlay.style.display = 'none';
    }
}

```

Debounced inputs (400ms for search, 300ms for category/price/sort) reduce server load:

```

function debouncedLoad() {
    clearTimeout(debounceTimer);
    debounceTimer = setTimeout(() => {
        currentPage = 1;
    }, 400);
}

```



```

        loadProducts();
    }, 300);
}
document.getElementById('categorySelect').addEventListener('change', debouncedLoad);
document.querySelectorAll('input[name="minPrice"],
input[name="maxPrice"]').forEach(input => {
    input.addEventListener('input', debouncedLoad);
});
document.getElementById('searchInput').addEventListener('input', function () {
    clearTimeout(debounceTimer);
    debounceTimer = setTimeout(() => { currentPage = 1; loadProducts(); }, 400);
});

```

Loading state overlay:

```

<div id="loadingOverlay" class="loading-overlay" style="display:none;">
    <div class="spinner-border text-primary" role="status">
        <span class="visually-hidden">Loading...</span>
    </div>
</div>

```

3.5.3 Product Grid Partial (_ProductGrid.cshtml)

This partial receives a `PaginatedList<Product>` and renders either the product grid with paginated navigation or an empty state:

```

@model PaginatedList<Product>

@if (Model.Items.Any())
{
    <div class="grid grid-auto-fill product-grid">
        @foreach (var product in Model.Items)
        {
            <article class="product-card">
                <div class="image-wrapper">
                    <div class="badge-wrapper">
                        @if (product.IsFeatured) { <span class="badge badge-
danger">Featured</span> }
                    </div>
                    <a asp-action="Details" asp-route-id="@product.Id">
                        
                    </a>
                    <button class="icon-btn icon-btn-card"
                        onclick="event.preventDefault();
togglewishlist(@product.Id, this)"
                        aria-label="Toggle wishlist for @product.Name">
                        <i class="bi bi-heart"></i>
                    </button>
                    <button class="ai-card-btn"
                        onclick="event.preventDefault(); openAIModal('...')" ...>
                        <i class="bi bi-sparkles"></i>
                    </button>
                </div>
            </article>
        }
    </div>

```

```

        <div class="card-content">
            <h3 class="product-title">
                <a asp-action="Details" asp-route-id="@product.Id"
class="product-title-link">@product.Name</a>
            </h3>
            <p class="product-desc">@(product.Description?.Length > 65 ?
product.Description[..65] + "..." : product.Description)</p>
            <div class="price-stock-row">
                <span class="product-price">@product.Price.ToString("C")</span>
                <span class="badge @(product.Stock > 0 ? "badge-success" :
"badge-danger")">
                    @(product.Stock > 0 ? "In Stock" : "Sold Out")
                </span>
            </div>
        </div>
        <div class="card-actions">
            <a asp-action="Details" asp-route-id="@product.Id" class="btn btn-
secondary btn-sm flex-fill">
                <i class="bi bi-eye"></i> View
            </a>
        </div>
    </article>
}
</div>

<!-- Pagination -->
<div class="flex items-center justify-between flex-wrap gap-md mt-5">
    <p class="page-info">Showing @( (Model.PageIndex - 1) * 12 + 1) -
@Math.Min(Model.PageIndex * 12, Model.TotalCount) of @Model.TotalCount</p>
    @if (Model.TotalPages > 1)
    {
        <nav class="pagination" aria-label="Product pages">
            <button class="page-link @(Model.HasPreviousPage ? "" : "disabled")"
onclick="loadPage(@(Model.PageIndex - 1))" ...>
                <i class="bi bi-chevron-left"></i>
            </button>
            @* Smart ellipsis: show first page, last page, and pages around current
index *@
            @{
                var startPage = Math.Max(1, Model.PageIndex - 2);
                var endPage = Math.Min(Model.TotalPages, Model.PageIndex + 2);
                if (startPage > 1) {
                    <button class="page-link" onclick="loadPage(1)">1</button>
                    if (startPage > 2) { <span class="page-link disabled" aria-
hidden="true">...</span> }
                }
                for (int i = startPage; i <= endPage; i++) {
                    <button class="page-link @(i == Model.PageIndex ? "active" :
"">
                        <i>@i</i>
                        onclick="loadPage(@i)" @(i == Model.PageIndex ? "aria-
current='page'" : "")>@i</button>
                }
                if (endPage < Model.TotalPages) {
                    if (endPage < Model.TotalPages - 1) { <span class="page-link
disabled" aria-hidden="true">...</span> }
                    <button class="page-link"
onclick="loadPage(@Model.TotalPages)">@Model.TotalPages</button>
                }
            }
            <button class="page-link @(Model.HasNextPage ? "" : "disabled")"
onclick="loadPage(@(Model.PageIndex + 1))" ...>

```

```
                <i class="bi bi-chevron-right"></i>
            </button>
        </nav>
    }
</div>
}
else
{
    <div class="empty-state">
        <div class="empty-state-icon"><i class="bi bi-inbox"></i></div>
        <h4>No products found</h4>
        <p>Try adjusting your search or filter criteria</p>
        <a asp-action="Index" class="btn btn-primary">Clear All Filters</a>
    </div>
}
```

Empty state — centered container with large icon, message, and a CTA button. Used consistently across products, cart, wishlist, orders, and reviews:

```
.empty-state {
    display: flex; flex-direction: column;
    align-items: center; justify-content: center;
    padding: var(--space-4xl) var(--space-xl);
    text-align: center;
}
.empty-state-icon {
    width: 80px; height: 80px;
    display: flex; align-items: center; justify-content: center;
    background: var(--bg-tertiary); border-radius: 50%;
    margin: 0 auto var(--space-lg);
    font-size: 2.25rem; color: var(--text-muted);
}
```

3.5.4 Product Card Component Breakdown

The `.product-card` is a self-contained compound component with:

Element	Class	Purpose
Image wrapper	<code>.image-wrapper</code>	Fixed <code>aspect-ratio: 4/3</code> , overflow hidden for zoom
Badge wrapper	<code>.badge-wrapper</code>	Absolute top-left, z-index 2
Product image	—	<code>object-fit: cover, hover: scale(1.07)</code>
Wishlist button	<code>.icon-btn.icon-btn-card</code>	Glass-background button, absolute top-right
AI button	<code>.ai-card-btn</code>	Appears on hover, absolute bottom-right
Card content	<code>.card-content</code>	Flex column with title, description, price/stock
Actions	<code>.card-actions</code>	Bottom button strip

```
.product-card {
    background: var(--bg-secondary);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-lg);
    overflow: hidden;
```

```

    transition: transform var(--transition), border-color var(--transition), box-shadow
var(--transition);
    display: flex;
    flex-direction: column;
    height: 100%;
}
.product-card:hover {
    transform: translate3d(0, -5px, 0);
    border-color: var(--accent);
    box-shadow: var(--shadow-lg), 0 0 0 1px var(--accent-muted);
}
.product-card img {
    width: 100%; height: 100%;
    object-fit: cover;
    transition: transform 0.4s var(--ease-out);
}
.product-card:hover img { transform: scale(1.07); }

.product-title {
    display: -webkit-box;
    -webkit-line-clamp: 2;
    -webkit-box-orient: vertical;
    overflow: hidden;
}
.product-desc {
    display: -webkit-box;
    -webkit-line-clamp: 2;
    -webkit-box-orient: vertical;
    overflow: hidden;
}

```

AI button hover reveal:

```

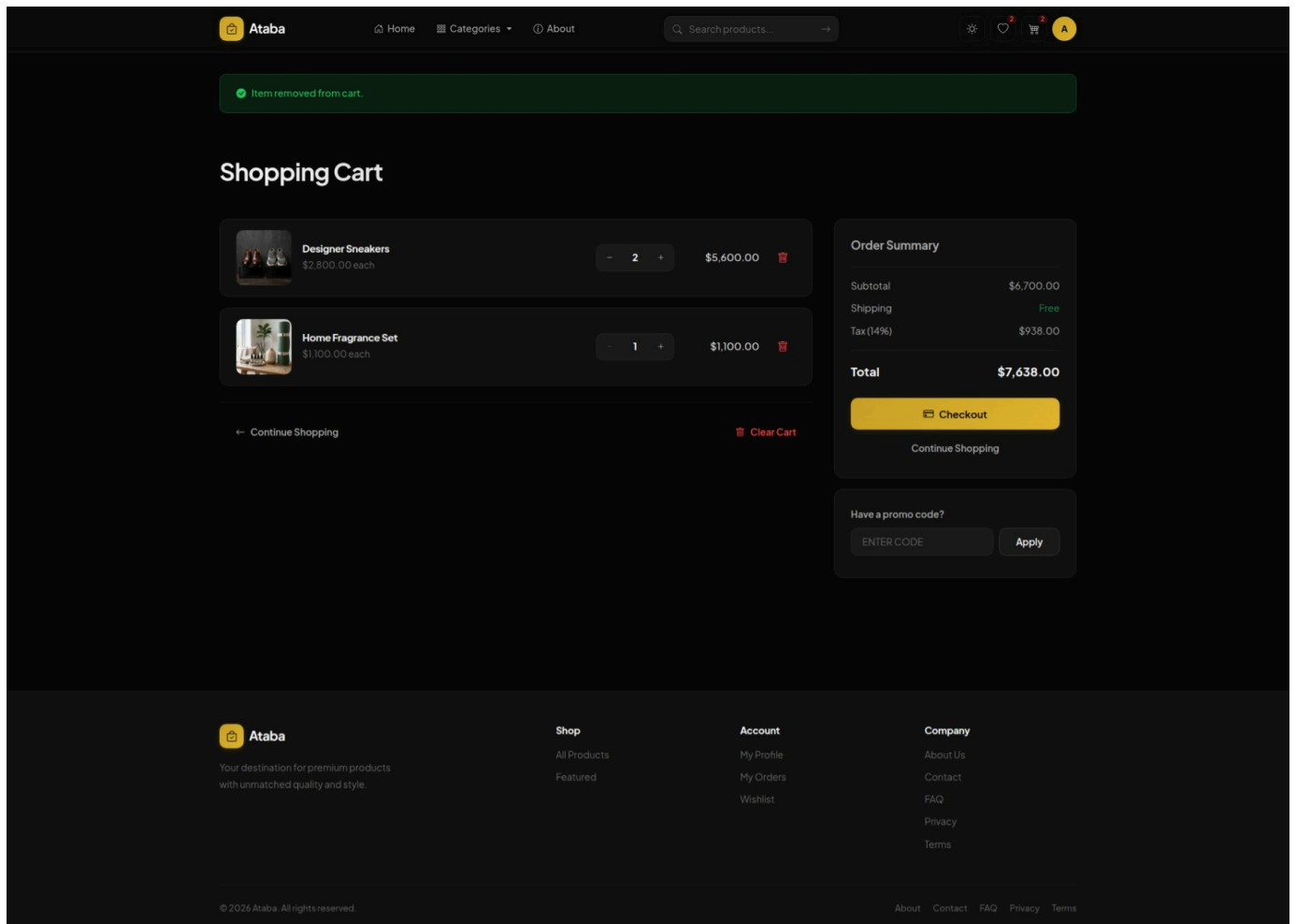
.ai-card-btn {
    position: absolute;
    bottom: var(--space-sm);
    right: var(--space-sm);
    width: 34px; height: 34px;
    opacity: 0;
    transform: scale(0.85);
    transition: opacity var(--transition), transform var(--transition);
}
.product-card:hover .ai-card-btn {
    opacity: 1;
    transform: scale(1);
}
@media (hover: none) { .ai-card-btn { opacity: 1; transform: scale(1); } }

```

The hover media query fallback ensures AI buttons are visible on touch devices where hover is not available.

3.6 Cart Page ([Views/Cart/Index.cshtml](#))

The designed shopping cart interface offers users clear summaries of selected items, quantity manipulation, and real-time calculation of taxes (14% VAT) and shipping costs, as shown in the screenshot below:



3.6.1 Cart Layout

Two-column grid (items list + summary sidebar). The summary uses `position: sticky` with `top: calc(var(--header-h) + var(--space-md))` to follow the user as they scroll:

```
.cart-layout {
  display: grid;
  grid-template-columns: 1fr 360px;
  gap: var(--space-xl);
  align-items: start;
}
.cart-summary {
  position: sticky;
  top: calc(var(--header-h) + var(--space-md));
}
@media (max-width: 1000px) {
  .cart-layout { grid-template-columns: 1fr; }
  .cart-summary { position: static; }
}
```

3.6.2 Cart Item Component

Each item is a horizontal flex row with image thumbnail, details (title, variant meta, price), quantity stepper, line total, and remove button:

```
<div class="cart-item">
  <div class="cart-item-image">
```

```

        <a asp-controller="Products" asp-action="Details" asp-route-
id="@item.Product.Id">
            
        </a>
    </div>
    <div class="cart-item-details">
        <h5 class="cart-item-title"><a ...>@item.Product.Name</a></h5>
        @if (item.Cart.ProductVariant != null)
        {
            <p class="cart-item-meta">@v.Size / @v.Color</p>
        }
        <p class="cart-item-price">@item.Product.Price.ToString("C") each</p>
    </div>
    <div class="quantity-input">
        <form asp-action="UpdateQuantity" method="post" class="d-inline">
            <input type="hidden" name="cartId" value="@item.Cart.Id" />
            <input type="hidden" name="quantity" value="@ (item.Cart.Quantity - 1)" />
            <button type="submit" class="quantity-btn" @(item.Cart.Quantity <= 1 ?
"disabled" : "")>
                <i class="bi bi-dash"></i>
            </button>
        </form>
        <span class="quantity-value">@item.Cart.Quantity</span>
        <form asp-action="UpdateQuantity" method="post" class="d-inline">
            <input type="hidden" name="cartId" value="@item.Cart.Id" />
            <input type="hidden" name="quantity" value="@ (item.Cart.Quantity + 1)" />
            <button type="submit" class="quantity-btn" @(item.Cart.Quantity >=
item.Product.Stock ? "disabled" : "")>
                <i class="bi bi-plus"></i>
            </button>
        </form>
    </div>
    <div class="cart-item-total">
        <span class="product-price">@ ((item.Product.Price *
item.Cart.Quantity).ToString("C"))</span>
    </div>
    <form asp-action="RemoveItem" method="post">
        <input type="hidden" name="cartId" value="@item.Cart.Id" />
        <button type="submit" class="icon-btn btn-text-danger" onclick="return
confirm('Remove this item from your cart?')">
            <i class="bi bi-trash"></i>
        </button>
    </form>
</div>

```

Quantity stepper styling:

```

.quantity-input {
    display: flex;
    align-items: center;
    gap: 2px;
    background: var(--bg-tertiary);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 3px;
}
.quantity-btn {
    width: 32px; height: 32px;

```

```

display: flex; align-items: center; justify-content: center;
background: transparent; border: none;
color: var(--text-secondary);
cursor: pointer;
border-radius: var(--radius-sm);
transition: background-color var(--transition), color var(--transition);
}
.quantity-btn:hover { background: var(--accent-muted); color: var(--accent); }
.quantity-btn:disabled { opacity: 0.3; cursor: not-allowed; pointer-events: none; }
.quantity-value {
width: 40px; text-align: center;
font-weight: 700; font-size: 0.9rem;
color: var(--text-primary);
}

```

3.6.3 Summary Sidebar

```

<div class="cart-summary">
  <h5 class="cart-summary-title">Order Summary</h5>
  <div class="cart-summary-row">
    <span>Subtotal</span> <span>@total.ToString("C")</span>
  </div>
  <div class="cart-summary-row">
    <span>Tax (14%)</span> <span>@tax.ToString("C")</span>
  </div>
  <div class="cart-summary-row">
    <span>Shipping</span> <span class="text-success">Free</span>
  </div>
  <div class="cart-summary-row total">
    <span>Total</span> <span>@grand.ToString("C")</span>
  </div>
  <div class="cart-actions">
    <a asp-controller="Checkout" asp-action="Index" class="btn btn-primary w-100">
      <i class="bi bi-lock"></i> Proceed to Checkout
    </a>
  </div>
</div>

```

3.7 Checkout Page (Views/Checkout/Index.cshtml)

The checkout page provides the final stage of purchase where shipping information is gathered, promotional coupon codes can be applied via AJAX, and the payment method (COD or Stripe) is selected, as illustrated below:

Shipping Information

Full Name
System Administrator

Email
admin@ecommerce.com

Phone Number
01000000000

Shipping Address
Cairo, Egypt

City
Cairo

State / Province
e.g. California

ZIP / Postal Code
e.g. 10001

Country
Egypt

Promo Code
ENTER CODE Apply

Payment Method
Select Payment

Notes (Optional)
Any special instructions?

Place Order

[Back to Cart](#)

Order Summary

Subtotal	\$6,700.00
Tax (14%)	\$938.00
Shipping	Free
Total	\$7,638.00

Ataba
Your destination for premium products with unmatched quality and style.

Shop
[All Products](#)
[Featured](#)

Account
[My Profile](#)
[My Orders](#)
[Wishlist](#)

Company
[About Us](#)
[Contact](#)
[FAQ](#)
[Privacy](#)
[Terms](#)

© 2026 Ataba. All rights reserved. [About](#) [Contact](#) [FAQ](#) [Privacy](#) [Terms](#)

Two-column layout: shipping form (left) + order summary (right, sticky). The form uses ASP.NET Core tag helpers for model binding and client-side validation:

```
<form asp-action="PlaceOrder" method="post">
  <div asp-validation-summary="All" class="alert alert-danger"></div>

  <div class="form-group">
    <label asp-for="FullName" class="form-label"></label>
    <input asp-for="FullName" class="form-control">
    <span asp-validation-for="FullName" class="text-danger"></span>
  </div>

  <div class="grid grid-2" style="gap: 1rem;">
    <div class="form-group">
      <label asp-for="Email" class="form-label"></label>
      <input asp-for="Email" class="form-control" type="email">
      <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="form-group">
      <label asp-for="PhoneNumber" class="form-label"></label>
      <input asp-for="PhoneNumber" class="form-control">
      <span asp-validation-for="PhoneNumber" class="text-danger"></span>
    </div>
  </div>
```



```

</div>

<div class="form-group">
  <label asp-for="Address" class="form-label"></label>
  <textarea asp-for="Address" class="form-control" rows="2"></textarea>
  <span asp-validation-for="Address" class="text-danger"></span>
</div>
<!-- City, State, Zip, Country in 2-column grids -->
...
<!-- Payment method selector -->
<div class="form-group">
  <label asp-for="PaymentMethod" class="form-label"></label>
  <select asp-for="PaymentMethod" class="form-select" required>
    <option value="">Select Payment</option>
    <option value="1">Cash on Delivery</option>
    <option value="2">Credit Card (Stripe)</option>
  </select>
</div>
...
</form>

```

Promo code validation — AJAX call to `CheckoutController.ValidatePromoCode`:

```

document.getElementById('applyPromoBtn').addEventListener('click', async function () {
  const code = document.getElementById('promoCodeInput').value.trim();
  const orderTotal = originalTotal - currentDiscount;

  const response = await fetch('@Url.Action("ValidatePromoCode", "Checkout")', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'RequestVerificationToken':
document.querySelector('input[name="__RequestVerificationToken"]')?.value
    },
    body: JSON.stringify({ code: code, orderTotal: orderTotal })
  });
  const data = await response.json();

  if (data.success) {
    currentDiscount = data.discountAmount;
    document.getElementById('discountRow').style.display = 'flex';
    document.getElementById('discountAmount').textContent = '-' +
data.discountAmount.toFixed(2);
    document.getElementById('totalAmount').textContent = data.newTotal.toFixed(2);
  } else {
    messageDiv.innerHTML = '<span class="text-danger">...</span>';
  }
});

```

The promo code input auto-uppercases and resets the discount display when the user modifies the field:

```

document.querySelector('input[name="PromoCode"]').addEventListener('input', function () {
  {
    this.value = this.value.toUpperCase();
    resetDiscount();
  }
});

```

3.8 AI Assistant Modal

3.8.1 Modal Structure

Defined globally in `_Layout.cshtml` — openable from any product card. Uses a `fixed` overlay with backdrop blur and a centered modal card with spring animation:

```
<div class="ai-modal-overlay" id="aiModal" onclick="if(event.target === this)
closeAIModal()">
  <div class="ai-modal" onclick="event.stopPropagation()">
    <div class="ai-modal-header">
      <div class="ai-modal-title">
        <i class="bi bi-sparkles"></i> AI Product Assistant
      </div>
      <button class="ai-modal-close" onclick="closeAIModal()"><i class="bi bi-x"></i></button>
    </div>
    <div class="ai-modal-body" id="aiResponse">
      <div class="ai-placeholder">
        <i class="bi bi-chat-dots"></i>
        <p>Ask me anything about products!</p>
      </div>
    </div>
    <div class="ai-modal-footer">
      <input type="text" class="form-control" id="aiQuestion"
        placeholder="e.g., What are the best features?"
        onkeydown="if(event.key==='Enter') askAI()">
      <button class="btn btn-primary" onclick="askAI()" id="aiAskBtn">
        <i class="bi bi-send"></i>
      </button>
    </div>
  </div>
</div>
```

3.8.2 Modal CSS

Overlay transition with spring for the modal card:

```
.ai-modal-overlay {
  position: fixed; inset: 0;
  background: rgba(0,0,0,0.65);
  backdrop-filter: blur(6px);
  z-index: 9999;
  display: flex; align-items: center; justify-content: center;
  padding: var(--space-lg);
  opacity: 0; visibility: hidden;
  transition: opacity 0.3s var(--ease-out), visibility 0.3s;
}
.ai-modal-overlay.show { opacity: 1; visibility: visible; }

.ai-modal {
  width: 100%; max-width: 500px;
  background: var(--bg-secondary);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-xl);
  overflow: hidden;
  transform: scale(0.94) translate3d(0, 22px, 0);
}
```

```

transition: transform 0.3s var(--ease-spring);
box-shadow: var(--shadow-xl);
}
.ai-modal-overlay.show .ai-modal {
  transform: scale(1) translate3d(0, 0, 0);
}

```

3.8.3 AI Fetch Logic

```

let currentProductName = '';
let currentProductDescription = '';
let aiRequestInProgress = false;

function openAIModal(productName, productDescription) {
  currentProductName = productName || '';
  currentProductDescription = productDescription || '';
  const modal = document.getElementById('aiModal');
  if (modal) {
    modal.classList.add('show');
    document.getElementById('aiQuestion')?.focus();
  }
}

function closeAIModal() { /* remove .show, reset content */ }

function askAI() {
  if (aiRequestInProgress) return;
  const question = document.getElementById('aiQuestion')?.value.trim();
  if (!question) return;

  aiRequestInProgress = true;
  const btn = document.getElementById('aiAskBtn');
  const responseDiv = document.getElementById('aiResponse');

  btn.disabled = true;
  btn.innerHTML = '<span class="ai-spinner"></span>';
  responseDiv.innerHTML = '<div class="ai-loading"><div class="ai-spinner-large">
</div><p>Thinking...</p></div>';

  fetch('/api/AIAssistant/ask', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      productName: currentProductName,
      productDescription: currentProductDescription,
      question: question
    })
  })
  .then(async res => {
    const data = await res.json();
    if (res.ok && data.success) {
      responseDiv.innerHTML = `<div class="ai-response-
text">${escapeHtml(data.response)}</div>`;
    } else {
      responseDiv.innerHTML = `<div class="ai-error"><i class="bi bi-exclamation-
triangle"></i>${escapeHtml(data.message || 'Something went wrong')}</div>`;
    }
  })
  .catch(() => {

```

```

        responseDiv.innerHTML = '<div class="ai-error"><i class="bi bi-wifi-off">
</i>Network error. Please try again.</div>';
    })
    .finally(() => {
        btn.disabled = false;
        btn.innerHTML = '<i class="bi bi-send"></i>';
        aiRequestInProgress = false;
    });
}

// XSS prevention
function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

// Close on Escape
document.addEventListener('keydown', function(e) {
    if (e.key === 'Escape') closeAIModal();
});

```

States handled: loading (spinner with "Thinking..."), success (response text), error (server error or network failure), and empty (placeholder). The `escapeHtml` function prevents XSS from AI-generated responses that might contain HTML.

3.9 Theme Toggle JavaScript

Inlined in `_Layout.cshtml` to execute immediately without waiting for external script loads:

```

function setThemeCookie(theme) {
    const maxAge = 365 * 24 * 60 * 60;
    document.cookie = `theme=${theme};max-age=${maxAge};path=/;SameSite=Lax`;
}

function toggleTheme() {
    const html = document.documentElement;
    const newTheme = html.getAttribute('data-bs-theme') === 'dark' ? 'light' : 'dark';
    html.setAttribute('data-bs-theme', newTheme);
    setThemeCookie(newTheme);
    updateThemeIcon(newTheme);
}

function updateThemeIcon(theme) {
    const btn = document.getElementById('themeToggleBtn');
    if (!btn) return;
    const icon = btn.querySelector('i');
    if (icon) icon.className = theme === 'dark' ? 'bi bi-sun' : 'bi bi-moon-stars';
}

```

The cookie uses `SameSite=Lax` (default-compatible with most browsers) and a 1-year expiry for persistence across sessions.

3.10 Toast Notification System (wwwroot/js/site.js)

A programmatic toast system that lazy-creates a container and auto-dismisses after 4.2 seconds with a slide-out animation:

```

let toastIdCounter = 0;

function showToast(message, type = 'success') {
  let container = document.getElementById('toast-container-global');
  if (!container) {
    container = document.createElement('div');
    container.id = 'toast-container-global';
    container.className = 'toast-container';
    document.body.appendChild(container);
  }

  const id = ++toastIdCounter;
  const toast = document.createElement('div');
  toast.className = `toast-notification toast-${type}`;
  toast.id = 'toast-' + id;
  toast.setAttribute('role', 'alert');
  toast.setAttribute('aria-live', 'assertive');
  toast.innerHTML = `<span>${message}</span><button class="toast-close"
onclick="dismissToast(${id})" aria-label="Dismiss">&times;</button>`;
  container.appendChild(toast);
  setTimeout(() => dismissToast(id), 4200);
}

function dismissToast(id) {
  const el = document.getElementById('toast-' + id);
  if (el) {
    el.classList.add('toast-dismissing');
    setTimeout(() => el.remove(), 320);
  }
}

```

CSS:

```

.toast-container {
  position: fixed;
  top: calc(var(--header-h) + var(--space-md));
  right: var(--space-lg);
  z-index: 99999;
  display: flex;
  flex-direction: column;
  gap: 10px;
  pointer-events: none;
}

.toast-notification {
  padding: 12px var(--space-md);
  background: var(--bg-secondary);
  border: 1px solid var(--border-light);
  border-radius: var(--radius-md);
  box-shadow: var(--shadow-lg);
  font-size: 0.9rem; font-weight: 600;
  display: flex; align-items: center; gap: var(--space-sm);
  max-width: 360px;
  pointer-events: auto;
  animation: slideInRight 0.35s var(--ease-out);
  transition: opacity 0.3s var(--ease-out), transform 0.3s var(--ease-out);
}

.toast-notification.toast-dismissing { opacity: 0; transform: translateX(110%); }

```

```
.toast-success { border-left: 3px solid var(--success); color: var(--success); }
.toast-error    { border-left: 3px solid var(--danger); color: var(--danger); }
```

The `pointer-events: none` on the container allows clicks to pass through gaps between toasts; each toast sets `pointer-events: auto` so it remains interactive.

3.11 Wishlist Toggle (Optimistic UI Update)

Located in `site.js`. The toggle immediately updates the button UI before the server responds, then reverts on failure:

```
function updateWishlistBtn(btn, isInWishlist) {
  if (!btn) return;
  const icon = btn.querySelector('i') || btn;
  if (isInWishlist) {
    btn.classList.add('btn-primary');
    btn.classList.remove('btn-secondary');
    icon.className = 'bi bi-heart-fill';
  } else {
    btn.classList.remove('btn-primary');
    btn.classList.add('btn-secondary');
    icon.className = 'bi bi-heart';
  }
}

function toggleWishlist(productId, btn) {
  const token = getToken();
  if (!token) {
    showToast('Please sign in to use the wishlist', 'error');
    return;
  }

  const isAdding = btn ? !btn.classList.contains('btn-primary') : true;
  const url = isAdding ? '/Wishlist/Add' : '/Wishlist/Remove';

  updateWishlistBtn(btn, isAdding); // Optimistic

  fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', 'RequestVerificationToken':
token },
    body: JSON.stringify({ productId })
  })
  .then(r => r.json())
  .then(data => {
    if (data.success) {
      showToast(isAdding ? 'Added to wishlist!' : 'Removed from wishlist',
'success');
    } else {
      updateWishlistBtn(btn, !isAdding); // Revert
      showToast(data.message || 'Something went wrong', 'error');
    }
  })
  .catch(() => {
    updateWishlistBtn(btn, !isAdding); // Revert on network error
    showToast('Network error', 'error');
  });
}
```

The CSRF token is extracted via:

```
function getToken() {
  return document.querySelector('input[name="__RequestVerificationToken"]')?.value;
}
```

An anti-forgery form is rendered on the products page for this purpose:

```
<form id="antiforgery-form" method="post" class="d-none">
  @Html.AntiForgeryToken()
</form>
```

3.12 Animation System

3.12.1 Keyframes

Four reusable animations — `fadeInUp` (primary entrance), `fadeIn`, `slideInRight` (toast entrance), `slideDown` (dropdown), `spin` (spinner), `skeletonPulse`, `shimmer`, `pulse-glow`:

```
@keyframes fadeInUp {
  from { opacity: 0; transform: translate3d(0, 22px, 0); }
  to   { opacity: 1; transform: translate3d(0, 0, 0); }
}
@keyframes pulse-glow {
  0%,100% { box-shadow: 0 0 0 0 var(--accent-glow); }
  50%     { box-shadow: 0 0 0 8px transparent; }
}
@keyframes skeletonPulse {
  0%,100% { background-color: var(--bg-tertiary); }
  50%     { background-color: var(--bg-elevated); }
}
```

3.12.2 Staggered Entrance

```
.animate-fade-in-up {
  animation: fadeInUp 0.55s var(--ease-out) forwards;
  opacity: 0;
}
.stagger-1 { animation-delay: 60ms; }
.stagger-2 { animation-delay: 120ms; }
.stagger-3 { animation-delay: 180ms; }
.stagger-4 { animation-delay: 240ms; }
.stagger-5 { animation-delay: 300ms; }
.stagger-6 { animation-delay: 360ms; }
```

3.12.3 Reduced Motion

All animations respect the user's OS-level motion preference:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
```

```

    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
  html { scroll-behavior: auto; }
}

```

3.13 Form Controls & Semantic Colours

3.13.1 Input Fields

Dark theme inputs use `--bg-tertiary` background with subtle border. On focus they gain gold border and a box-shadow ring:

```

.form-control, .form-select {
  width: 100%;
  padding: 9px var(--space-md);
  font-family: inherit;
  font-size: 0.9rem;
  background: var(--bg-tertiary);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-md);
  color: var(--text-primary);
  transition: border-color var(--transition), box-shadow var(--transition), background-color var(--transition);
}
.form-control:focus, .form-select:focus {
  outline: none;
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-muted);
  background: var(--bg-secondary);
}

```

3.13.2 Semantic Colours

Badges, alerts, and status indicators use muted-colour backgrounds with matching text, all driven by the same token variables:

```

.badge-success { background: rgba(34,197,94,0.12); color: #22c55e; }
.badge-danger  { background: rgba(239,68,68,0.12); color: #ef4444; }

.alert-success { background: rgba(34,197,94,0.12); color: #22c55e; border: 1px solid rgba(34,197,94,0.25); }
.alert-danger  { background: rgba(239,68,68,0.12); color: #ef4444; border: 1px solid rgba(239,68,68,0.25); }

```

Status badges for order tracking — each status has light and dark mode variants:

```

.status-paid      { background: rgba(56,189,248,0.15); color: #38bdf8; }
.status-processing { background: rgba(99,102,241,0.15); color: #6366f1; }
.status-shipped   { background: rgba(100,116,139,0.15); color: #64748b; }
.status-delivered { background: rgba(34,197,94,0.15); color: #22c55e; }
.status-cancelled { background: rgba(239,68,68,0.15); color: #ef4444; }

[data-bs-theme="dark"] .status-paid      { background: rgba(56,189,248,0.22); color:

```



```
#7dd3fc; }
[data-bs-theme="dark"] .status-delivered { background: rgba(34,197,94,0.22); color:
#4ade80; }
/* etc */
```

3.14 Admin Dashboard Styles

3.14.1 Stat Cards

Gradient background cards with icon, label, value, and optional link:

```
.stat-card-indigo { background: linear-gradient(135deg, #6366f1, #818cf8); }
.stat-card-green { background: linear-gradient(135deg, #22c55e, #4ade80); }
.stat-card-yellow { background: linear-gradient(135deg, #eab308, #fde047); }
/* 8 colour variants total */

.stat-card .card-body { padding: 1.25rem; }
.stat-card-label { color: rgba(255,255,255,0.7); font-size: 0.8rem; }
.stat-card-value { color: #fff; font-weight: 700; font-size: 1.5rem; }
.stat-card-icon { width: 36px; height: 36px; background: rgba(255,255,255,0.15);
border-radius: 8px; }
```

Dark-on-light variants use `.stat-card-dark` which inverts the text colours to black:

```
.stat-card-dark .stat-card-label { color: rgba(0,0,0,0.6); }
.stat-card-dark .stat-card-value { color: #000; }
```

3.14.2 Dashboard Chart Grid

```
.chart-grid { display: grid; grid-template-columns: 1fr 1.5fr; gap: var(--space-lg); }
@media (max-width: 768px) { .chart-grid { grid-template-columns: 1fr; } }
```

3.14.3 Dashboard Table

Compact table with uppercase header labels:

```
.table-dashboard { font-size: 0.85rem; border: none; margin-bottom: 0; }
.table-dashboard thead th {
  border-bottom: 1px solid var(--border-subtle);
  font-weight: 700; text-transform: uppercase;
  font-size: 0.7rem; letter-spacing: 0.05em;
  color: var(--text-muted);
}
```

3.15 Grid System

A minimal utility grid system using CSS Grid — no framework dependency:

```
.grid { display: grid; gap: var(--space-lg); }
.grid-2 { grid-template-columns: repeat(2, 1fr); }
.grid-3 { grid-template-columns: repeat(3, 1fr); }
.grid-4 { grid-template-columns: repeat(4, 1fr); }
.grid-auto-fill { grid-template-columns: repeat(auto-fill, minmax(260px, 1fr)); }
.grid-auto-200 { grid-template-columns: repeat(auto-fill, minmax(160px, 1fr)); }
```

Responsive overrides collapse columns at breakpoints:

```
@media (max-width: 1100px) { .grid-4 { grid-template-columns: repeat(3, 1fr); } }
@media (max-width: 768px) { .grid-3, .grid-4 { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 520px) { .grid-2, .grid-3, .grid-4, .grid-auto-fill { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 360px) { .grid-2, .grid-3, .grid-4, .grid-auto-fill { grid-template-columns: 1fr; } }
```

3.16 Miscellaneous Components

3.16.1 Page Header

```
.page-header-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: var(--space-sm);
}
.page-title {
  font-size: clamp(1.75rem, 3.5vw, 2.25rem);
  font-weight: 800;
  background: linear-gradient(135deg, var(--text-primary) 0%, var(--text-secondary) 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}
```

3.16.2 Variant Selection (Product Detail)

Radio-button driven selection with checked-state styling via `:has()`:

```
.variant-option {
  display: block;
  cursor: pointer;
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-md);
  overflow: hidden;
  transition: border-color var(--transition), box-shadow var(--transition), background-color var(--transition);
}
.variant-option:hover { border-color: var(--accent); background: var(--accent-muted); }
```

```
.variant-option:has(input:checked) {  
  border-color: var(--accent);  
  box-shadow: 0 0 0 2px var(--accent-muted);  
  background: var(--accent-muted);  
}  
.variant-option.disabled { opacity: 0.38; cursor: not-allowed; pointer-events: none; }
```

3.16.3 Product Detail Layout

Below is the designed UI for the product detail page, showcasing variant selection, the AI product assistant trigger, and related products list:

Ataba

Home

Categories

About

Search products...

A

Home / Products / Premium Wireless Earbuds

FEATURED

Premium Wireless Earbuds

\$2,800.00

High-quality wireless earbuds with noise cancellation and premium sound.

Availability: IN STOCK (50 AVAILABLE)

Quantity

1

Add to Cart

Ask AI Assistant

★ Customer Reviews

Write a Review

Rating

★★★★★

Title (Optional)

Summarize your experience

Your Review

Share your thoughts about this product

Submit Review

No reviews yet

Be the first to share your experience with this product

Related Products

Smart Watch Pro

\$5,500.00

View

Portable Speaker

\$1,800.00

View

Ataba

Your destination for premium products with unmatched quality and style.

Shop

All Products

Featured

Account

My Profile

My Orders

Wishlist

Company

About Us

Contact

FAQ

Privacy

Terms

© 2024 Ataba. All rights reserved.

Home / Contact / FAQ / Privacy / Terms

52 / 130

```
.detail-grid { display: grid; grid-template-columns: 1fr 1fr; gap: var(--space-2xl);
align-items: start; }
.detail-image-wrap { position: sticky; top: calc(var(--header-h) + var(--space-md)); }
@media (max-width: 1024px) {
  .detail-grid { grid-template-columns: 1fr; gap: var(--space-lg); }
  .detail-image-wrap { position: static; }
}
```

3.16.4 Review System

```
.review-summary-grid {
  display: grid;
  grid-template-columns: 260px 1fr;
  gap: var(--space-xl);
  margin-bottom: var(--space-xl);
}
@media (max-width: 1024px) { .review-summary-grid { grid-template-columns: 1fr; } }

.review-bar-fill {
  height: 100%;
  background: linear-gradient(90deg, var(--accent-dim) 0%, var(--accent) 100%);
  border-radius: 4px;
  transition: width 0.6s var(--ease-out);
}
```

3.16.5 Breadcrumb

```
.breadcrumb {
  display: flex; flex-wrap: wrap; gap: var(--space-xs);
  padding: var(--space-sm) 0; margin-bottom: var(--space-lg);
  font-size: 0.8375rem; color: var(--text-muted);
}
.breadcrumb-item + .breadcrumb-item::before {
  content: '/';
  margin-right: var(--space-xs); color: var(--text-muted); opacity: 0.5;
}
```

3.16.6 Custom Scrollbar

```
::-webkit-scrollbar { width: 8px; height: 8px; }
::-webkit-scrollbar-track { background: var(--bg-primary); }
::-webkit-scrollbar-thumb { background: var(--bg-elevated); border-radius: 4px; border:
2px solid var(--bg-primary); }
::-webkit-scrollbar-thumb:hover { background: var(--accent); }
::selection { background: var(--accent); color: #000; }
```

3.17 Responsive Design Strategy

The site uses four breakpoints:

Breakpoint	Width	Changes
Desktop	>1024px	Full layout: hero 2-column, detail 2-column, checkout 2-column, 4-col grids, sticky sidebars
Tablet-landscape	1024px	Grid-4 → 3 cols, hero text scales, detail/checkout → single column, sidebars become static
Tablet-portrait	768px	Hero single column (visual hidden), header nav hidden (mobile menu visible), grids → 2 cols, footer → 2 cols, hero padding reduced
Mobile	520px	Grids → 2 cols (compact cards), cart items wrap
Small mobile	360px	All grids → 1 col, container padding reduced

Key responsive behaviours:

- **Header:** Desktop navigation hides at 768px, mobile hamburger menu appears. The search bar hides below 992px.
- **Hero:** The decorative visual ring is `d-none d-lg-block` — hidden below 992px. Buttons stack vertically on mobile.
- **Cart:** Two-column layout collapses to single column at 1000px. Cart items wrap their content on 480px to accommodate the quantity stepper and remove button.
- **Checkout:** Same 2 → 1 column collapse at 1024px.
- **Product detail:** Image goes from sticky left column to top (static) at 1024px.

3.18 Accessibility Considerations

1. **ARIA attributes:** `aria-label` on icon buttons (wishlist, AI, remove, theme toggle), `aria-current="page"` on active nav links, `aria-expanded` on mobile menu toggle, `aria-live="assertive"` on toasts.
2. **Focus management:** Visible focus ring using `:focus-visible` with gold outline. Modal traps focus implicitly — Escape key closes it.
3. **Reduced motion:** All animations respect `prefers-reduced-motion: reduce` — set to near-zero duration.
4. **Screen reader text:** `.visually-hidden` class (from Bootstrap) used on spinner labels.
5. **Semantic markup:** `<nav>`, `<main>`, `<footer>`, `<article>` elements used appropriately. Headings in hierarchical order.
6. **Colour contrast:** Gold (#c9a227) on dark backgrounds. Light theme uses dark text on white. Semantic colours are tested against WCAG AA.

3.19 Script Loading Strategy

- **Critical inline JS** (theme toggle, mobile menu, AI assistant, cart/wishlist counts) is inlined at the bottom of `_Layout.cshtml` — no blocking render.
- **Non-critical JS** (jQuery, Bootstrap bundle, site.js) loaded via `<script src="...">` with `asp-append-version="true"` for cache-busting.
- **Page-specific JS** rendered via `@section Scripts` (e.g., product filtering, checkout promo code validation).
- **Validation scripts** loaded via partial `_ValidationScriptsPartial` only on pages that need them.

3.20 Utility Classes

Lightweight utility set (avoiding a utility framework dependency):

```
.text-center      { text-align: center; }
.text-muted      { color: var(--text-muted) !important; }
.text-success    { color: var(--success); }
.text-danger     { color: var(--danger); }
.text-accent     { color: var(--accent); }
.mt-0, .mt-2, .mt-3, .mt-4, .mt-5 { /* margin-top spacing */ }
.mb-0 through .mb-5 { /* margin-bottom spacing */ }
.py-4, .py-5 { /* padding-y */ }
```

```
.w-100 { width: 100%; }  
.flex, .flex-col, .items-center, .items-start, .justify-between, .justify-center,  
.gap-xs through .gap-lg, .flex-fill, .flex-shrink-0, .flex-wrap { /* flex utilities */  
}  
.d-none, .d-block, .d-inline, .d-flex { /* display */ }  
.d-md-flex, .d-md-block, .d-md-none, .d-lg-flex, .d-lg-none { /* responsive display */  
}
```

4. AI Integration — Google Gemini Product Assistant

4.1 Objective and Architectural Context

Modern e-commerce platforms require more than static product descriptions to guide purchase decisions. The AI Product Assistant addresses this by embedding a conversational layer powered by Google Gemini directly into the product browsing experience. Users can ask natural-language questions about any product — "What are the best features?", "How does this compare to similar models?", "Is this suitable for formal events?" — and receive context-aware, real-time responses without leaving the page.

The integration follows a layered architecture:

```
Browser (JS) → AIAssistantController (API, [Authorize]) → GeminiService → Google Gemini API
```

The assistant is exposed as a global modal in `_Layout.cshtml` so it is available from any page. The product context (name + description) is injected into the prompt at the moment the user opens the modal, allowing the AI to answer specific questions about that particular product.

4.2 API Configuration and Secure Key Management

4.2.1 Configuration Store

The Gemini API key and model parameters are stored in `appsettings.json` under a dedicated `Gemini` section:

```
{
  "Gemini": {
    "ApiKey": "AIzaSyD82D98r_uFoBC1YE2G68RKDl2wc4mY9FU",
    "Model": "gemini-flash-latest",
    "MaxTokens": 800,
    "Temperature": 0.7
  }
}
```

Key	Purpose
ApiKey	Google Gemini API authentication key
Model	Model identifier (e.g. <code>gemini-flash-latest</code> , <code>gemini-pro</code>)
MaxTokens	Maximum output token count to limit response size and cost
Temperature	Response creativity (0.0 = deterministic, 1.0 = most creative)

In production, the `ApiKey` should be stored in **User Secrets** (development) or **Azure Key Vault / environment variables** (production), never committed to source control. The current value in `appsettings.json` is a placeholder.

4.2.2 Program.cs Registration

The service is registered with a named `HttpClient` via the `IHttpClientFactory` pattern, which provides connection pooling, automatic retry handling, and lifetime management:


```
// Named HttpClient for Gemini (connection pooling, timeout, DNS refresh)
builder.Services.AddHttpClient("Gemini", client =>
{
    client.Timeout = TimeSpan.FromSeconds(30);
})
.ConfigurePrimaryHttpMessageHandler(() => new HttpClientHandler
{
    MaxConnectionsPerServer = 5
});

builder.Services.AddScoped<IGeminiService, GeminiService>();
```

Design decisions:

- **Named client ("Gemini")** — isolates the Gemini connection pool from other HTTP clients (Stripe, etc.), preventing head-of-line blocking.
- **30-second timeout** — LLM inference can be slow under load; 30 seconds provides a reasonable upper bound without hanging the server indefinitely.
- **MaxConnectionsPerServer = 5** — Limits concurrent connections to the Gemini API to avoid overwhelming the downstream service.
- **Scoped lifetime** — `GeminiService` is scoped per request, consistent with `IUnitOfWork`.

4.3 GeminiService Implementation

4.3.1 Interface

```
public interface IGeminiService
{
    Task<string> GetProductAssistantResponseAsync(
        string productName,
        string productDescription,
        string userQuestion);
}
```

The single method accepts the product context and the user's question, returning a plain-text response from Gemini.

4.3.2 Constructor and Configuration Injection

```
public class GeminiService : IGeminiService
{
    private readonly IHttpClientFactory _httpClientFactory;
    private readonly IConfiguration _configuration;
    private readonly ILogger<GeminiService> _logger;
    private readonly string _apiKey;
    private readonly string _model;
    private readonly int _maxTokens;
    private readonly double _temperature;

    public GeminiService(
        IHttpClientFactory httpClientFactory,
        IConfiguration configuration,
        ILogger<GeminiService> logger)
    {
        _httpClientFactory = httpClientFactory;
    }
}
```

```

        _configuration = configuration;
        _logger = logger;

        _apiKey = _configuration["Gemini:ApiKey"]
            ?? throw new InvalidOperationException("Gemini API key not configured");
        _model = _configuration["Gemini:Model"] ?? "gemini-flash-latest";
        _maxTokens = int.Parse(_configuration["Gemini:MaxTokens"] ?? "800");
        _temperature = double.Parse(_configuration["Gemini:Temperature"] ?? "0.7");
    }
}

```

All configuration values are read at construction time with sensible defaults for everything except `ApiKey`, which throws at startup if missing — failing fast rather than failing at runtime when a user asks a question.

4.3.3 Prompt Engineering

```

private string BuildPrompt(string productName, string productDescription, string
userQuestion)
{
    return $"You are a helpful product assistant for an e-commerce store.
Product Name: {productName}
Product Description: {productDescription}

Customer Question: {userQuestion}

Provide a helpful, concise, and friendly response about this product. If the question
is not related to the product, politely redirect to product-related topics.";
}

```

The prompt uses a system-level instruction to constrain the assistant to product-related topics only, preventing abuse via prompt injection. The product context (name + description) is injected dynamically so the same endpoint works for all products without hardcoding.

4.3.4 JSON Request Body Construction

Gemini's API expects a structured JSON payload. The `BuildRequestBody` method constructs this using an anonymous object serialized with `System.Text.Json` and camelCase naming:

```

private string BuildRequestBody(string prompt)
{
    var request = new
    {
        contents = new[]
        {
            new
            {
                parts = new[]
                {
                    new { text = prompt }
                }
            }
        },
        generationConfig = new
        {
            maxOutputTokens = _maxTokens,
            temperature = _temperature
        }
    }
}

```

```

    }
};

return JsonSerializer.Serialize(request, new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase
});
}

```

The resulting JSON structure sent to the API looks like this:

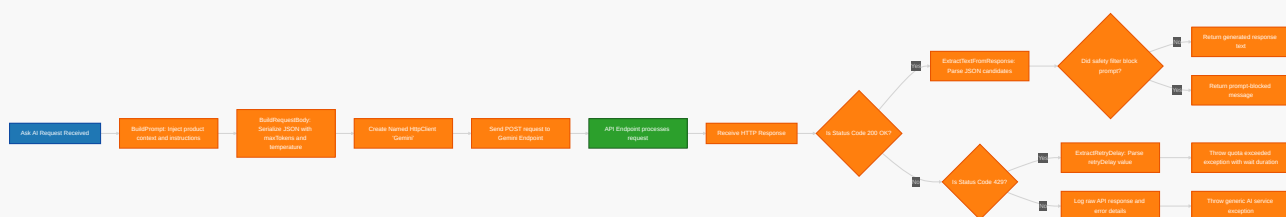
```

{
  "contents": [
    {
      "parts": [
        {
          "text": "You are a helpful product assistant...\nProduct Name: Premium Wool
Blazer\n..."
        }
      ]
    }
  ],
  "generationConfig": {
    "maxOutputTokens": 800,
    "temperature": 0.7
  }
}

```

4.3.5 API Call with Rate-Limit Handling

The flowchart below displays the sequence of steps executed by the **GeminiService** when querying the API, detailing prompt wrapping, HTTP execution, HTTP status checking, rate-limit parsing (429), and safety filtering checks:



```

public async Task<string> GetProductAssistantResponseAsync(
    string productName, string productDescription, string userQuestion)
{
    var prompt = BuildPrompt(productName, productDescription, userQuestion);
    var requestBody = BuildRequestBody(prompt);

    var client = _httpClientFactory.CreateClient("Gemini");

    var url =
        $"https://generativelanguage.googleapis.com/v1beta/models/{_model}:generateContent";

    var request = new HttpRequestMessage(HttpMethod.Post, url);
    request.Headers.Add("X-goog-api-key", _apiKey);
    request.Content = new StringContent(requestBody, Encoding.UTF8,

```

```
"application/json");

var response = await client.SendAsync(request);
var responseContent = await response.Content.ReadAsStringAsync();

if (!response.IsSuccessStatusCode)
{
    _logger.LogError("Gemini API {StatusCode}: {Error}", response.StatusCode,
responseContent);

    if ((int)response.StatusCode == 429)
    {
        var retrySeconds = ExtractRetryDelay(responseContent);
        var message = retrySeconds > 0
            ? $"AI service quota exceeded. Please try again in {retrySeconds}
seconds."
            : "AI service quota exceeded. Please try again later.";
        throw new HttpRequestException(message);
    }

    throw new HttpRequestException("AI service error. Please try again later.");
}

return ExtractTextFromResponse(responseContent);
}
```

To illustrate the user interface and how the AI Assistant is triggered in the product detail view, the screenshot below shows the interface with the sparkle button:

Ataba

HomeCategoriesAbout

Search products...

⚙️

📖

🛒

A

Home / Products / Premium Wireless Earbuds

FEATURED

Premium Wireless Earbuds

\$2,800.00

High-quality wireless earbuds with noise cancellation and premium sound.

Availability: IN STOCK (50 AVAILABLE)

Quantity

1

🛒 Add to Cart

📖

Ask AI Assistant

★ Customer Reviews

Write a Review

Rating

★★★★★

Title (Optional)

Summarize your experience

Your Review

Share your thoughts about this product

📝 Submit Review

No reviews yet

Be the first to share your experience with this product

Related Products

Smart Watch Pro

\$5,500.00

👁️ View

Portable Speaker

\$1,800.00

👁️ View

Ataba

Your destination for premium products with unmatched quality and style.

Shop

All Products

Featured

Account

My Profile

My Orders

Wishlist

Company

About Us

Contact

FAQ

Privacy

Terms

© 2024 Ataba. All rights reserved.

Home / Contact / FAQ / Privacy / Terms

61 / 130

Key design decisions:

- **API key in header** (`X-goog-api-key`) rather than query parameter — keeps the URL clean and avoids accidental key exposure in server logs.
- **429 (Rate Limit) handling** — parses the `retryDelay` field from the error response to surface a user-friendly message with the specific wait time.
- **Logging** — all failures are logged at Error level with both status code and response body for debugging.

4.3.6 Retry Delay Extraction

Google's API returns a structured error body on 429 with an optional `retryDelay` duration:

```
{
  "error": {
    "details": [
      {
        "@type": "type.googleapis.com/google.rpc.RetryInfo",
        "retryDelay": "30s"
      }
    ]
  }
}
```

```
private static int ExtractRetryDelay(string errorJson)
{
    try
    {
        using var document = JsonDocument.Parse(errorJson);
        var details = document.RootElement
            .GetProperty("error")
            .GetProperty("details")
            .EnumerateArray();

        foreach (var detail in details)
        {
            if (detail.TryGetProperty("retryDelay", out var delay))
            {
                var delayStr = delay.GetString();
                if (delayStr != null && delayStr.EndsWith("s") &&
                    int.TryParse(delayStr.TrimEnd('s'), out var seconds))
                {
                    return seconds;
                }
            }
        }
    }
    catch { /* swallow parse errors */ }

    return 0;
}
```

The method safely navigates the nested JSON, returning 0 if the `retryDelay` field is absent or unparseable. The `try/catch` prevents an error in the error-handling path from masking the original 429.

4.3.7 Response Parsing

```

private string ExtractTextFromResponse(string jsonResponse)
{
    try
    {
        using var document = JsonDocument.Parse(jsonResponse);
        var root = document.RootElement;

        if (root.TryGetProperty("candidates", out var candidates) &&
            candidates.ValueKind == JsonValueKind.Array &&
            candidates.GetArrayLength() > 0)
        {
            var firstCandidate = candidates[0];

            if (firstCandidate.TryGetProperty("content", out var content) &&
                content.TryGetProperty("parts", out var parts) &&
                parts.ValueKind == JsonValueKind.Array &&
                parts.GetArrayLength() > 0)
            {
                var fullText = new StringBuilder();

                foreach (var part in parts.EnumerateArray())
                {
                    if (part.TryGetProperty("text", out var textElement))
                    {
                        var text = textElement.GetString();
                        if (!string.IsNullOrEmpty(text))
                        {
                            fullText.Append(text);
                        }
                    }
                }

                if (fullText.Length > 0)
                {
                    return fullText.ToString();
                }
            }
        }

        // Prompt blocked by safety filters
        if (root.TryGetProperty("promptFeedback", out var feedback))
        {
            _logger.LogWarning("Gemini prompt was blocked: {Feedback}",
                feedback.GetRawText());
            return "I'm sorry, I couldn't process that request. Please try a different question.";
        }

        _logger.LogWarning("Unexpected Gemini response format: {Response}",
            jsonResponse);
        return "I'm sorry, I couldn't process that request.";
    }
    catch (JsonException ex)
    {
        _logger.LogError(ex, "Failed to parse Gemini response");
        return "I'm sorry, there was an error processing the response.";
    }
}

```

The Gemini API response structure is:

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "The Premium Wool Blazer features..." }
        ]
      }
    }
  ],
  "promptFeedback": { ... } // present only if blocked
}
```

The method handles five distinct states:

1. **Success** — extracts text from `candidates[0].content.parts[n].text`
2. **Blocked content** — `promptFeedback` present without candidates (user asked something inappropriate)
3. **Empty response** — candidates exist but have no text parts
4. **Malformed JSON** — `JsonException` caught and logged
5. **Unexpected format** — logs the raw response for debugging

4.4 Controller Integration

4.4.1 API Controller with Authorization

```
[Route("api/[controller]")]
[ApiController]
[Authorize]
public class AIAssistantController : ControllerBase
{
    private readonly IGeminiService _geminiService;
    private readonly ILogger<AIAssistantController> _logger;

    public AIAssistantController(IGeminiService geminiService,
        ILogger<AIAssistantController> logger)
    {
        _geminiService = geminiService;
        _logger = logger;
    }

    [HttpPost("ask")]
    public async Task<IActionResult> Ask([FromBody] AskRequest request)
    {
        if (string.IsNullOrEmpty(request.Question))
        {
            return BadRequest(new { success = false, message = "Question is required"
        });
        }

        try
        {
            var response = await _geminiService.GetProductAssistantResponseAsync(
                request.ProductName ?? "",
                request.ProductDescription ?? "",
                request.Question
            );
        }
    }
}
```



```

        return Ok(new { success = true, response });
    }
    catch (HttpRequestException ex)
    {
        _logger.LogError(ex, "Gemini API error");
        return StatusCode(429, new { success = false, message = ex.Message });
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "AI Assistant unexpected error");
        return StatusCode(500, new { success = false, message = "An unexpected
error occurred." });
    }
}

public class AskRequest
{
    public string ProductName { get; set; } = string.Empty;
    public string? ProductDescription { get; set; }
    public string Question { get; set; } = string.Empty;
}

```

4.4.2 Why [Authorize] is Critical

The `[Authorize]` attribute on the controller class is a deliberate security measure:

- 1. API abuse prevention.** Without authentication, anyone could call `/api/AIAssistant/ask` programmatically, burning through the Gemini API quota and incurring costs. Authentication ties each request to a valid user account, enabling rate-limiting at the user level in future iterations.
- 2. Scope limitation.** The assistant is a value-added feature for logged-in users (shoppers who have created accounts), not a public endpoint. This is consistent with the wishlist and cart features, which also require authentication.
- 3. Audit trail.** All requests are tied to an authenticated identity via the ASP.NET Core `HttpContext.User`, allowing logging and debugging.
- 4. The flow:** If an unauthenticated user clicks the AI button, the `[Authorize]` filter returns a 401, which the AJAX caller cannot use. The UI prevents this by only rendering the AI button to authenticated users, but the server-side guard is the real protection.

4.4.3 Frontend Integration

The JavaScript in `_Layout.cshtml` that calls the AI endpoint:

```

function askAI() {
    if (aiRequestInProgress) return;

    const question = document.getElementById('aiQuestion')?.value.trim();
    if (!question) return;

    aiRequestInProgress = true;
    const btn = document.getElementById('aiAskBtn');
    const responseDiv = document.getElementById('aiResponse');

    btn.disabled = true;
    btn.innerHTML = '<span class="ai-spinner"></span>';
    responseDiv.innerHTML = '<div class="ai-loading"><div class="ai-spinner-large">

```

```
</div><p>Thinking...</p></div>';

    fetch('/api/AIAssistant/ask', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        productName: currentProductName,
        productDescription: currentProductDescription,
        question: question
      })
    })
    .then(async res => {
      const data = await res.json();
      if (res.ok && data.success) {
        responseDiv.innerHTML = `<div class="ai-response-
text">${escapeHtml(data.response)}</div>`;
      } else {
        responseDiv.innerHTML = `<div class="ai-error"><i class="bi bi-exclamation-
triangle"></i>${escapeHtml(data.message || 'Something went wrong')}</div>`;
      }
    })
    .catch(() => {
      responseDiv.innerHTML = '<div class="ai-error"><i class="bi bi-wifi-off">
</i>Network error. Please try again.</div>';
    })
    .finally(() => {
      btn.disabled = false;
      btn.innerHTML = '<i class="bi bi-send"></i>';
      aiRequestInProgress = false;
    });
  }
}
```

The `askAI()` function is guarded by `aiRequestInProgress` to prevent duplicate submissions. The button is disabled during the request and shows a spinner. Three response states are rendered: success (response text), error (server message), and network failure (wifi-off icon).

4.5 Error Handling Strategy

Error Type	User Message	Log Level
429 Rate Limit	"AI service quota exceeded. Please try again in N seconds."	Error
Non-200 Status	"AI service error. Please try again later."	Error
Prompt blocked	"I'm sorry, I couldn't process that request."	Warning
Malformed response	"I'm sorry, there was an error processing the response."	Error
Network failure	"Network error. Please try again."	(handled client-side)
Unauthorized	401 returned by ASP.NET Core middleware	(handled by framework)

All server-side errors are logged with full context (status code, response body, or stack trace) while the user receives a sanitized, non-technical message appropriate for a public-facing application.

4.6 Cost and Performance Considerations

- **MaxTokens = 800** — limits output size to approximately 600 words, keeping API costs predictable.
- **Temperature = 0.7** — balances creativity with factual accuracy. Lower values (0.2–0.4) would be more deterministic but less natural.

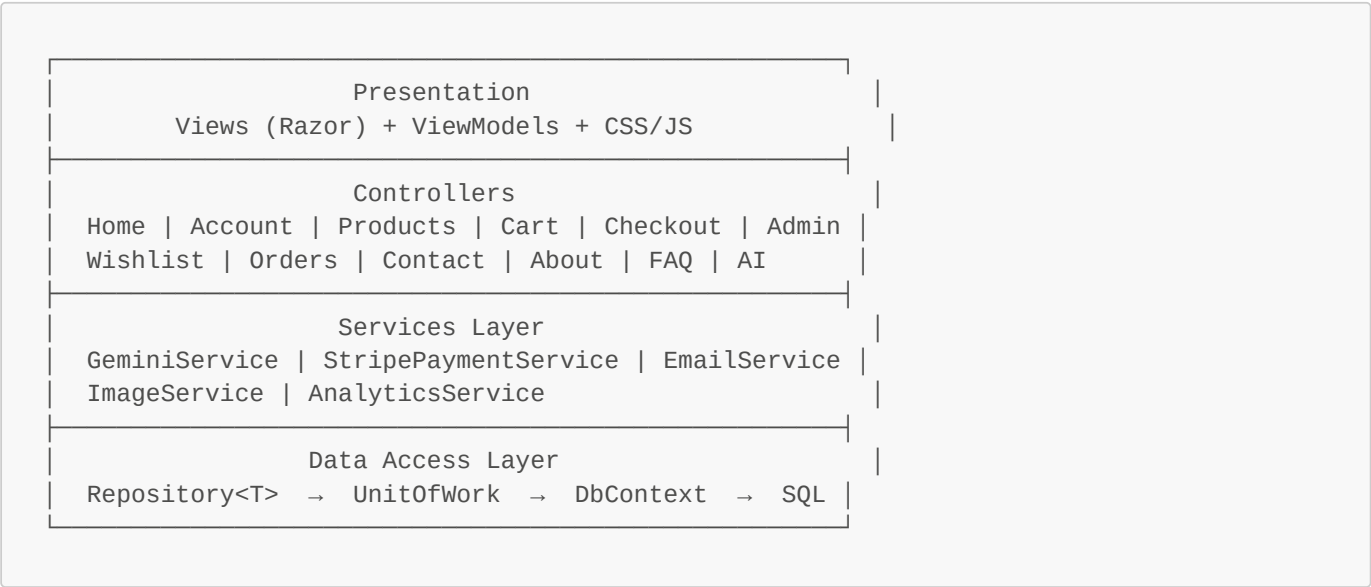
- **30-second HTTP timeout** — prevents server thread starvation if the Gemini API is slow.
- **Named HttpClient** — connection pooling reuses TCP connections, reducing latency on subsequent requests.

The Gemini Flash model (`gemini-flash-latest`) is chosen intentionally: it is Google's fastest and most cost-efficient model, suitable for real-time conversational use cases where sub-second response times are expected. The Pro model could be substituted for higher-quality responses at higher latency and cost.

5. Backend Architecture — ASP.NET Core MVC E-Commerce Engine

5.1 Project Overview and Architectural Pattern

This project follows the **Model-View-Controller (MVC)** pattern layered on top of **Entity Framework Core** with SQL Server, adopting a **Repository + Unit of Work** abstraction for the data access layer. The architecture enforces strict **Separation of Concerns** across four logical tiers:



Dependency Injection wires every layer together in `Program.cs`. Controllers never instantiate their dependencies; they receive them via constructor injection, making the system testable and loosely coupled.

5.2 Data Access Layer

5.2.1 ApplicationDbContext

The `ApplicationDbContext` extends `IdentityDbContext<ApplicationUser>` to integrate ASP.NET Core Identity tables (AspNetUsers, AspNetRoles, etc.) with the application's domain tables:

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }

    public DbSet<Category> Categories { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<ProductVariant> ProductVariants { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
    public DbSet<ShoppingCart> ShoppingCarts { get; set; }
    public DbSet<Payment> Payments { get; set; }
    public DbSet<ProductReview> ProductReviews { get; set; }
    public DbSet<PromoCode> PromoCodes { get; set; }
    public DbSet<Wishlist> Wishlists { get; set; }
}
```

Entity relationship configuration is done fluently in `OnModelCreating`. Key decisions:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // Category → Products (1:N) – restrict delete to prevent orphaned products
    modelBuilder.Entity<Category>()
        .HasMany(c => c.Products)
        .WithOne(p => p.Category)
        .HasForeignKey(p => p.CategoryId)
        .OnDelete(DeleteBehavior.Restrict);

    // Product → Variants (1:N) – cascade delete (variants are worthless without the
    // product)
    modelBuilder.Entity<Product>()
        .HasMany(p => p.ProductVariants)
        .WithOne(pv => pv.Product)
        .HasForeignKey(pv => pv.ProductId)
        .OnDelete(DeleteBehavior.Cascade);

    // Order → Payment (1:1)
    modelBuilder.Entity<Order>()
        .HasOne(o => o.Payment)
        .WithOne(p => p.Order)
        .HasForeignKey<Payment>(p => p.OrderId)
        .OnDelete(DeleteBehavior.Cascade);

    // Decimal precision for all monetary columns
    modelBuilder.Entity<Product>()
        .Property(p => p.Price).HasPrecision(18, 2);

    // PromoCode unique index
    modelBuilder.Entity<PromoCode>(entity =>
    {
        entity.HasIndex(p => p.Code).IsUnique();
    });

    // Wishlist unique constraint per user+product
    modelBuilder.Entity<Wishlist>(entity =>
    {
        entity.HasIndex(w => new { w.UserId, w.ProductId }).IsUnique();
    });
}
```

The `DeleteBehavior.Restrict` on Category → Products is intentional: it prevents accidental deletion of a category that has active products. The admin must reassign or delete the products first, enforcing data integrity at the database level.

5.2.2 Generic Repository Pattern

The `IRepository<T>` interface abstracts all CRUD operations, providing a consistent data access contract:

```
public interface IRepository<T> where T : class
{
    IQueryable<T> GetQueryable(bool asNoTracking);

    Task<IEnumerable<T>> GetAllAsync();
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter);
}
```

```

    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter, params
Expression<Func<T, object>>[] includes);
    Task<IEnumerable<T>> GetAsync(Expression<Func<T, bool>> filter, bool asNoTracking,
params Expression<Func<T, object>>[] includes);

    Task<T?> GetByIdAsync(int id);
    Task<T?> GetByIdAsync(int id, bool asNoTracking, params Expression<Func<T, object>>
[] includes);

    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter);
    Task<T?> GetFirstOrDefaultAsync(Expression<Func<T, bool>> filter, bool
asNoTracking, params Expression<Func<T, object>>[] includes);

    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
    void DeleteRange(IEnumerable<T> entities);

    Task<bool> AnyAsync(Expression<Func<T, bool>> filter);
    Task<int> CountAsync(Expression<Func<T, bool>>? filter = null);
}

```

The concrete `Repository<T>` class implements each method against EF Core:

```

public class Repository<T> : IRepository<T> where T : class
{
    private readonly ApplicationDbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repository(ApplicationDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public IQueryable<T> GetQueryable(bool asNoTracking)
    {
        return asNoTracking ? _dbSet.AsNoTracking() : _dbSet;
    }

    private IQueryable<T> ApplyIncludes(IQueryable<T> query, params Expression<Func<T,
object>>[] includes)
    {
        if (includes is { Length: > 0 })
        {
            foreach (var include in includes)
            {
                query = query.Include(include);
            }
        }
        return query;
    }

    public async Task<T?> GetByIdAsync(int id, bool asNoTracking, params
Expression<Func<T, object>>[] includes)
    {
        var query = GetQueryable(asNoTracking);
        query = ApplyIncludes(query, includes);
        return await query.FirstOrDefaultAsync(e => EF.Property<int>(e, "Id") == id);
    }
}

```

```

    }

    // ... remaining implementations follow the same pattern
}

```

AsNoTracking overload design. Every read method has an `asNoTracking` parameter (defaulting to `false` for the simpler overloads). This is deliberate:

- **Read-only GET operations** pass `asNoTracking: true` to avoid the EF Core change tracker overhead, improving performance and reducing memory usage.
- **Write operations** (add, update, delete) pass nothing (tracking on), ensuring EF Core correctly detects changes.
- The `GetQueryable` base method lets callers compose further LINQ operations (`.Where()`, `.OrderBy()`, `.Skip()`, `.Take()`) while still controlling tracking behavior.

5.2.3 Unit of Work

The `IUnitOfWork` interface exposes one repository per entity type and two transaction methods:

```

public interface IUnitOfWork : IDisposable
{
    IRepository<Category> Categories { get; }
    IRepository<Product> Products { get; }
    IRepository<ProductVariant> ProductVariants { get; }
    IRepository<Order> Orders { get; }
    IRepository<OrderItem> OrderItems { get; }
    IRepository<ShoppingCart> ShoppingCarts { get; }
    IRepository<Payment> Payments { get; }
    IRepository<ApplicationUser> Users { get; }
    IRepository<ProductReview> ProductReviews { get; }
    IRepository<PromoCode> PromoCodes { get; }
    IRepository<Wishlist> Wishlists { get; }

    Task<int> SaveAsync();
    Task<IDbContextTransaction> BeginTransactionAsync();
}

```

The concrete `UnitOfWork` class instantiates each repository with the shared `DbContext`, ensuring all operations within a single request share the same change tracker:

```

public class UnitOfWork : IUnitOfWork
{
    private readonly ApplicationDbContext _context;

    public IRepository<Category> Categories { get; private set; }
    public IRepository<Product> Products { get; private set; }
    // ... all repositories

    public UnitOfWork(ApplicationDbContext context)
    {
        _context = context;
        Categories = new Repository<Category>(_context);
        Products = new Repository<Product>(_context);
        // ... initialize all repositories
    }

    public async Task<int> SaveAsync()

```

```

    {
        return await _context.SaveChangesAsync();
    }

    public async Task<IDbContextTransaction> BeginTransactionAsync()
    {
        return await _context.Database.BeginTransactionAsync();
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}

```

Why Unit of Work? Without it, each repository would need its own `DbContext` instance, making cross-entity transactions impossible. With UoW, controllers call `SaveAsync()` once at the end of a request, and if any operation fails, the entire batch rolls back. The `BeginTransactionAsync()` method enables explicit database transactions for critical paths like checkout.

5.2.4 DI Registration in Program.cs

```

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IUnitOfWork, UnitOfWork>();

```

`AddScoped` ensures one `UnitOfWork` (and therefore one `DbContext`) per HTTP request — the standard lifetime for EF Core in web applications.

5.3 Entities and Database Schema

5.3.1 ApplicationUser (Extended Identity)

```

public class ApplicationUser : IdentityUser
{
    public string FullName { get; set; } = string.Empty;
    public string? Address { get; set; }
    public string? City { get; set; }
    public string? Country { get; set; }
    public DateTime CreatedDate { get; set; } = DateTime.Now;

    // Navigation Properties
    public virtual ICollection<Order> Orders { get; set; } = new List<Order>();
    public virtual ICollection<ShoppingCart> ShoppingCarts { get; set; } = new
    List<ShoppingCart>();
}

```

Extends the `IdentityUser` base class (which provides `Id`, `UserName`, `Email`, `PasswordHash`, etc.) with profile information needed for shipping and display. The `Orders` and `ShoppingCarts` navigation properties enable LINQ queries like `user.Orders.Where(o => o.Status == OrderStatus.Pending)`.

5.3.2 Category


```

[Key]
public int Id { get; set; }

[Required(ErrorMessage = "اسم الفئة مطلوب")]
[StringLength(100)]
public string Name { get; set; } = string.Empty;

[StringLength(500)]
public string? Description { get; set; }

public string? ImageUrl { get; set; }
public DateTime CreatedDate { get; set; } = DateTime.Now;
public bool IsActive { get; set; } = true;

public virtual ICollection<Product> Products { get; set; } = new List<Product>();

```

5.3.3 Product (with Concurrency Token)

```

public class Product
{
    public int Id { get; set; }

    [Required, StringLength(200)]
    public string Name { get; set; } = string.Empty;

    [StringLength(2000)]
    public string? Description { get; set; }

    [Required, Column(TypeName = "decimal(18,2)"), Range(0.01, double.MaxValue)]
    public decimal Price { get; set; }

    [Required, Range(0, int.MaxValue)]
    public int Stock { get; set; }

    public string? ImageUrl { get; set; }

    [Required]
    public int CategoryId { get; set; }

    public DateTime CreatedDate { get; set; } = DateTime.Now;
    public bool IsActive { get; set; } = true;
    public bool IsFeatured { get; set; } = false;

    [Timestamp]
    public byte[] RowVersion { get; set; } = [];

    // Navigation Properties
    public virtual Category Category { get; set; } = null!;
    public virtual ICollection<OrderItem> OrderItems { get; set; } = new
List<OrderItem>();
    public virtual ICollection<ShoppingCart> ShoppingCarts { get; set; } = new
List<ShoppingCart>();
    public virtual ICollection<ProductVariant> ProductVariants { get; set; } = new
List<ProductVariant>();
}

```

[Timestamp] RowVersion. This is the concurrency token that EF Core maps to SQL Server's `rowversion` type. Every time the row is updated, SQL Server automatically increments the rowversion value. When two users attempt to update the same product simultaneously, EF Core detects the mismatch between the original and current rowversion values and throws a `DbUpdateConcurrencyException`. This is critical for stock management during checkout (see §5.7).

5.3.4 ProductVariant

```
public class ProductVariant
{
    public int Id { get; set; }
    [Required]
    public int ProductId { get; set; }

    [StringLength(50)]
    public string? Size { get; set; } // S, M, L, XL

    [StringLength(50)]
    public string? Color { get; set; }

    [Column(TypeName = "decimal(18,2)")]
    public decimal AdditionalPrice { get; set; } = 0;

    public int Stock { get; set; } = 0;

    [Timestamp]
    public byte[] RowVersion { get; set; } = [];

    public bool IsActive { get; set; } = true;

    public virtual Product Product { get; set; } = null!;
}
```

Variants allow a product to have different size/color combinations, each with its own stock and price delta. The concurrency token on both `Product` and `ProductVariant` prevents race conditions when two shoppers buy the last unit of a specific size.

5.3.5 ShoppingCart

```
public class ShoppingCart
{
    public int Id { get; set; }
    [Required]
    public string UserId { get; set; } = string.Empty;
    [Required]
    public int ProductId { get; set; }
    [Required]
    public int Quantity { get; set; } = 1;
    public int? ProductVariantId { get; set; }
    public DateTime AddedDate { get; set; } = DateTime.Now;

    public virtual ApplicationUser User { get; set; } = null!;
    public virtual Product Product { get; set; } = null!;
    public virtual ProductVariant? ProductVariant { get; set; }
}
```

Composite key logic is handled in application code (not the database): a user can have at most one cart entry per (**ProductId**, **VariantId**) combination. This is enforced by the **AddToCart** method which first checks for an existing entry and increments quantity if found, rather than inserting a duplicate.

5.3.6 Order and OrderItem

```
public class Order
{
    public int Id { get; set; }
    [Required]
    public string UserId { get; set; } = string.Empty;
    public DateTime OrderDate { get; set; } = DateTime.Now;

    [Required, Column(TypeName = "decimal(18,2)")]
    public decimal TotalAmount { get; set; }

    [Required]
    public OrderStatus Status { get; set; } // enum: Pending, Paid, Processing,
    Shipped, Delivered, Cancelled, Refunded

    [Required]
    public PaymentMethod PaymentMethod { get; set; } // enum: CashOnDelivery,
    CreditCard

    [Required, StringLength(500)]
    public string ShippingAddress { get; set; } = string.Empty;
    [Required, StringLength(100)]
    public string City { get; set; } = string.Empty;
    [Required, StringLength(100)]
    public string Country { get; set; } = string.Empty;
    public string? State { get; set; }
    public string? ZipCode { get; set; }
    [Required, Phone]
    public string PhoneNumber { get; set; } = string.Empty;
    public string Notes { get; set; } = string.Empty;
    public DateTime? DeliveredDate { get; set; }

    // PromoCode
    public int? PromoCodeId { get; set; }
    [Column(TypeName = "decimal(18,2)")]
    public decimal DiscountAmount { get; set; } = 0;

    public virtual ApplicationUser? User { get; set; }
    public virtual PromoCode? PromoCode { get; set; }
    public virtual ICollection<OrderItem> OrderItems { get; set; } = new
    List<OrderItem>();
    public virtual Payment? Payment { get; set; }
}

public class OrderItem
{
    public int Id { get; set; }
    [Required]
    public int OrderId { get; set; }
    [Required]
    public int ProductId { get; set; }
    [Required]
    public int Quantity { get; set; }
    [Required, Column(TypeName = "decimal(18,2)")]
    public decimal UnitPrice { get; set; }
```

```
[Required, Column(TypeName = "decimal(18,2)")]
public decimal TotalPrice { get; set; }

public virtual Order Order { get; set; } = null!;
public virtual Product Product { get; set; } = null!;
}
```

The `UnitPrice` is stored at order time rather than computed from the current product price — this is an intentional snapshot pattern. If the admin changes a product's price later, past orders still reflect the price the customer actually paid. `TotalPrice = UnitPrice * Quantity` is precomputed for efficient analytics queries without runtime multiplication.

5.3.7 OrderStatus Enum

```
public enum OrderStatus
{
    Pending = 1,
    Paid = 2,
    Processing = 3,
    Shipped = 4,
    Delivered = 5,
    Cancelled = 6,
    Refunded = 7
}
```

The status progression is: `Pending` → `Paid` → `Processing` → `Shipped` → `Delivered`. Cancellation is possible from `Pending` or `Paid` states. The `Refunded` status terminates the lifecycle after delivery.

5.3.8 PaymentMethod Enum

```
public enum PaymentMethod
{
    CashOnDelivery = 1,
    CreditCard = 2
}
```

5.3.9 PromoCode

```
public class PromoCode
{
    public int Id { get; set; }
    [Required, StringLength(50)]
    public string Code { get; set; } = string.Empty;

    public DiscountType DiscountType { get; set; } // Percentage or FixedAmount
    [Required]
    public decimal DiscountValue { get; set; }

    public decimal? MinimumPurchase { get; set; }
    public decimal? MaximumDiscount { get; set; } // cap for percentage discounts
    public DateTime? StartDate { get; set; }
    public DateTime? EndDate { get; set; }

    public int? UsageLimit { get; set; } // global cap
    public int UsageCount { get; set; } = 0;
}
```

```
public int? UsageLimitPerUser { get; set; }

[Timestamp]
public byte[] RowVersion { get; set; } = [];

public bool IsActive { get; set; } = true;
public DateTime CreatedDate { get; set; } = DateTime.Now;

public virtual ICollection<Order> Orders { get; set; } = new List<Order>();
}
```

The **RowVersion** on **PromoCode** prevents the race condition where two users apply the same limited-use promo code simultaneously, potentially allowing usage beyond the **UsageLimit**. The checkout transaction (\$5.7) handles this with a retry loop.

5.3.10 Database Diagram (Relational Summary)

```
Users (AspNetUsers)
├── Orders (UserId FK, Restrict)
│   ├── OrderItems (OrderId FK, Cascade)
│   │   └── Products (ProductId FK, Restrict)
│   │       ├── Categories (CategoryId FK, Restrict)
│   │       └── ProductVariants (ProductId FK, Cascade)
│   ├── Payments (OrderId FK, Cascade, 1:1)
│   └── PromoCodes (PromoCodeId FK, SetNull)
└── ShoppingCarts (UserId FK, Cascade)
    └── Products (ProductId FK)
```

5.4 Business Logic — Product & Category Management (Admin)

The administration panel provides managers with direct control over catalog categories and coupon/promo code parameters. Below are the administrative screens for managing categories and promotional discount codes:

Ataba

HomeCategoriesAbout

Search products...

⚙️

🔖

🛒

A

Manage Categories

Add New Category



Accessories

Premium accessories for every occasion

3 PRODUCTS

06 Jun 2026



Fashion & Style

Trendy clothing and accessories

3 PRODUCTS

06 Jun 2026



Footwear

Elegant shoes for all occasions

3 PRODUCTS

06 Jun 2026



Lifestyle

Premium lifestyle products

3 PRODUCTS

06 Jun 2026



Luxury Watches

Premium timepieces from world-renowned brands

3 PRODUCTS

06 Jun 2026



Smart Electronics

Latest gadgets and technology

3 PRODUCTS

06 Jun 2026

Ataba

Your destination for premium products with unmatched quality and style.

Shop

All Products

Featured

Account

My Profile

My Orders

Wishlist

Company

About Us

Contact

FAQ

Privacy

Terms

© 2026 Ataba. All rights reserved.

AboutContactFAQPrivacyTerms

78 / 130

The screenshot shows the Ataba website interface. At the top, there's a navigation bar with the Ataba logo, links for Home, Categories, and About, a search bar, and user icons. A modal titled "Create New Promo Code" is open in the center. The form includes fields for Code, Description, DiscountType (with a dropdown set to Percentage), DiscountValue, MinimumPurchase, MaximumDiscount, StartDate, and EndDate (both with date pickers). There's a UsageLimit field with a note "Leave empty for unlimited usage" and a checked checkbox for "IsActive". At the bottom of the modal are "Create Promo Code" and "Cancel" buttons. The footer contains the Ataba logo, a tagline, and navigation links for Shop, Account, and Company.

5.4.1 Product CRUD with Image Upload

The `AdminController` handles all product management. The `CreateProduct` action demonstrates the pattern:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CreateProduct(Product product, IFormFile? ImageFile)
{
    ModelState.Remove("Category");
    ModelState.Remove("OrderItems");
    ModelState.Remove("ShoppingCarts");
    ModelState.Remove("ProductVariants");

    if (!ModelState.IsValid)
    {
        var categories = await _unitOfWork.Categories.GetAsync(c => c.IsActive);
        ViewBag.Categories = categories.ToList();
        return View(product);
    }

    try
    {
        if (ImageFile != null && ImageFile.Length > 0)
        {
            product.ImageUrl = await _imageService.UploadImageAsync(ImageFile,
"products");
        }

        product.CreatedDate = DateTime.Now;
    }
}
```

```

        product.IsActive = true;

        await _unitOfWork.Products.AddAsync(product);
        await _unitOfWork.SaveAsync();

        TempData["SuccessMessage"] = "Product created successfully!";
        return RedirectToAction(nameof(Products));
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "AdminController error");
        ModelState.AddModelError("", $"Error: {ex.Message}");
        // Re-populate categories for the failed form
        var categories = await _unitOfWork.Categories.GetAsync(c => c.IsActive);
        ViewBag.Categories = categories.ToList();
        return View(product);
    }
}

```

Key details:

- `ModelState.Remove(...)` strips navigation properties from validation — the Category navigation property is populated by EF after save, not from the form.
- Image upload delegates to `IImageService`; if no file is provided, `ImageUrl` remains null.
- Success/failure follows the Post-Redirect-Get (PRG) pattern using `TempData`.

5.4.2 Image Service

```

public class ImageService : IImageService
{
    private readonly IWebHostEnvironment _environment;
    private readonly ILogger<ImageService> _logger;

    public ImageService(IWebHostEnvironment environment, ILogger<ImageService> logger)
    {
        _environment = environment;
        _logger = logger;
    }

    public async Task<string> UploadImageAsync(IFormFile file, string folder)
    {
        if (file == null || file.Length == 0)
            return string.Empty;

        try
        {
            var uploadsFolder = Path.Combine(_environment.WebRootPath, "images",
folder);

            if (!Directory.Exists(uploadsFolder))
                Directory.CreateDirectory(uploadsFolder);

            var fileName = Guid.NewGuid().ToString() +
Path.GetExtension(file.FileName);
            var filePath = Path.Combine(uploadsFolder, fileName);

            using (var stream = new FileStream(filePath, FileMode.Create))
            {

```



```

        await file.CopyToAsync(stream);
    }

    return $"/images/{folder}/{fileName}";
}
catch (Exception ex)
{
    _logger.LogError($"Error uploading image: {ex.Message}");
    throw;
}
}

public async Task<bool> DeleteImageAsync(string imagePath)
{
    if (string.IsNullOrEmpty(imagePath))
        return true;

    try
    {
        var fullPath = Path.Combine(_environment.WebRootPath,
imagePath.TrimStart('/'));
        if (File.Exists(fullPath))
            File.Delete(fullPath);
        return true;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error deleting image {ImagePath}", imagePath);
        return false;
    }
}
}

```

Design decisions:

- `Guid.NewGuid()` generates a unique filename — prevents collisions and path-traversal attacks.
- The original extension is preserved for browser compatibility.
- Files are stored under `wwwroot/images/{folder}/` so they are served directly by the static file middleware without a controller action.
- Deletion silently returns `true` even if the file doesn't exist (idempotent).

5.5 Performance Optimization — In-Memory Caching

5.5.1 Registration

```

builder.Services.AddMemoryCache();
builder.Services.AddResponseCaching();

```

`IMemoryCache` is used for server-side caching of expensive, infrequently-changed data. `ResponseCaching` is separately configured for static pages like Privacy and Terms.

5.5.2 Cached Data Sources

Three data sets are cached to reduce database load:

HomePage featured products (HomeController):

```
var featuredProducts = await _memoryCache.GetOrCreateAsync("FeaturedProducts", async
entry =>
{
    entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
    entry.SlidingExpiration = TimeSpan.FromMinutes(2);
    return await _unitOfWork.Products.GetQueryable(asNoTracking: true)
        .Where(p => p.IsFeatured && p.IsActive)
        .OrderBy(p => p.Id)
        .Take(8)
        .ToListAsync();
});
```

Navigation categories (_Layout.cshtml via IMemoryCache injection):

```
@inject IMemoryCache MemoryCache
@inject IUnitOfWork UnitOfWork
@{
    var navCategories = await MemoryCache.GetOrCreateAsync("NavCategories", async entry
=>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
        entry.SlidingExpiration = TimeSpan.FromMinutes(2);
        return (await UnitOfWork.Categories.GetAsync(c => c.IsActive)).ToList();
    });
}
```

Product listing categories (ProductsController):

```
private async Task<List<Category>> GetCachedCategoriesAsync()
{
    return await _memoryCache.GetOrCreateAsync("ProductCategories", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
        entry.SlidingExpiration = TimeSpan.FromMinutes(2);
        return (await _unitOfWork.Categories.GetAsync(c => c.IsActive, asNoTracking:
true)).ToList();
    }) ?? [];
}
```

Cache eviction strategy:

- **Absolute expiration (10 minutes)** — guarantees stale data is eventually refreshed even without activity.
- **Sliding expiration (2 minutes)** — extends the cache lifetime as long as the data is being accessed frequently, preventing cache churn under load.
- Both conditions must expire: the cache entry lives until either 10 minutes from creation OR 2 minutes of inactivity, whichever expires last.

5.6 Cart Logic with Stock Validation

5.6.1 AddToCart — Server-Side Stock Check

```
[HttpPost]
[ValidateAntiForgeryToken]
```

```

public async Task<IActionResult> AddToCart(int productId, int quantity = 1, int?
variantId = null)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    if (string.IsNullOrEmpty(userId))
        return RedirectToAction("Login", "Account");

    // 1. Verify product exists and is active
    var product = await _unitOfWork.Products.GetByIdAsync(productId);
    if (product == null || !product.IsActive) { /* redirect with error */ }

    // 2. If variant selected, verify it and check its stock
    ProductVariant? selectedVariant = null;
    if (variantId.HasValue)
    {
        selectedVariant = await _unitOfWork.ProductVariants.GetFirstOrDefaultAsync(
            v => v.Id == variantId && v.ProductId == productId && v.IsActive);
        if (selectedVariant == null) { /* error */ }
        if (selectedVariant.Stock < quantity) { /* error */ }
    }

    // 3. Check base stock
    var availableStock = selectedVariant?.Stock ?? product.Stock;
    if (availableStock < quantity) { /* error */ }

    // 4. Check for existing cart entry
    var existingCartItem = await _unitOfWork.ShoppingCarts.GetFirstOrDefaultAsync(
        c => c.UserId == userId && c.ProductId == productId && c.ProductVariantId ==
variantId);

    if (existingCartItem != null)
    {
        // Increment, checking cumulative stock
        existingCartItem.Quantity += quantity;
        if (existingCartItem.Quantity > availableStock) { /* error */ }
        _unitOfWork.ShoppingCarts.Update(existingCartItem);
    }
    else
    {
        // New entry
        await _unitOfWork.ShoppingCarts.AddAsync(new ShoppingCart { ... });
    }

    await _unitOfWork.SaveAsync();
    TempData["SuccessMessage"] = $"{product.Name} added to cart successfully!";
    return RedirectToAction(nameof(Index));
}

```

Validation flow: product active → variant valid → individual stock sufficient → cumulative cart quantity within stock. Each check returns a specific error message to the user, preventing overselling at the add-to-cart stage.

5.6.2 Quantity Update with Stock Boundary

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> UpdateQuantity(int cartId, int quantity)
{
    var cartItem = await _unitOfWork.ShoppingCarts.GetByIdAsync(cartId);
    if (cartItem == null || cartItem.UserId != userId) { /* error */ }

```

```

var product = await _unitOfWork.Products.GetByIdAsync(cartItem.ProductId);
if (quantity > product.Stock) { /* error: "Only N units available" */ }

if (quantity <= 0)
{
    _unitOfWork.ShoppingCarts.Delete(cartItem); // Remove if quantity drops to zero
}
else
{
    cartItem.Quantity = quantity;
    _unitOfWork.ShoppingCarts.Update(cartItem);
}

await _unitOfWork.SaveChangesAsync();
}

```

Quantity ≤ 0 triggers deletion rather than storing a zero-quantity cart item, keeping the data clean.

5.7 Checkout, Concurrency, and Payments

5.7.1 Checkout Index — Pre-fill User Profile

```

public async Task<IActionResult> Index(string? promoCode = null)
{
    var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
    var user = await _userManager.FindByIdAsync(userId);

    var model = new CheckoutViewModel
    {
        FullName = user?.FullName ?? "",
        Email = user?.Email ?? "",
        PhoneNumber = user?.PhoneNumber ?? "",
        Address = user?.Address ?? "",
        City = user?.City ?? "",
        Country = user?.Country ?? "",
        PromoCode = promoCode
    };

    // Cart summary for display
    var cartItems = await _unitOfWork.ShoppingCarts.GetQueryable(asNoTracking: true)
        .Where(c => c.UserId == userId)
        .Include(c => c.Product)
        .ToListAsync();

    var subtotal = cartItems.Sum(item => item.Product.Price * item.Quantity);
    ViewBag.Subtotal = subtotal;
    ViewBag.Tax = subtotal * 0.14m;
    ViewBag.Total = subtotal * 1.14m;

    return View(model);
}

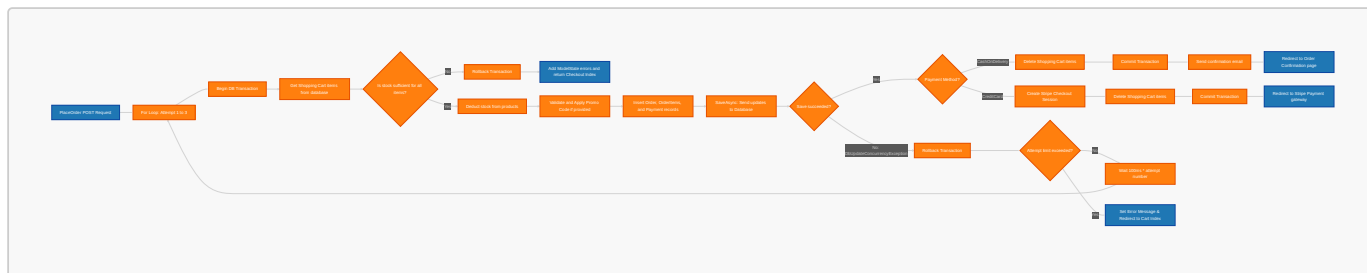
```

The form is pre-populated from the user's profile, reducing friction for returning customers.

5.7.2 PlaceOrder — The Transaction Kernel

This is the most critical method in the application. It orchestrates: stock deduction, promo code application, order creation, payment record creation, and cart cleanup — all within a single database transaction with concurrency conflict retry.

The flowchart below traces the concurrency-safe transactional checkout workflow, highlighting the database transaction boundaries and retry mechanics:



```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> PlaceOrder(CheckoutViewModel model)
{
    const int maxRetries = 3;

    for (int attempt = 1; attempt <= maxRetries; attempt++)
    {
        await using var transaction = await _unitOfWork.BeginTransactionAsync();

        try
        {
            // --- Step 1: Load cart ---
            var cartItems = await _unitOfWork.ShoppingCarts.GetAsync(
                c => c.UserId == userId, c => c.Product);
            if (!cartItems.Any()) { /* redirect: cart empty */ }

            var cartItemList = cartItems.ToList();

            // --- Step 2: Aggregate ALL stock failures before processing ---
            var outOfStockItems = cartItemList
                .Where(ci => ci.Product == null || !ci.Product.IsActive ||
                    ci.Product.Stock < ci.Quantity)
                .Select(ci => ci.Product?.Name ?? "Unknown")
                .ToList();

            if (outOfStockItems.Any())
            {
                await transaction.RollbackAsync();
                foreach (var item in outOfStockItems)
                    ModelState.AddModelError(string.Empty, $"{item}\n stock
insufficient.");
                return View("Index", model);
            }

            // --- Step 3: Deduct stock and build OrderItems ---
            decimal subtotal = 0;
            var orderItems = new List<OrderItem>();

            foreach (var cartItem in cartItemList)
            {
                var product = cartItem.Product;
                if (product == null || !product.IsActive) continue;

                var itemTotal = product.Price * cartItem.Quantity;

```

```

        subtotal += itemTotal;

        orderItems.Add(new OrderItem
        {
            ProductId = product.Id,
            Quantity = cartItem.Quantity,
            UnitPrice = product.Price,
            TotalPrice = itemTotal
        });

        product.Stock -= cartItem.Quantity; // Deduct stock
        _unitOfWork.Products.Update(product); // Triggers concurrency token
check on SaveAsync
    }

    // --- Step 4: Calculate totals ---
    decimal tax = subtotal * 0.14m;
    decimal totalAmount = subtotal + tax;

    // --- Step 5: Apply promo code (if provided) ---
    int? promoCodeId = null;
    decimal discountAmount = 0;

    if (!string.IsNullOrEmpty(model.PromoCode))
    {
        var promoCode = await _unitOfWork.PromoCodes.GetFirstOrDefaultAsync(
            p => p.Code.ToUpper() == model.PromoCode.ToUpper() && p.IsActive);

        if (promoCode != null)
        {
            bool isValid = ValidatePromoCode(promoCode, totalAmount);

            if (isValid)
            {
                discountAmount = CalculateDiscount(promoCode, totalAmount);
                totalAmount -= discountAmount;
                promoCodeId = promoCode.Id;

                promoCode.UsageCount++; // Increment usage
                _unitOfWork.PromoCodes.Update(promoCode); // Concurrency check
            }
        }
    }

    // --- Step 6: Create Order ---
    var order = new Order
    {
        UserId = userId,
        OrderDate = DateTime.Now,
        TotalAmount = totalAmount,
        Status = OrderStatus.Pending,
        PaymentMethod = model.PaymentMethod,
        ShippingAddress = model.Address,
        City = model.City, State = model.State,
        ZipCode = model.ZipCode, Country = model.Country,
        PhoneNumber = model.PhoneNumber,
        Notes = model.Notes ?? string.Empty,
        PromoCodeId = promoCodeId,
        DiscountAmount = discountAmount,
        OrderItems = orderItems
    };

```

```

        await _unitOfWork.Orders.AddAsync(order);

// --- Step 7: Create Payment ---
var payment = new Payment
{
    Amount = totalAmount,
    PaymentDate = DateTime.Now,
    PaymentMethod = model.PaymentMethod,
    Status = PaymentStatus.Pending,
    TransactionId = $"PENDING-{DateTime.Now.Ticks}"
};
order.Payment = payment;

await _unitOfWork.SaveAsync(); // Single SaveChanges – ALL or NOTHING

// --- Step 8: Handle payment method ---
if (model.PaymentMethod == PaymentMethod.CashOnDelivery)
{
    // COD: complete immediately
    _unitOfWork.ShoppingCarts.DeleteRange(cartItems);
    await _unitOfWork.SaveAsync();
    await transaction.CommitAsync();

    // Send confirmation email (fire-and-forget)
    await SendOrderConfirmationEmail(userId, order.Id, totalAmount);

    return RedirectToAction("OrderConfirmation", new { orderId = order.Id
});
}

// Credit Card: redirect to Stripe
var checkoutUrl = await _paymentService.CreateCheckoutSessionAsync(
    order.Id, totalAmount,
    cartItemList.Select(ci => ci.Product!.Name).ToList());

_unitOfWork.ShoppingCarts.DeleteRange(cartItems);
await _unitOfWork.SaveAsync();
await transaction.CommitAsync();

return Redirect(checkoutUrl);
}
catch (DbUpdateConcurrencyException) when (attempt < maxRetries)
{
    // --- Concurrency conflict: rollback and retry ---
    await transaction.RollbackAsync();
    _logger.LogWarning("Concurrency conflict (attempt {Attempt}/{MaxRetries})",
attempt, maxRetries);
    await Task.Delay(100 * attempt); // Exponential back-off: 100ms, 200ms,
300ms
    continue;
}
catch (DbUpdateConcurrencyException) when (attempt >= maxRetries)
{
    await transaction.RollbackAsync();
    _logger.LogError("Concurrency conflict – max retries reached");
    TempData["ErrorMessage"] = "Some items were just purchased. Please review
your cart.";
    return RedirectToAction("Index");
}
catch (DbUpdateException dbEx)

```

```

        {
            await transaction.RollbackAsync();
            _logger.LogError(dbEx, "Database error placing order");
            ModelState.AddModelError(string.Empty, $"DB ERROR: {dbEx.InnerException?.Message ?? dbEx.Message}");
            return View("Index", model);
        }
        catch
        {
            await transaction.RollbackAsync();
            throw; // Let the global error handler process this
        }
    }

    return RedirectToAction("Index");
}

```

5.7.3 Concurrency Handling in Detail

The `Product.Stock` field and `PromoCode.UsageCount` field both have `[Timestamp] byte[] RowVersion` attributes. When the checkout calls `_unitOfWork.SaveAsync()`, EF Core generates an SQL UPDATE like:

```

UPDATE Products SET Stock = @NewStock
WHERE Id = @Id AND RowVersion = @OriginalRowVersion;

UPDATE PromoCodes SET UsageCount = @NewCount
WHERE Id = @Id AND RowVersion = @OriginalRowVersion;

```

If another request has already modified either row between the SELECT and UPDATE, the `WHERE RowVersion = @OriginalRowVersion` matches zero rows, and EF Core throws `DbUpdateConcurrencyException`.

The retry loop handles this:

1. **Rollback** the entire transaction (all changes discarded).
2. **Wait** (100ms × attempt number) — exponential back-off reduces retry storms.
3. **Re-read** fresh data (new SELECT inside the next iteration).
4. **Retry** the entire order placement.

After 3 consecutive failures, the user receives a clear message asking them to review their cart — the items they wanted were purchased by someone else in the seconds between adding to cart and checking out.

Why aggregate stock failures (§5.7.2 Step 2)? All stock checks are performed *before* any writes. If multiple products are out of stock, the user sees *all* the failures at once, rather than fixing one at a time.

5.7.4 Promo Code Validation (Server-Side)

The validation endpoint called by the checkout page before order placement:

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> ValidatePromoCode([FromBody] ValidatePromoCodeRequest request)
{
    var promoCode = await _unitOfWork.PromoCodes.GetFirstOrDefaultAsync(
        p => p.Code.ToUpper() == request.Code.ToUpper());
}

```



```

    if (promoCode == null)
        return Json(new { success = false, message = "Invalid promo code." });

    if (!promoCode.IsActive)
        return Json(new { success = false, message = "This promo code is no longer active." });

    if (promoCode.StartDate.HasValue && DateTime.Now < promoCode.StartDate.Value)
        return Json(new { success = false, message = "This promo code is not yet valid." });

    if (promoCode.EndDate.HasValue && DateTime.Now > promoCode.EndDate.Value)
        return Json(new { success = false, message = "This promo code has expired." });

    if (promoCode.UsageLimit.HasValue && promoCode.UsageCount >=
        promoCode.UsageLimit.Value)
        return Json(new { success = false, message = "This promo code has reached its usage limit." });

    if (promoCode.MinimumPurchase.HasValue && request.OrderTotal <
        promoCode.MinimumPurchase.Value)
        return Json(new { success = false, message = $"Minimum purchase of {promoCode.MinimumPurchase.Value:C} required." });

    // Calculate discount
    decimal discountAmount = 0;
    if (promoCode.DiscountType == DiscountType.Percentage)
    {
        discountAmount = request.OrderTotal * (promoCode.DiscountValue / 100);
        if (promoCode.MaximumDiscount.HasValue && discountAmount >
            promoCode.MaximumDiscount.Value)
            discountAmount = promoCode.MaximumDiscount.Value;
    }
    else
    {
        discountAmount = promoCode.DiscountValue;
    }

    if (discountAmount > request.OrderTotal)
        discountAmount = request.OrderTotal;

    var newTotal = request.OrderTotal - discountAmount;

    return Json(new
    {
        success = true,
        discountAmount,
        newTotal,
        message = $"You saved {discountAmount:C}"
    });
}

```

Five validation checks: existence, active flag, start date, end date, usage limit, and minimum purchase. The `DiscountType` enum determines whether the value is a percentage (capped by `MaximumDiscount`) or a fixed amount. The discount can never exceed the order total.

5.7.5 Stripe Payment Integration

```

public class StripePaymentService : IPaymentService
{
    private readonly string _secretKey;
    private readonly string _domain;

    public StripePaymentService(IConfiguration configuration,
        ILogger<StripePaymentService> logger)
    {
        _secretKey = configuration["Stripe:SecretKey"] ?? throw new
ArgumentNullException("Stripe SecretKey");
        _domain = configuration["Stripe:Domain"] ?? throw new
ArgumentNullException("Stripe Domain");
        StripeConfiguration.ApiKey = _secretKey;
    }

    public async Task<string> CreateCheckoutSessionAsync(int orderId, decimal amount,
        List<string> productNames)
    {
        var lineItems = new List<SessionLineItemOptions>
        {
            new SessionLineItemOptions
            {
                PriceData = new SessionLineItemPriceDataOptions
                {
                    Currency = "usd",
                    ProductData = new SessionLineItemPriceDataProductDataOptions
                    {
                        Name = $"Order #{orderId}",
                        Description = string.Join(", ", productNames.Take(3))
                    },
                    UnitAmount = (long)(amount * 100), // Stripe uses cents
                },
                Quantity = 1,
            }
        };

        var options = new SessionCreateOptions
        {
            PaymentMethodTypes = new List<string> { "card" },
            LineItems = lineItems,
            Mode = "payment",
            SuccessUrl = $"({_domain})/Checkout/PaymentSuccess?orderId={orderId}",
            CancelUrl = $"({_domain})/Checkout/PaymentCancelled?orderId={orderId}",
            Metadata = new Dictionary<string, string>
            {
                { "order_id", orderId.ToString() }
            }
        };

        var service = new SessionService();
        var session = await service.CreateAsync(options);
        return session.Url;
    }
}

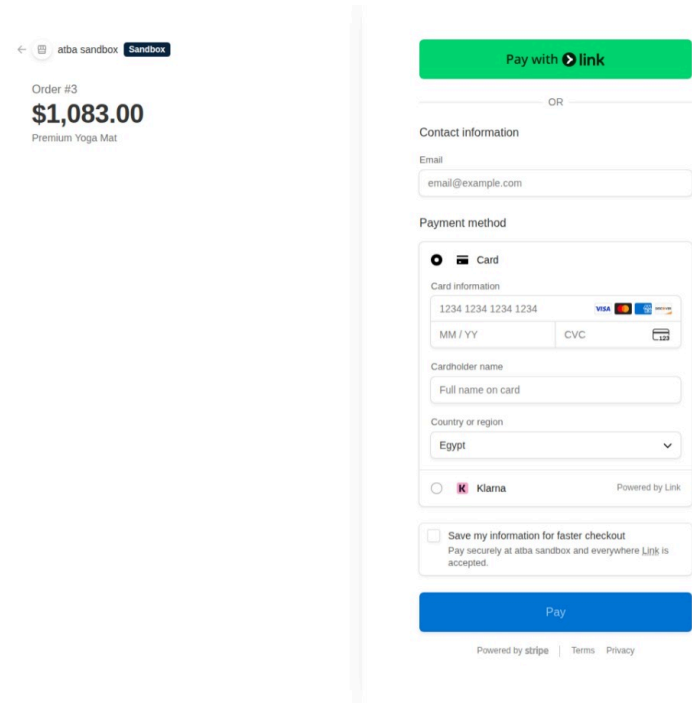
```

Key details:

- Stripe operates in cents (smallest currency unit). The `amount * 100` converts USD dollars to cents.
- `SuccessUrl` and `CancelUrl` define what happens after the Stripe-hosted checkout page.

- The **Metadata** dictionary carries the order ID through the redirect, enabling the **PaymentSuccess** action to identify which order was paid.
- The **IPaymentService** interface abstracts Stripe-specific code, allowing future payment providers (e.g., PayPal) without changing the controller.

The secure customer-facing checkout portal created dynamically by the **CreateCheckoutSessionAsync** service provides a localized and responsive card-input form:



All successful payments, subscription events, developer credentials, and webhook deliveries are monitored in real-time within the Stripe Merchant Dashboard:

Transactions

\$14,250.00 USD Succeeded ✓

Charged to **Abdulrahman Hossam Sayed**

Recent activity

+ Add note

- ✓ Payment authorized
[Jun 9, 1:37 PM](#)
- 📄 Payment started
[Jun 9, 1:37 PM](#)

Checkout summary

Customer **abdulrahmn398@outlook.com**
Abdulrahman Hossam Sayed
Egypt

ITEMS		QTY	UNIT PRICE	AMOUNT
Order #2	Classic Silver Timepiece	1	\$14,250.00	\$14,250.00
			Total	\$14,250.00

Furthermore, the administrator can drill down into specific customer transaction cards to view audit trails, charge fees, and transaction details:

Customers

  [+ Add customer](#)

All Remaining balances ...

[+ Filter](#) 

☐	Customer	Email	Description	Country	Created
☐	Abdulrahman Hossam Sayed Guest	abduhrahmn398@outlook.com	—	 Egypt	6/9/26, 1:39 PM

< > 1–1 of 1 results

Detailed log endpoints and webhook payloads can also be checked to verify server-to-server connectivity and status checks:

 Create and send your first invoice directly from the Stripe Dashboard—no code required.
 [Send first invoice](#) 

All
1

Succeeded
1

Refunded
0

Disputed
0

Failed
0

Uncaptured
0

5.7.6 Payment Callback Handling

PaymentSuccess: Updates payment status to **Completed**, order status to **Paid**, sends confirmation email.

```
public async Task<IActionResult> PaymentSuccess(int orderId)
{
    var order = await _unitOfWork.Orders.GetByIdAsync(orderId);
    var payment = await _unitOfWork.Payments.GetFirstOrDefaultAsync(p => p.OrderId ==
orderId);

    payment.Status = PaymentStatus.Completed;
    payment.PaymentDate = DateTime.Now;
    _unitOfWork.Payments.Update(payment);

    order.Status = OrderStatus.Paid;
    _unitOfWork.Orders.Update(order);
    await _unitOfWork.SaveAsync();

    // Send confirmation email
    await SendOrderConfirmationEmail(userId, order.Id, order.TotalAmount);

    return RedirectToAction("OrderConfirmation", new { orderId });
}
```

PaymentCancelled: Sets payment to **Failed**, order to **Cancelled**, allowing the user to retry.

```
public async Task<IActionResult> PaymentCancelled(int orderId)
{
    var order = await _unitOfWork.Orders.GetByIdAsync(orderId);
    var payment = await _unitOfWork.Payments.GetFirstOrDefaultAsync(p => p.OrderId ==
orderId);
```

```
payment.Status = PaymentStatus.Failed;
_unitOfWork.Payments.Update(payment);

order.Status = OrderStatus.Cancelled;
_unitOfWork.Orders.Update(order);
await _unitOfWork.SaveAsync();

TempData["ErrorMessage"] = "Payment was cancelled. Please try again.";
return RedirectToAction("Index");
}
```

5.8 Security and Error Handling

5.8.1 ASP.NET Core Identity

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    // Password policy
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;

    // Email uniqueness
    options.User.RequireUniqueEmail = true;

    // Sign-in
    options.SignIn.RequireConfirmedEmail = false;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders(); // Email confirmation, password reset tokens

// Cookie configuration
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.LogoutPath = "/Account/Logout";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.SlidingExpiration = true;
});
```

Password policy balances security with usability: requires digit + lowercase + uppercase (at least 6 characters). The non-alphanumeric requirement is relaxed since the other rules provide sufficient entropy for an e-commerce application.

The 30-day sliding cookie means users stay logged in across return visits as long as they are active within the 30-day window.

5.8.2 Role-Based Authorization

Admin-only actions are protected with `[Authorize(Roles = "Admin")]`:

```
[Authorize(Roles = "Admin")]
```

```
public class AdminController : Controller { ... }
```

The AI Assistant API requires any authenticated user (`[Authorize]` without a role), as discussed in §4.4.2.

5.8.3 Anti-Forgery Tokens

All POST endpoints use `[ValidateAntiForgeryToken]` paired with `@Html.AntiForgeryToken()` in the form. AJAX POSTs (wishlist, promo code validation, add-to-cart) include the token in the request header:

```
// Server: standard ValidateAntiForgeryToken
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> AddToCart(...)

// Client: token extracted from hidden field
function getToken() {
    return document.querySelector('input[name="__RequestVerificationToken"]')?.value;
}

fetch(url, {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'RequestVerificationToken': token
    },
    body: JSON.stringify({ ... })
});
```

5.8.4 Global Error Handling

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

In production, unhandled exceptions are caught by the `UseExceptionHandler` middleware, which redirects to a user-friendly error page without exposing stack traces. HSTS enforces HTTPS in production.

Per-controller error handling is implemented in critical actions:

- **CheckoutController.PlaceOrder** — catches `DbUpdateConcurrencyException`, `DbUpdateException`, and general exceptions with rollback and specific user messages.
- **ProductsController.AddReview** — catches exceptions and returns a generic "Failed to submit review" message.
- **AIAssistantController.Ask** — returns structured JSON error responses.

5.8.5 Input Validation

Server-side validation is enforced through Data Annotations on ViewModels:

```
public class CheckoutViewModel
{
    [Required(ErrorMessage = "Full name is required")]
    [StringLength(100)]
```

```
public string FullName { get; set; } = string.Empty;

[Required, EmailAddress]
public string Email { get; set; } = string.Empty;

[Required, Phone]
public string PhoneNumber { get; set; } = string.Empty;

[Required(ErrorMessage = "Address is required")]
[StringLength(500)]
public string Address { get; set; } = string.Empty;

// ... more fields
}
```

Client-side validation is enabled via the `_ValidationScriptsPartial` partial view, which includes jQuery Validation and Unobtrusive Validation scripts. This provides instant feedback without a server round-trip, while the server-side `ModelState.IsValid` check ensures security (client validation can be bypassed).

5.9 Database Seeding

The `DbInitializer.SeedAsync` method populates the database on first run:

```

public static async Task SeedAsync(ApplicationDbContext context,
    UserManager<ApplicationUser> userManager,
    RoleManager<IdentityRole> roleManager)
{
    // 1. Create roles: Admin, Seller, Customer
    if (!await roleManager.RoleExistsAsync("Admin"))
        await roleManager.CreateAsync(new IdentityRole("Admin"));
    if (!await roleManager.RoleExistsAsync("Seller"))
        await roleManager.CreateAsync(new IdentityRole("Seller"));
    if (!await roleManager.RoleExistsAsync("Customer"))
        await roleManager.CreateAsync(new IdentityRole("Customer"));

    // 2. Create admin user (idempotent – checks email before creating)
    if (await userManager.FindByEmailAsync("admin@ecommerce.com") == null)
    {
        var adminUser = new ApplicationUser { ... };
        await userManager.CreateAsync(adminUser, "Admin@123");
        await userManager.AddToRoleAsync(adminUser, "Admin");
    }

    // 3. Seed 6 categories with 18 products (3 per category)
    if (!context.Categories.Any())
    {
        var categories = new List<Category> { ... };
        context.Categories.AddRange(categories);
        await context.SaveChangesAsync();
    }

    if (!context.Products.Any())
    {
        var products = new List<Product> { ... };
        context.Products.AddRange(products);
        await context.SaveChangesAsync();
    }
}

```

Called from `Program.cs` with error logging:

```

using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    var userManager =
scope.ServiceProvider.GetRequiredService<UserManager<ApplicationUser>>();
    var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    await DbInitializer.SeedAsync(context, userManager, roleManager);
}

```

The seed logic is **idempotent** — it checks for existing data before inserting, allowing the application to restart without duplicate data errors.

5.10 Email Service (Transactional Emails)

The `EmailService` uses SMTP via Gmail for sending transactional emails:


```

public class EmailService : IEmailService
{
    private readonly EmailSettings _emailSettings;

    public EmailService(IOptions<EmailSettings> emailSettings)
    {
        _emailSettings = emailSettings.Value;
    }

    public async Task SendEmailAsync(string toEmail, string subject, string body)
    {
        var smtpClient = new SmtpClient(_emailSettings.SMTPServer)
        {
            Port = _emailSettings.SMTPPort,
            Credentials = new NetworkCredential(_emailSettings.SenderEmail,
            _emailSettings.SenderPassword),
            EnableSsl = true,
        };

        var mailMessage = new MailMessage
        {
            From = new MailAddress(_emailSettings.SenderEmail,
            _emailSettings.SenderName),
            Subject = subject,
            Body = body,
            IsBodyHtml = true,
        };

        mailMessage.To.Add(toEmail);
        await smtpClient.SendMailAsync(mailMessage);
    }
}

```

Three email types are supported:

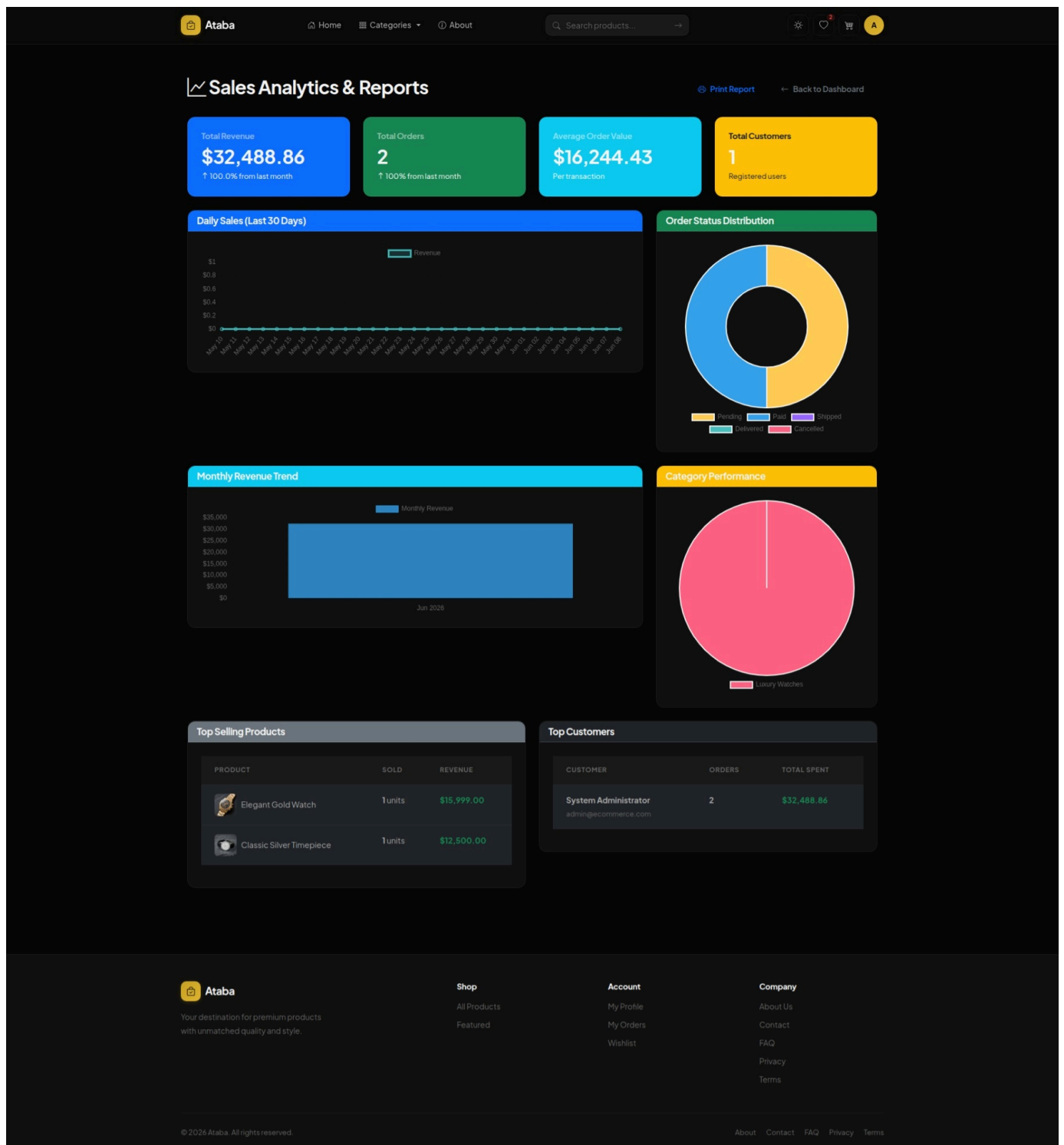
- **SendOrderConfirmationEmailAsync** — triggered after successful order placement
- **SendPasswordResetEmailAsync** — triggered by "Forgot Password" flow
- **SendRegistrationConfirmationEmailAsync** — triggered after account creation

Each method embeds its own HTML template with inline CSS for maximum email-client compatibility. SMTP credentials are stored in `appsettings.json` under `EmailSettings` and should be Gmail App Passwords (not the primary account password).

Email sending intentionally **does not block the checkout flow** — the order confirmation email is sent in a try/catch after the transaction commits, and failures are logged as warnings rather than errors. The checkout succeeds regardless of email delivery status.

5.11 Analytics Service

The administrative analytics service generates summaries of revenue trends, average order values, and category performance. These figures are visualized on the dashboard, as shown in the screenshot below:



The **AnalyticsService** provides server-side aggregated data for the admin dashboard:

```
public async Task<SalesAnalyticsViewModel> GetSalesAnalyticsAsync()
{
    var model = new SalesAnalyticsViewModel();
    var orderQuery = _unitOfWork.Orders.GetQueryable(asNoTracking: true);
    var now = DateTime.Now;
    var thisMonthStart = new DateTime(now.Year, now.Month, 1);

    // Overview
    model.TotalOrders = await orderQuery.CountAsync();
    model.TotalRevenue = await orderQuery.SumAsync(o => o.TotalAmount);
    model.AverageOrderValue = model.TotalOrders > 0 ? model.TotalRevenue /
model.TotalOrders : 0;
```

```
// Month-over-month growth
model.OrdersThisMonth = await orderQuery.CountAsync(o => o.OrderDate >=
thisMonthStart);
model.RevenueThisMonth = await orderQuery.Where(o => o.OrderDate >= thisMonthStart)
.SumAsync(o => o.TotalAmount);

// Top selling products (aggregated server-side)
var topProducts = await _unitOfWork.OrderItems.GetQueryable(asNoTracking: true)
.GroupBy(oi => oi.ProductId)
.Select(g => new { ProductId = g.Key, QuantitySold = g.Sum(oi => oi.Quantity)
})
.OrderByDescending(x => x.QuantitySold)
.Take(10)
.ToListAsync();

// Daily sales (last 30 days) with zero-fill for days with no orders
model.DailySales = await GetDailySalesAsync(30);

return model;
}
```

All aggregation happens at the database level (LINQ translates to SQL `GROUP BY`, `SUM`, `COUNT`), avoiding in-memory loading of entire tables. The zero-fill pattern for daily sales ensures chart rendering does not have gaps for inactive days.

5.12 Dependency Injection Summary (Program.cs)

Lifetime	Service	Implementation
Scoped	<code>IUnitOfWork</code>	<code>UnitOfWork</code>
Scoped	<code>IGeminiService</code>	<code>GeminiService</code>
Scoped	<code>IPaymentService</code>	<code>StripePaymentService</code>
Scoped	<code>IImageService</code>	<code>ImageService</code>
Scoped	<code>IEmailService</code>	<code>EmailService</code>
Scoped	<code>IAalyticsService</code>	<code>AnalyticsService</code>
Singleton	<code>IMemoryCache</code>	(built-in)
Singleton	<code>IConfiguration</code>	(built-in)
Singleton	<code>IHttpClientFactory</code>	(built-in, via <code>AddHttpClient</code>)
Scoped	<code>UserManager<ApplicationUser></code>	(Identity)
Scoped	<code>RoleManager<IdentityRole></code>	(Identity)
Scoped	<code>SignInManager<ApplicationUser></code>	(Identity)
Scoped	<code>ApplicationDbContext</code>	(EF Core)

All services are **scoped** (one instance per HTTP request), matching the `DbContext` lifetime. Singletons are used only for truly stateless or shared services (cache, configuration).

6. Agile User Stories & Requirements Traceability

6.1 Introduction

This project was developed following **Agile methodologies**, specifically a tailored Scrum-Kanban hybrid suited for a solo developer context. Work was organized into **two-week sprints** with a prioritized product backlog maintained throughout the development lifecycle. Each feature was decomposed into **vertical slices** spanning the full stack — from Razor views and CSS styling down to database migrations and service-layer logic. The user stories below represent the complete functional scope of the e-commerce platform, organized into logical **Epics**. Each story includes a **priority rating** (High / Medium / Low) and **acceptance criteria** expressed as concrete, testable conditions. These stories collectively trace back to every controller action, service method, database entity, and view component in the system.

6.2 Epic 1: User Identity & Security

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-001	New visitor	Register a new account with my name, email, phone, address, and password	I can access personalized features like cart, wishlist, and order history	- Registration form validates all required fields (<code>FullName</code>, <code>Email</code>, <code>PhoneNumber</code>, <code>Password</code>) - Password must meet Identity policy (6+ chars, digit, lowercase, uppercase) - Duplicate email returns a validation error - On success, user is signed in automatically and redirected to Home - A welcome email is sent to the registered address	High
US-002	Registered user	Log in using my email and password	I can resume my shopping session	- Login form validates email format and non-empty password - Invalid credentials show a generic "Invalid login attempt" message - Successful login redirects to the originally requested page or Home - Cookie is set with 30-day sliding expiration - Locked-out users see an access-denied message	High
US-003	Authenticated user	Log out of my account	I can securely end my session on shared devices	- Logout button is available in the user dropdown menu - POST to <code>/Account/Logout</code> clears the authentication cookie - After logout, user is redirected to Home - Protected pages (Cart, Checkout, Profile) are no longer accessible	High
US-004	Registered user	Reset my password if I forget it	I can regain access to my account	"Forgot Password" link is visible on the Login page - User enters their email and receives a password-reset link - The link contains an Identity-generated token and expires after 1 hour - Token reuse is prevented (single-use) - New password must satisfy the same policy as registration	Medium

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-005	Authenticated user	View and edit my profile (name, phone, address, city, country)	I can keep my shipping information up to date	• Profile page pre-fills all fields from the database • Changes are persisted via <code>userManager<ApplicationUser>.UpdateAsync()</code> • Email is read-only (cannot be changed) • Success/failure messages are displayed via TempData	Medium
US-006	Authenticated user	Change my password from within my account settings	I can update my credentials without contacting support	• Current password must be provided for verification • New password must satisfy Identity policy • Uses <code>userManager.ChangePasswordAsync()</code> with built-in validation • On success, user is notified and remains logged in	Medium
US-007	Authenticated user	Delete my account permanently	I can exercise my right to data removal	• GET <code>/Account/DeleteAccount</code> shows a confirmation page with a clear warning • POST <code>/Account/DeleteAccountConfirmed</code> deletes cart items, wishlist items, reviews, and orders before calling <code>userManager.DeleteAsync()</code> • The user is signed out immediately after deletion • An admin cannot delete their own account via this endpoint • 404 is returned if the user is not found	Low

6.3 Epic 2: Product Browsing & Discovery

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-007	Any visitor	Browse a paginated grid of all active products	I can discover what the store offers	• Products are displayed in an <code>auto-fill</code> CSS grid (min 260px per card) • Inactive products are excluded server-side via <code>p.IsActive</code> filter • Pagination shows 12 products per page with smart-ellipsis navigation • Each card shows: image, name, truncated description, price, stock badge • "In Stock" / "Sold Out" badges are colour-coded (green/red)	High
US-008	Any visitor	Filter products by category, search term, and price range	I can narrow down products to what I need	• Filter form with: category dropdown, search text input, min/max price, sort • Filters apply client-side via debounced AJAX (400ms for search, 300ms for others) • AJAX response replaces the product grid partial without full page reload • URL is updated via <code>history.replaceState</code> for back-button support • A loading spinner overlay is shown during AJAX requests	High

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-009	Any visitor	Sort products by newest, price (low-high / high-low), and name (A-Z / Z-A)	I can find products in my preferred order	- Sort dropdown triggers AJAX reload (no page refresh) <code>price_asc</code> / <code>price_desc</code> sorts by <code>Product.Price</code> <code>name_asc</code> / <code>name_desc</code> sorts by <code>Product.Name</code> <code>newest</code> (default) sorts by <code>Product.CreatedDate</code> descending - Server-side LINQ translates to SQL <code>ORDER BY</code>	Medium
US-010	Any visitor	View a product's full details, variants, reviews, and related products	I can make an informed purchase decision	- Detail page shows: large image, price, description, stock, category - Variants (size/color) are displayed as selectable cards - Selecting a variant updates the displayed price (base + additional) - Rating summary shows 1-5 star breakdown with visual bars 4 related products from the same category are shown at the bottom "Verified Purchase" badge appears on reviews from confirmed buyers	High
US-011	Any visitor	Browse products by category from the homepage	I can discover products in a specific category quickly	- Category section on Home shows up to 6 categories in an auto-fill grid - Each category card has an icon, name, and links to filtered product list - Categories are cached in <code>IMemoryCache</code> for 10 minutes <code>/Products/ByCategory/{id}</code> route filters and paginates correctly	Medium
US-012	Any visitor	View featured products on the homepage	I can see the store's highlighted selections	- Featured products (<code>IsFeatured = true</code>) are shown in a grid on Home - Limited to 8 products, sorted by Id - Each featured card includes a gold "Featured" badge - Data is cached with 10-minute absolute + 2-minute sliding expiration	Medium

6.4 Epic 3: Shopping Cart & Wishlist

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-013	Authenticated user	Add a product to my cart with a specific quantity and variant	I can prepare to purchase it	- Clicking "Add to Cart" creates/updates a <code>ShoppingCart</code> record - If a variant is selected, the variant's stock is validated - If the product already exists in the cart (same product + variant), quantity is incremented	High

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
				- Cumulative quantity must not exceed available stock - Success confirmation is shown via TempData	
US-014	Authenticated user	View my cart with item details, quantities, and total	I can review my selections before checkout	- Cart page shows: thumbnail, name, variant info, quantity stepper, line total, remove button - Inactive products are filtered out from the view - Summary sidebar shows: subtotal, 14% tax, shipping, grand total - Sticky sidebar follows the user on scroll (desktop) - Empty cart shows a friendly empty-state with a "Shop Now" CTA	High
US-015	Authenticated user	Increase or decrease item quantities in the cart	I can adjust my order before checkout	- Plus/minus buttons submit POST to <code>UpdateQuantity</code> - Quantity is bounded: min 1 (minus button disables at 1), max = stock - Setting quantity to 0 removes the item entirely - Stock is re-verified server-side on each update	High
US-016	Authenticated user	Remove individual items or clear the entire cart	I can start over or remove unwanted items	- Remove button (trash icon) with JavaScript confirmation dialog "Clear Cart" button removes all items for the current user - Both operations call POST endpoints with <code>[ValidateAntiForgeryToken]</code> - Cart badge count in the header updates after any change	High
US-017	Authenticated user	Toggle products in my wishlist	I can save items for future consideration	- Heart icon button on product cards toggles wishlist state - Optimistic UI update: icon changes immediately, reverts on failure - CSRF token is sent via <code>RequestVerificationToken</code> header - Duplicate (user, product) is prevented by unique index in DB "Please sign in" toast appears for unauthenticated clicks	Medium
US-018	Authenticated user	View my wishlist	I can see all saved products in one place	- Wishlist page lists all saved products with thumbnail and price - Quick "Add to Cart" button for each wishlist item - Empty wishlist shows an empty-state message - Wishlist count badge in the header updates on page load	Low

6.5 Epic 4: Checkout & Payments

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-019	Authenticated user	Proceed to checkout with pre-filled shipping information	I can complete my purchase quickly	• Checkout page pre-fills name, email, phone, address, city, country from user profile • Cart items are re-loaded and totals (subtotal, 14% tax, total) are calculated • Empty cart redirects back to cart with an error message • Form includes all required fields: FullName, Email, Phone, Address, City, Country, PaymentMethod	High
US-020	Authenticated user	Apply a promo code to my order	I can receive a discount	• Promo code input with "Apply" button dispatches an AJAX POST to <code>ValidatePromoCode</code> • Server validates: code exists, is active, within date range, usage limit not reached, minimum purchase met • For percentage discounts, the cap (<code>MaximumDiscount</code>) is enforced • Discount amount and new total are displayed immediately without page reload • Invalid/expired codes show a red error message	Medium
US-021	Authenticated user	Place an order with Cash on Delivery	I can pay when the package arrives	• <code>PlaceOrder</code> creates: <code>Order</code> (status: Pending), <code>Payment</code> (status: Pending, method: COD) • Product stock is decremented atomically within a database transaction • Cart items are deleted after successful order creation • Order confirmation email is sent to the user • User is redirected to <code>OrderConfirmation</code> page with order details	High
US-022	Authenticated user	Pay via credit card through Stripe	I can pay online securely	• Selecting "Credit Card (Stripe)" triggers redirect to Stripe Checkout • Stripe session includes: order ID, total (in cents), product names • On success, Stripe redirects to <code>/Checkout/PaymentSuccess</code> which updates status to Paid • On cancellation, Stripe redirects to <code>/Checkout/PaymentCancelled</code> which sets status to Cancelled • If Stripe keys are not configured, a mock success path is used for development	High

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-023	Authenticated user	View an order confirmation page after placing an order	I have a record of my purchase	- Confirmation page shows: order ID, total, items list, payment status - Items are loaded via <code>OrderItems</code> with eager-loaded <code>Product</code> - Only the owning user can view the order (authorization check via <code>UserId</code>) 404 returned if the order does not belong to the current user	High
US-024	Authenticated user	View my complete order history	I can track all past purchases in one place	<code>OrdersController.MyOrders</code> returns all orders for the authenticated user sorted by date descending <code>OrdersController.Details(id)</code> shows full order detail including line items and payment info - Access is restricted: 404 is returned if the order does not belong to the requesting user - Both pages require <code>[Authorize]</code>	Medium
US-025	System	Prevent overselling when two users checkout the same product simultaneously	I can maintain inventory accuracy	<code>Product</code> entity has <code>[Timestamp] byte[] RowVersion</code> concurrency token <code>PlaceOrder</code> runs in a <code>for</code> loop with up to 3 retries - On <code>DbUpdateConcurrencyException</code>, the transaction rolls back, waits (100ms × attempt), and retries - After 3 failures, user sees: "Some items were just purchased by another customer" - Promo code <code>UsageCount</code> also uses <code>[Timestamp]</code> to prevent double-spending	High

6.6 Epic 5: AI Product Assistant (Gemini Integration)

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-026	Authenticated user	Ask a question about a product to an AI assistant	I can get instant answers without contacting support	- Each product card has an AI button (sparkle icon) visible on hover - Clicking opens a global modal with the product name + description pre-loaded - User types a question and presses Enter or clicks Send - A loading spinner with "Thinking..." is shown during API call - The response appears in a styled text box below the question input	Medium
US-027	Authenticated user	Receive relevant answers even if my	I can get helpful information without	The system prompt instructs Gemini to answer concisely and	Medium

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
		question is vague	phrasing perfectly	- redirect off-topic questions • Product context (name + description) is injected into every prompt • Max output is 800 tokens — responses are concise • Temperature = 0.7 balances creativity with factual accuracy	
US-028	Authenticated user	See a friendly error message if the AI service is unavailable	I understand what happened and can try again later	• 429 (rate limit) returns: "AI service quota exceeded. Please try again in N seconds." • Network errors show: "Network error. Please try again." • Prompt blocked by safety filters shows: "I'm sorry, I couldn't process that request." • Unauthenticated users receive a 401 from the <code>[Authorize]</code> filter	Low

6.7 Epic 6: Admin Dashboard & Management

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-029	Admin	View a dashboard with key business metrics	I can monitor store performance at a glance	• Dashboard shows: total revenue, total orders, total products, total users • Statistics for: pending orders, completed orders, low-stock products (< 10) • Top 5 selling products by quantity are displayed • Orders grouped by payment method are shown • Recent 5 orders display with user name and total • All numbers are aggregated server-side via LINQ <code>Sum()</code> and <code>Count()</code>	High
US-030	Admin	Manage products (create, edit, delete, toggle featured)	I can keep the product catalog up to date	• Product list is paginated (15 per page) with search • Create product form includes: name, description, price, stock, category, image upload, featured flag • Image upload generates a GUID filename and stores in <code>wwwroot/images/products/</code> • Edit product pre-fills all fields and allows image replacement • Old image is deleted from disk when a new one is uploaded • Delete is confirmed via JavaScript prompt	High

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-031	Admin	View and manage registered users	I can activate/deactivate or delete accounts	• User list is paginated with order count per user and assigned roles • Role lookups are batched per role (not per user) to avoid N+1 queries • Admin can toggle account lockout (activate/deactivate) • Admin can delete users (with cascade cleanup of cart items) • Self-deactivation and self-deletion are prevented	High
US-032	Admin	View detailed sales analytics with charts	I can track business performance over time	• Analytics include: total orders, total revenue, average order value • Month-over-month order and revenue growth percentages are calculated • Top 10 selling products with quantity and revenue • Top 10 customers by total spent • Category performance (products sold per category) • Daily sales chart data (last 30 days) with zero-fill for inactive days • Monthly sales chart data (last 12 months)	Medium

6.8 Epic 7: Core Infrastructure & User Experience

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-033	Any visitor	Toggle between dark and light theme	I can browse in my preferred visual mode	• Theme toggle button in the header switches <code>data-bs-theme</code> attribute • Theme preference is persisted via a cookie with 1-year expiry • FOUC is prevented: server reads the cookie and sets the attribute before HTML is sent • All design tokens (backgrounds, text, borders, shadows) switch correctly • Sun/moon icon toggles to reflect the current theme	Medium
US-034	Any visitor	Navigate the site on my mobile phone	I can shop from any device	• Header collapses to hamburger menu at 768px • Mobile nav includes: Home, Products, About, Contact, FAQ, and all categories • Product grid collapses to 2 columns at 520px, 1 column at 360px • Cart layout becomes single column below 1000px • Hero section hides the decorative visual ring on mobile • Quantity stepper and buttons remain touch-friendly	High

ID	As a...	I want to...	So that...	Acceptance Criteria	Priority
US-035	Any visitor	Receive instant feedback when I perform actions	I know whether my action succeeded or failed	• Toast notifications appear for: add/remove wishlist, add to cart, errors • Toasts slide in from the right and auto-dismiss after 4.2 seconds • Toasts are colour-coded (green for success, red for error) • Each toast has a dismiss button • Multiple toasts stack vertically	Medium
US-036	Any visitor	View a sitemap.xml for search engine indexing	The store can be discovered via search engines	• <code>/Home/Sitemap</code> returns valid XML with all product and category URLs • Each URL has <code><changefreq>weekly</changefreq></code> and <code><priority>0.8</priority></code> • Response is cached for 86400 seconds (24 hours) • Content-Type is <code>application/xml</code>	Low
US-037	Any visitor	Access privacy policy, terms of service, about, contact, and FAQ pages	I can learn about the store and its policies	• Each page has a dedicated controller action and Razor view • Privacy and Terms are response-cached for 3600 seconds • These pages are linked from the footer in all four columns • All links use <code>asp-controller</code> and <code>asp-action</code> tag helpers for correct URL generation	Low

6.9 Story Point Summary

Epic	Stories	High	Medium	Low
1: User Identity & Security	7	3	3	1
2: Product Browsing & Discovery	6	3	3	0
3: Shopping Cart & Wishlist	6	4	1	1
4: Checkout & Payments	7	5	2	0
5: AI Assistant	3	0	2	1
6: Admin Dashboard	4	3	1	0
7: Core Infrastructure & UX	5	2	2	1
Total	38	20	14	4

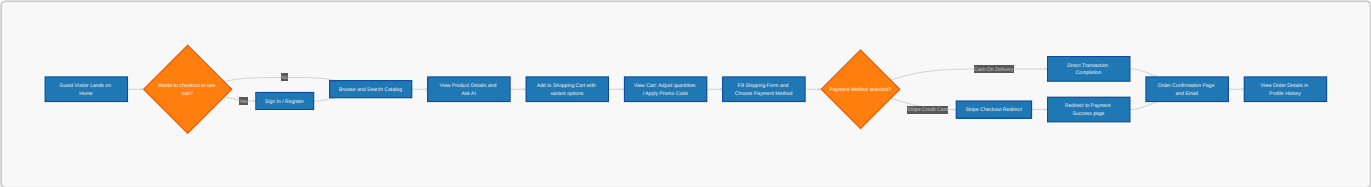
All **38 user stories** map directly to implemented code across the three-tier architecture — from Entity Framework Core entities and LINQ queries, through service-layer business logic and controller actions, to Razor views, CSS custom properties, and client-side JavaScript. The **20 high-priority stories** represent the core e-commerce workflow (auth → browse → cart → checkout) that was implemented first, with medium and low priorities rounding out account management, AI assistant, analytics, and SEO features.

6.10 Proposed System Screens and Interaction Walkthrough

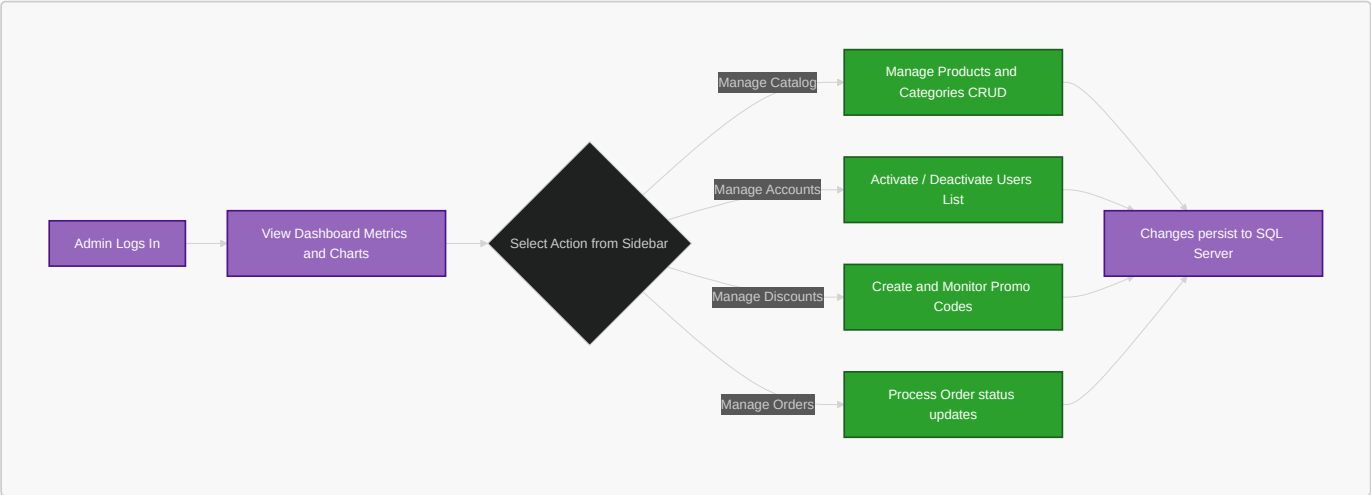
To demonstrate the functioning proposed system and show how the user stories are fully fulfilled, the following walkthrough details the screen transitions, steps of user interaction, and provides illustrative screenshots of the actual designed system interfaces.

6.10.1 Interaction Flowcharts

The flowchart below maps out the sequence of steps a Customer takes when interacting with the system:



The flowchart below maps out the sequence of actions an Administrator takes to manage the platform operations:

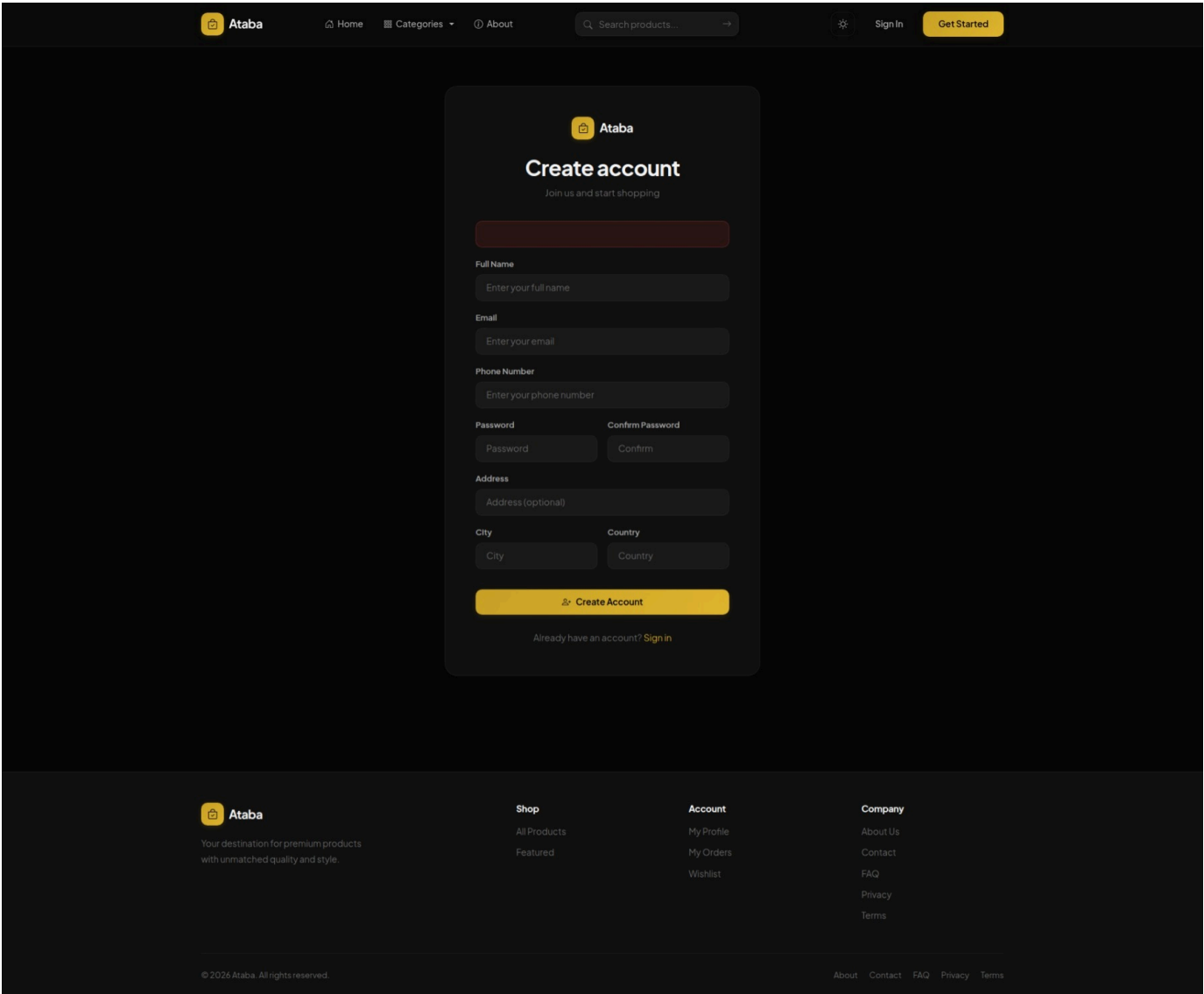


6.10.2 Customer Account Operations

This phase details the user identity steps (Epic 1) including registering, logging in, updating profile details, and changing credentials.

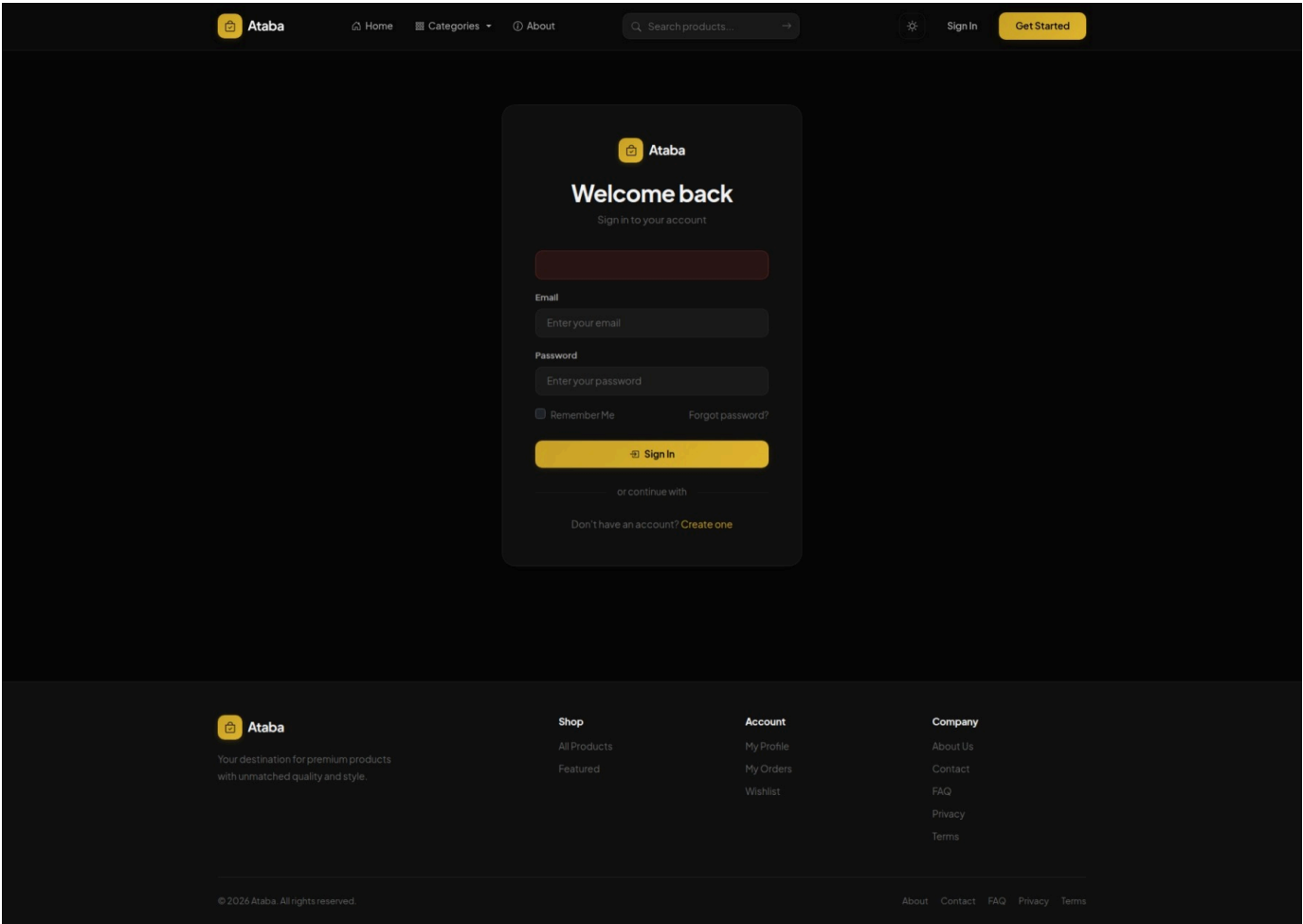
Step 1: User Registration

New visitors submit their email and details using the registration form. Client-side and server-side validation are enforced to meet security policies (e.g., strong password constraints), as shown in the screenshot below:



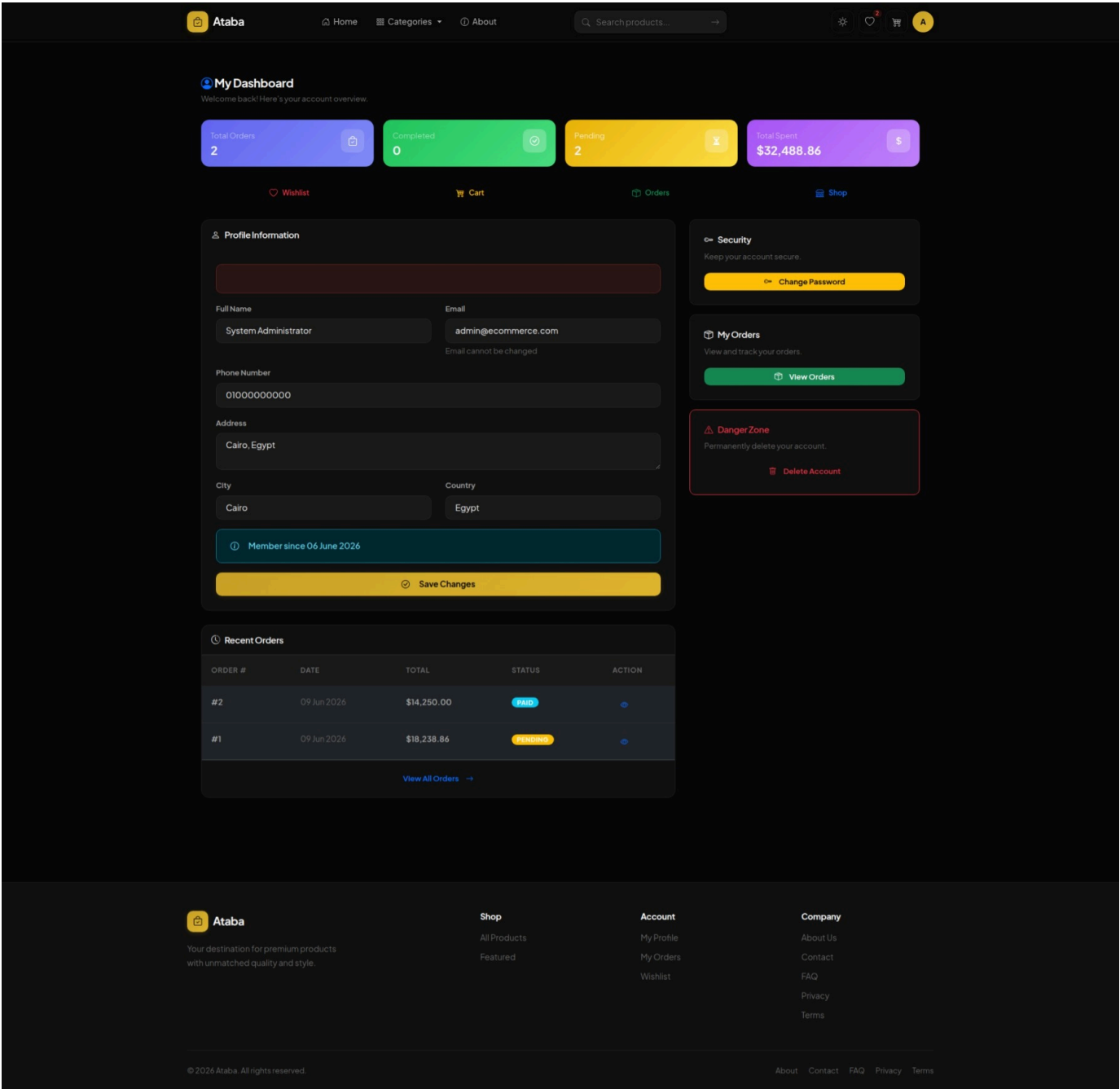
Step 2: User Login

Registered customers authenticate via the secure login portal using their credentials, setting a persistent authentication cookie upon success:



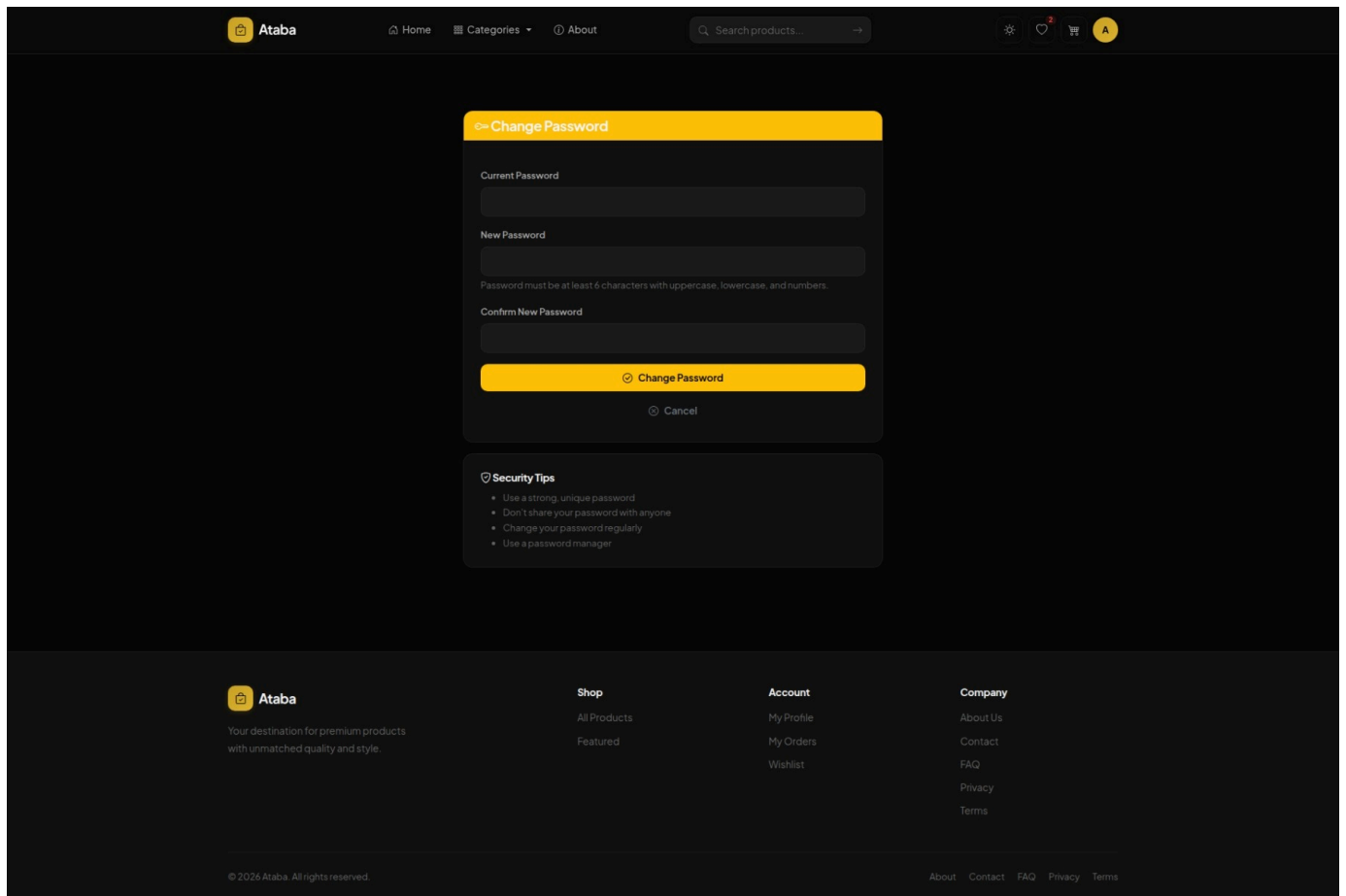
Step 3: Account Profile

Once logged in, the customer has access to a centralized profile dashboard showing their basic details, shipping address, and quick links to update their security settings:



Step 4: Password Change Settings

From their profile, users can navigate to the password change form where they must supply their current password to establish a new one:



6.10.3 Customer Shopping & Checkout Journey

This phase covers product browsing, catalog searching, variant selection, AI integration, shopping cart management, checkout form completion, payment selection, and final order confirmation (Epics 2, 3, 4, 5).

Step 5: Product Catalog (Home Page & Filtering)

The default catalog view displays featured products and categories. The user can search or filter products by price and category via debounced AJAX calls that update the product grid asynchronously without a full page refresh:

- **Browse home view:** (Visualized in Section 1.6: [images/home.jpeg](#))
- **Product catalog with sidebar filters:** (Visualized in Section 2.5: [images/products.jpeg](#))

Step 6: Product Details and AI Q&A Assistant

Clicking on a product opens its detail page. Users can select variations (such as size/color) and trigger the real-time Google Gemini assistant to ask questions regarding the product:

- **Product details with variant selection & AI modal trigger:** (Visualized in Section 3.16.3 / Section 4.3.5: [images/product-details.jpeg](#))

Step 7: Cart Management

Customers review selected items in their cart, change quantities dynamically, or remove items. The system calculates a running total including shipping and a 14% local tax:

- **Shopping cart layout:** (Visualized in Section 3.6: [images/cart.jpeg](#))

Step 8: Checkout and Promo Coupon Application

At checkout, the user's shipping details are pre-filled. They can apply active promotional codes to recalculate totals and select cash on delivery or credit card payment options:

- **Checkout forms and promo code application:** (Visualized in Section 3.7: [images/checkout.jpeg](#))

Step 8b: Stripe Card Checkout & Payment Monitoring

If the customer selects "Credit Card (Stripe)" as their payment option, they are redirected to a secure, dynamically generated Stripe Checkout portal to enter their payment details:

The screenshot displays a Stripe Checkout page. On the left, the order summary shows 'Order #3' with a total of '\$1,083.00' for 'Premium Yoga Mat'. The top right features a green 'Pay with link' button. Below this is a section for 'Contact information' with an email field containing 'email@example.com'. The 'Payment method' section is active, showing a 'Card' payment option. The card details form includes fields for the card number (1234 1234 1234 1234), expiration date (MM / YY), CVC, and cardholder name (Full name on card). The country is set to 'Egypt'. There is a checkbox for 'Save my information for faster checkout' and a 'Pay' button at the bottom. The footer indicates 'Powered by stripe' with links to 'Terms' and 'Privacy'.

Once payment is processed, the system receives the return callback. Administrators can monitor incoming transactions, webhook logs, and audit histories directly inside the Stripe Merchant Dashboard:

Transactions

\$14,250.00 USD

Succeeded ✓

Charged to Abdulrahman Hossam Sayed

Recent activity

+ Add note

- ✓

Payment authorized

Jun 9, 1:37 PM
- 📄

Payment started

Jun 9, 1:37 PM

Checkout summary

Customer

abdulrahmn398@outlook.com

Abdulrahman Hossam Sayed

Egypt

ITEMS		QTY	UNIT PRICE	AMOUNT
Order #2	Classic Silver Timepiece	1	\$14,250.00	\$14,250.00
			Total	\$14,250.00

Step 9: Order Confirmation

Upon placement of a Cash on Delivery (COD) order, or on successful return from Stripe payment, the customer receives an immediate confirmation showing their invoice summary:

Order placed successfully!

✓

Order Confirmed!

Thank you for your order. Check your email for details.

Ataba

HomeCategoriesAbout

Search products...

⚙️

📖

🛒

A

Status

PENDING

Payment

CashOnDelivery

Shipping Address

Bani Swayfe

6th of October, Egypt

Phone: 01004591659

TransactionID

PENDING-639166210964449364

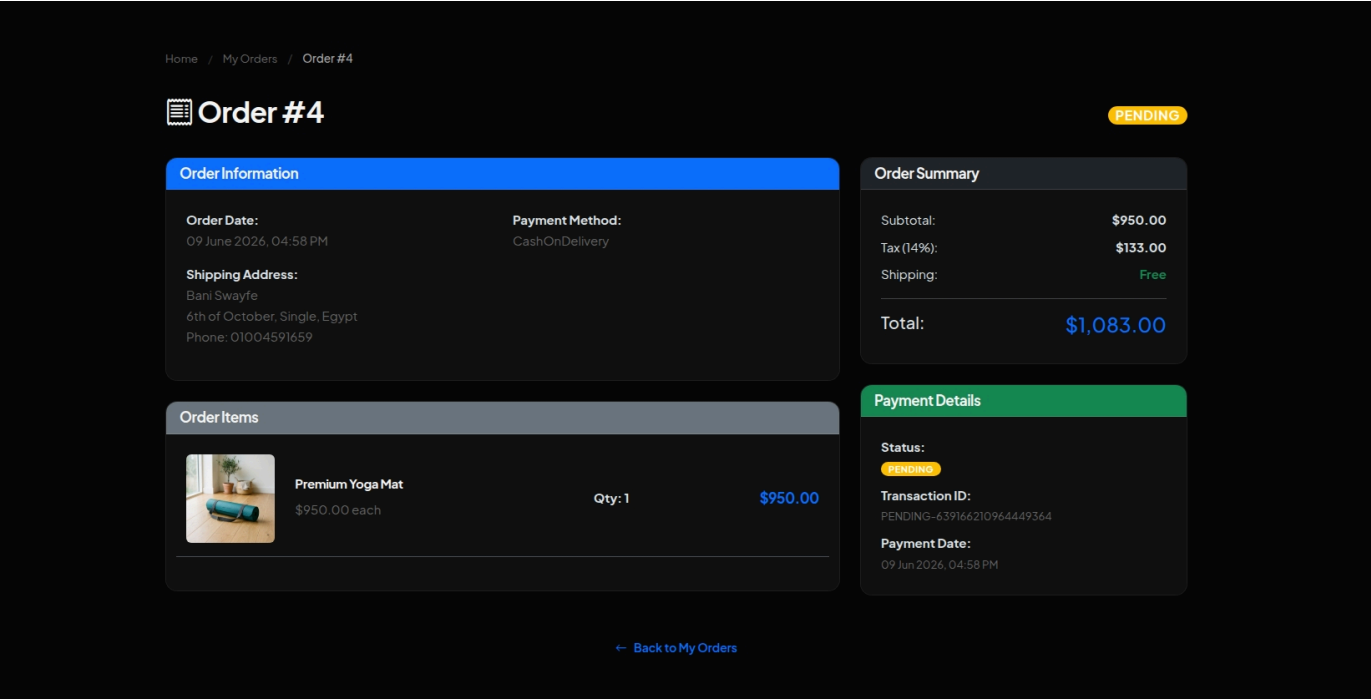
PENDING

My Orders

Home

Step 10: Personal Order Details and History

Customers can view their transaction history from their profile to track the payment and delivery status of past orders:

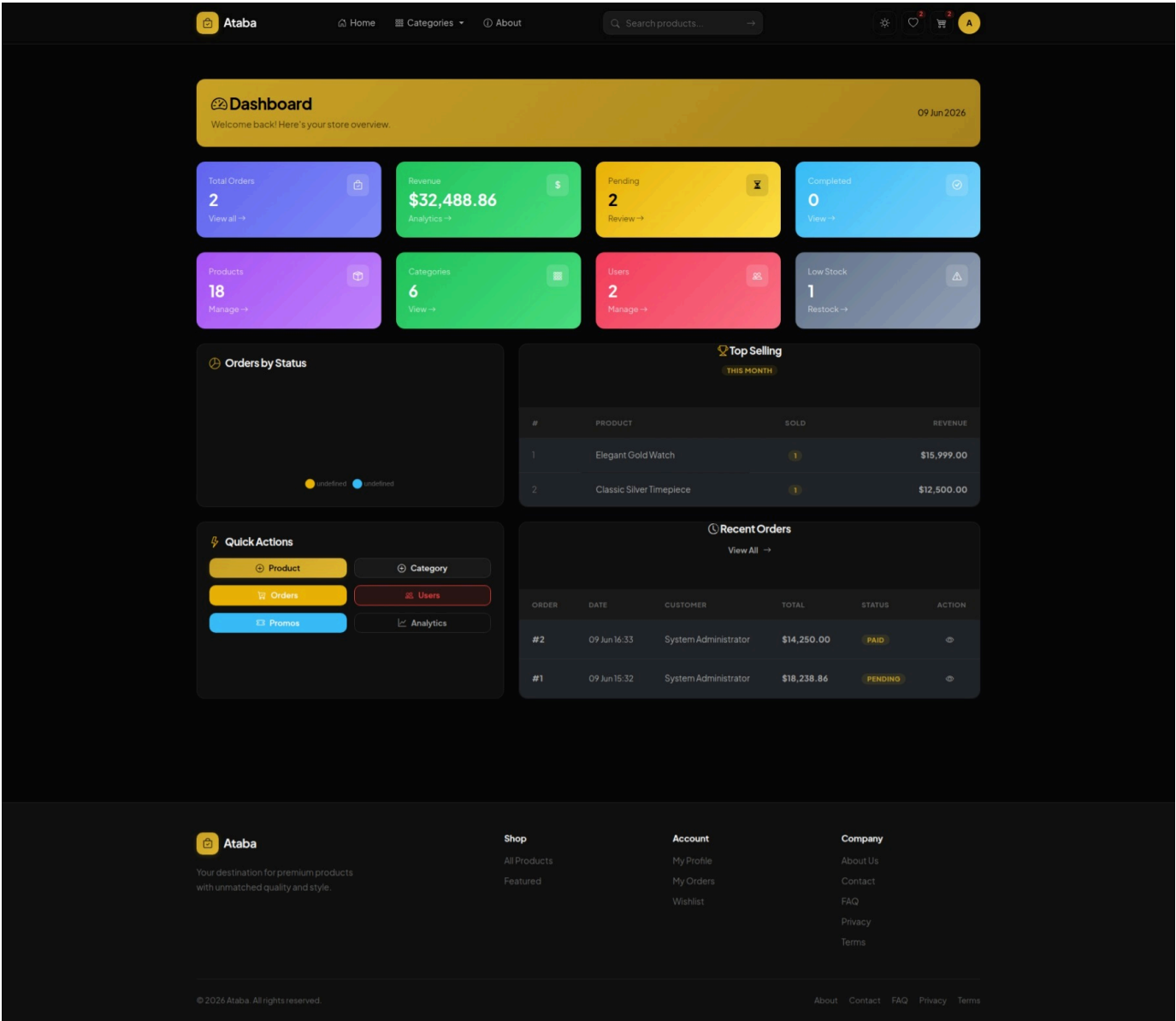


6.10.4 Administrator System Management

This phase traces administrative oversight, including dashboard charts, product/category CRUD, promo codes, user roles, and order processing (Epic 6).

Step 11: Admin Dashboard and Sales Reports

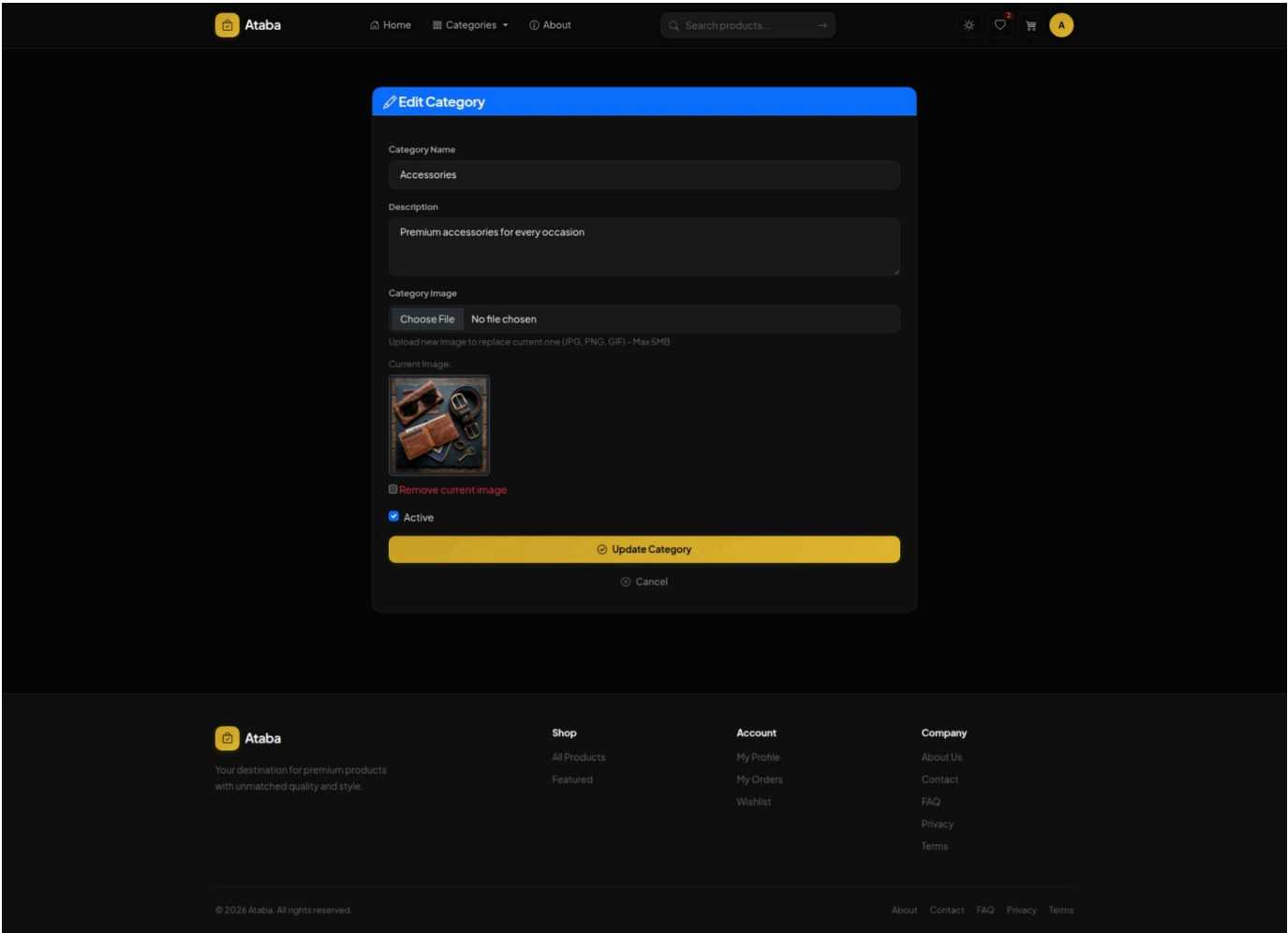
Administrators access a comprehensive dashboard showing sales statistics, order status summaries, and inventory warning levels:



- Detailed sales analytics with Chart.js visualization: (Visualized in Section 5.11: [images/admin-anlyitcs.jpeg](#))

Step 12: Admin Category CRUD

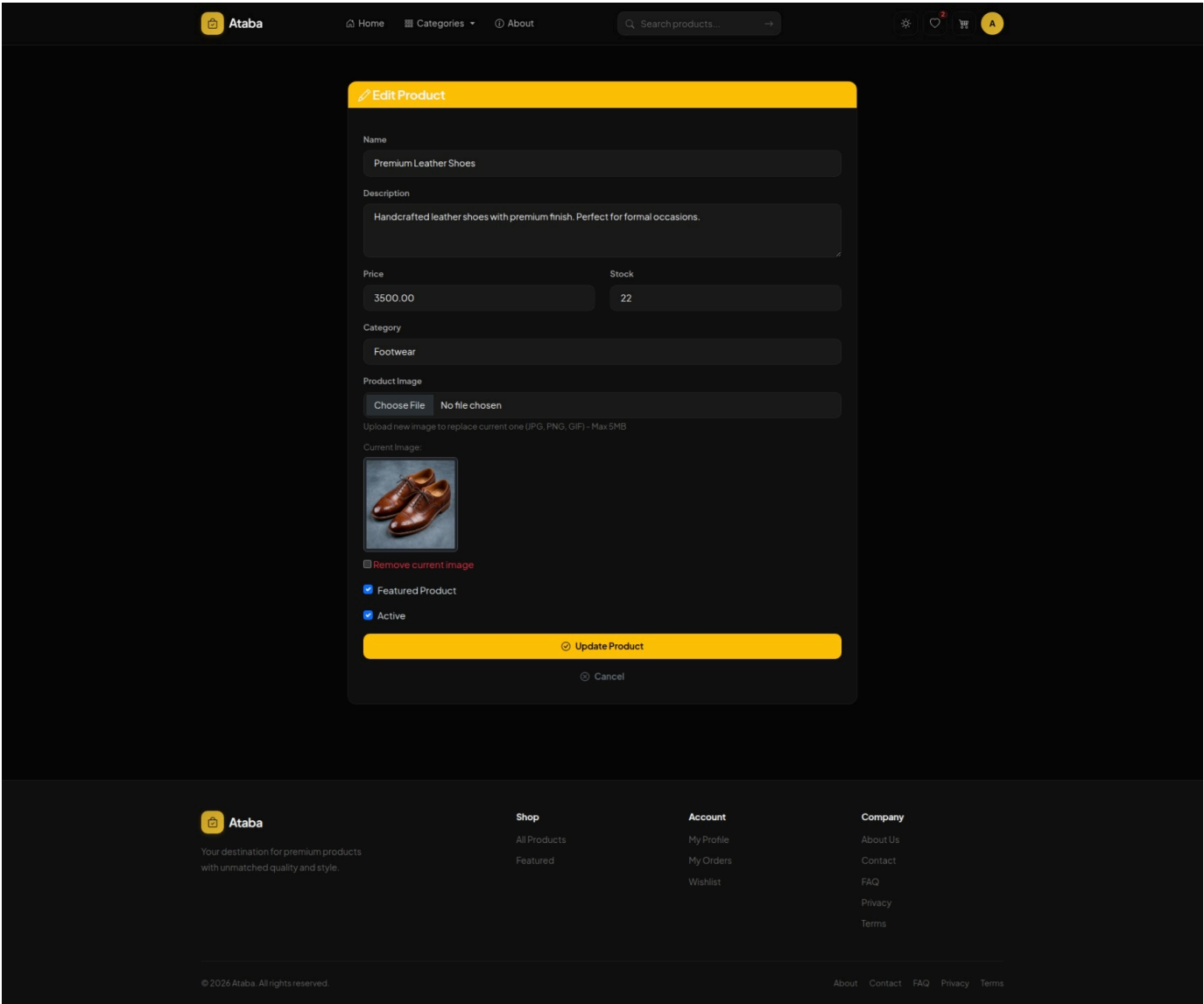
Admin operators maintain the product catalog. The category creation and modification view allows them to define category names, descriptions, and assign visual badges:



- Main category management list: (Visualized in Section 5.4: [images/admin-catigories.jpeg](#))

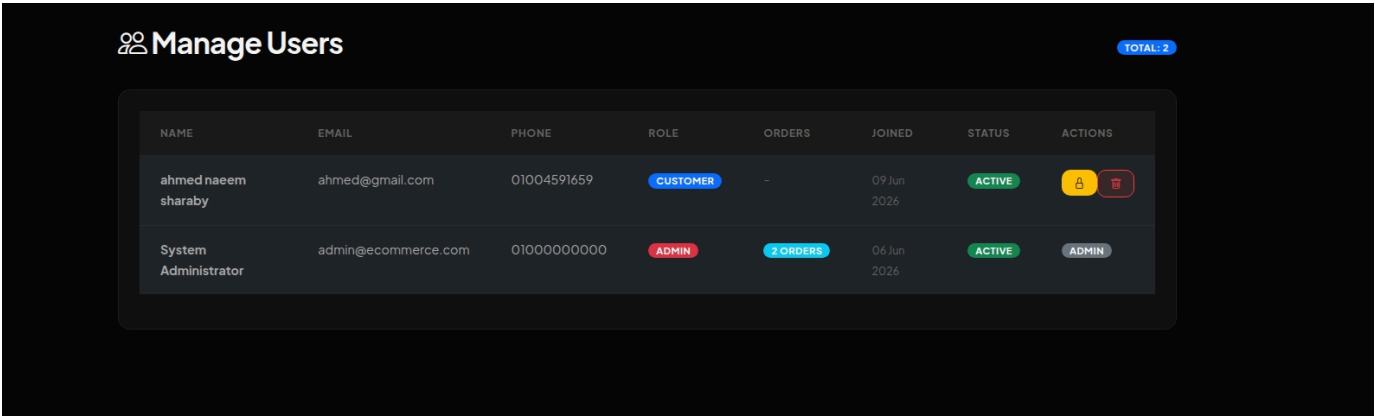
Step 13: Admin Product CRUD

Admin operators upload product photos and configure description parameters:



Step 14: User Accounts Management

Administrators can activate/deactivate user profiles, change roles, and check customer account statuses:



Step 15: Admin Order Processing and Status Updates

Administrators track client purchases, check transaction credentials, and trigger delivery progressions (from Pending to Paid, Shipped, and Delivered):

Ataba

[Home](#) [Categories](#) [About](#)

⚙️

♥️

🛒

A

Order #2

← Back to Orders

Update Order Status

Current Status

PAID

New Status

Paid

Update

Customer Information

Payment Information

Name: System Administrator

Email: admin@ecommerce.com

Phone: 01000000000

Shipping Address:

Cairo, Egypt

Cairo, Egypt

Order Date: 09 June 2026, 04:33 PM

Method: CreditCard

Status: COMPLETED

Transaction ID:

PENDING-639166196182944463

Amount: \$14,250.00

Order Items

IMAGE	PRODUCT	PRICE	QUANTITY	TOTAL
	Classic Silver Timepiece	\$12,500.00	1	\$12,500.00
Total:				\$14,250.00

Ataba

Your destination for premium products with unmatched quality and style.

Shop

[All Products](#)

[Featured](#)

Account

[My Profile](#)

[My Orders](#)

[Wishlist](#)

Company

[About Us](#)

[Contact](#)

[FAQ](#)

[Privacy](#)

[Terms](#)

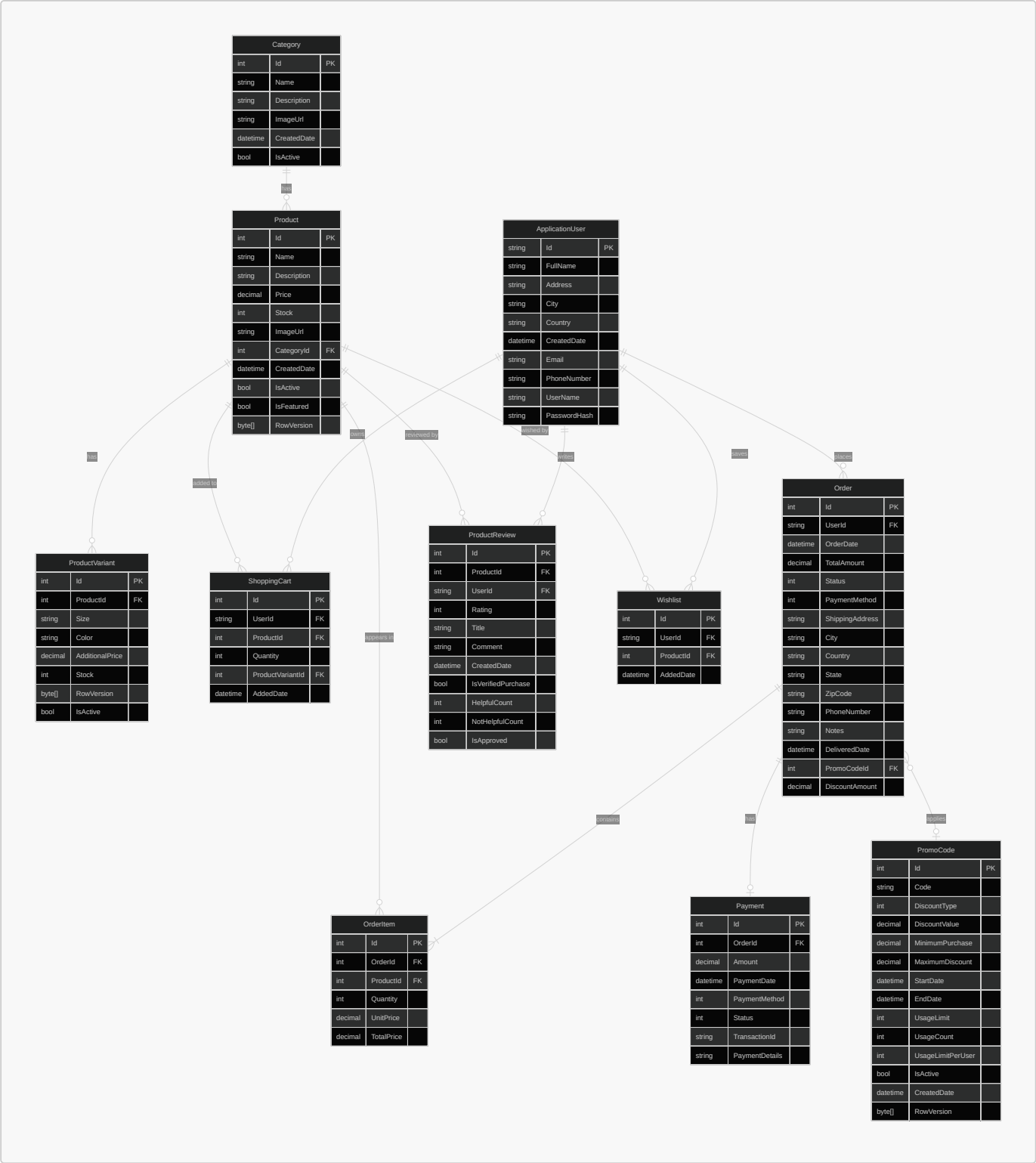
© 2026 Ataba. All rights reserved.

[About](#) [Contact](#) [FAQ](#) [Privacy](#) [Terms](#)

7. System Modeling & Architectural Diagrams

7.1 Entity-Relationship Diagram (ERD)

The database schema of the Ataba platform is built on **Microsoft SQL Server** and managed through **Entity Framework Core** using the Code-First approach. The schema models ten core entities that collectively support user management, product cataloging, shopping cart operations, order processing, payment tracking, promotional discounts, and customer reviews. The design enforces **referential integrity** through explicit foreign key constraints with carefully chosen cascade and restrict delete behaviours — for instance, deleting a **Product** cascades to its **ProductVariants** but restricts deletion if active **OrderItems** reference it. Monetary columns across all entities use a uniform `decimal(18, 2)` precision to ensure consistency in financial calculations. The following ERD captures the entities, their attributes, and the relationships between them:



The diagram illustrates seven **one-to-many** (1:N) relationships and one **one-to-one** (1:1) relationship between **Order** and **Payment**. The **PromoCode** relationship with **Order** is optional (many-to-one with **SET NULL** on delete), allowing orders to exist without a discount code while preserving the code's history for reporting. All foreign key columns are indexed through EF Core conventions or explicit configuration to maintain query performance under load.

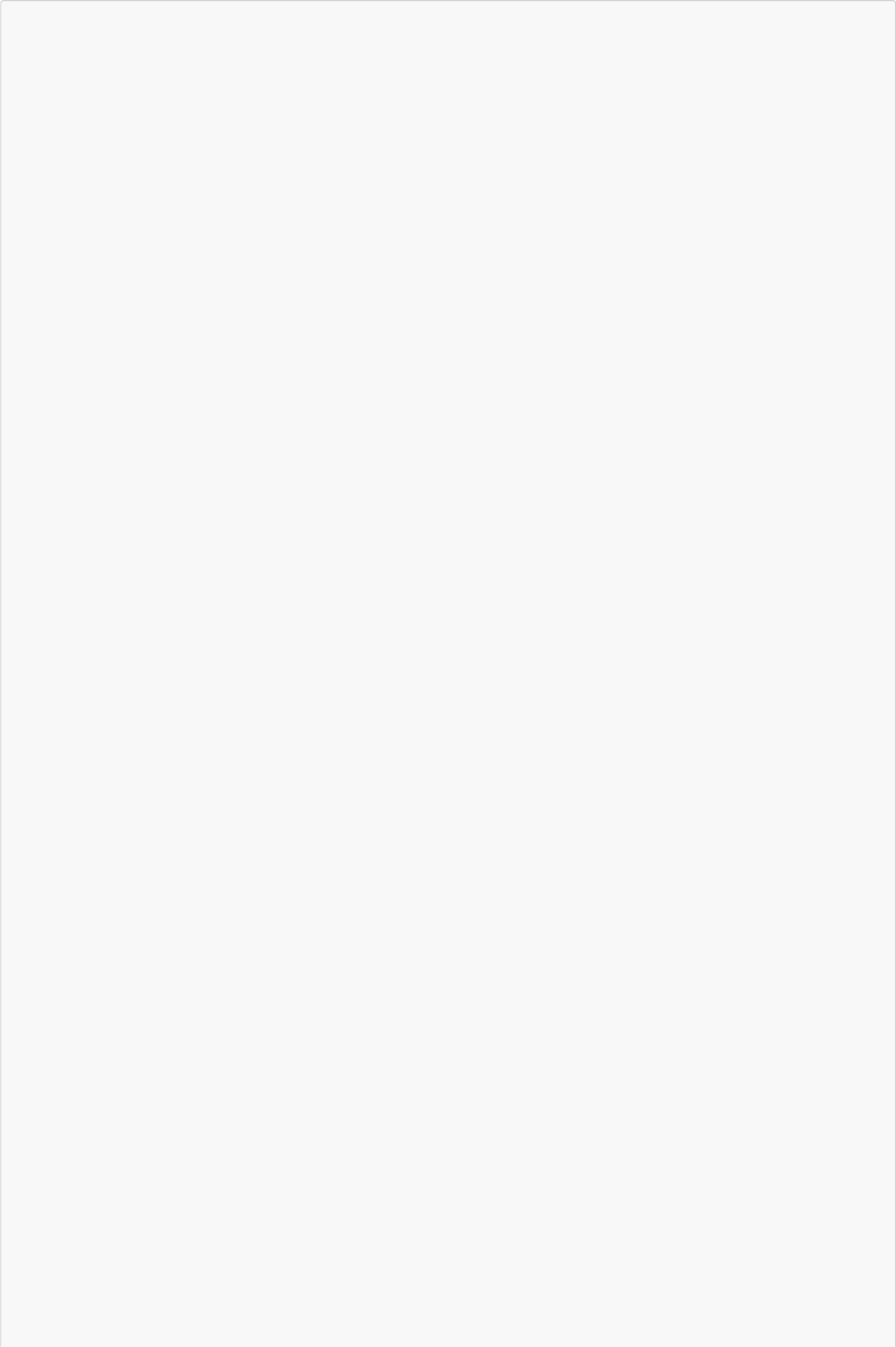
7.2 System Use Case Diagram

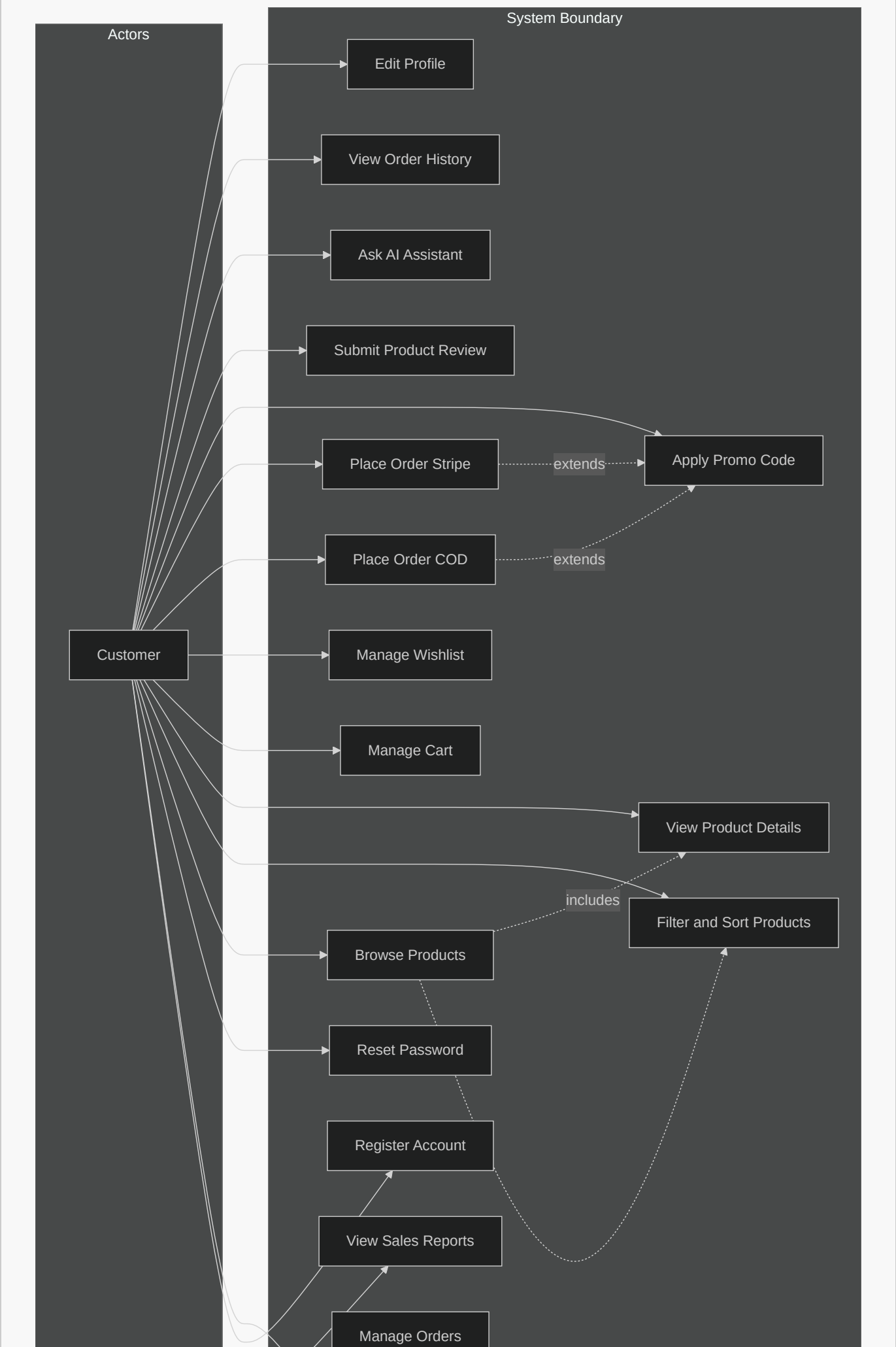
The Ataba platform defines **two primary actors** with distinct responsibilities and access levels:

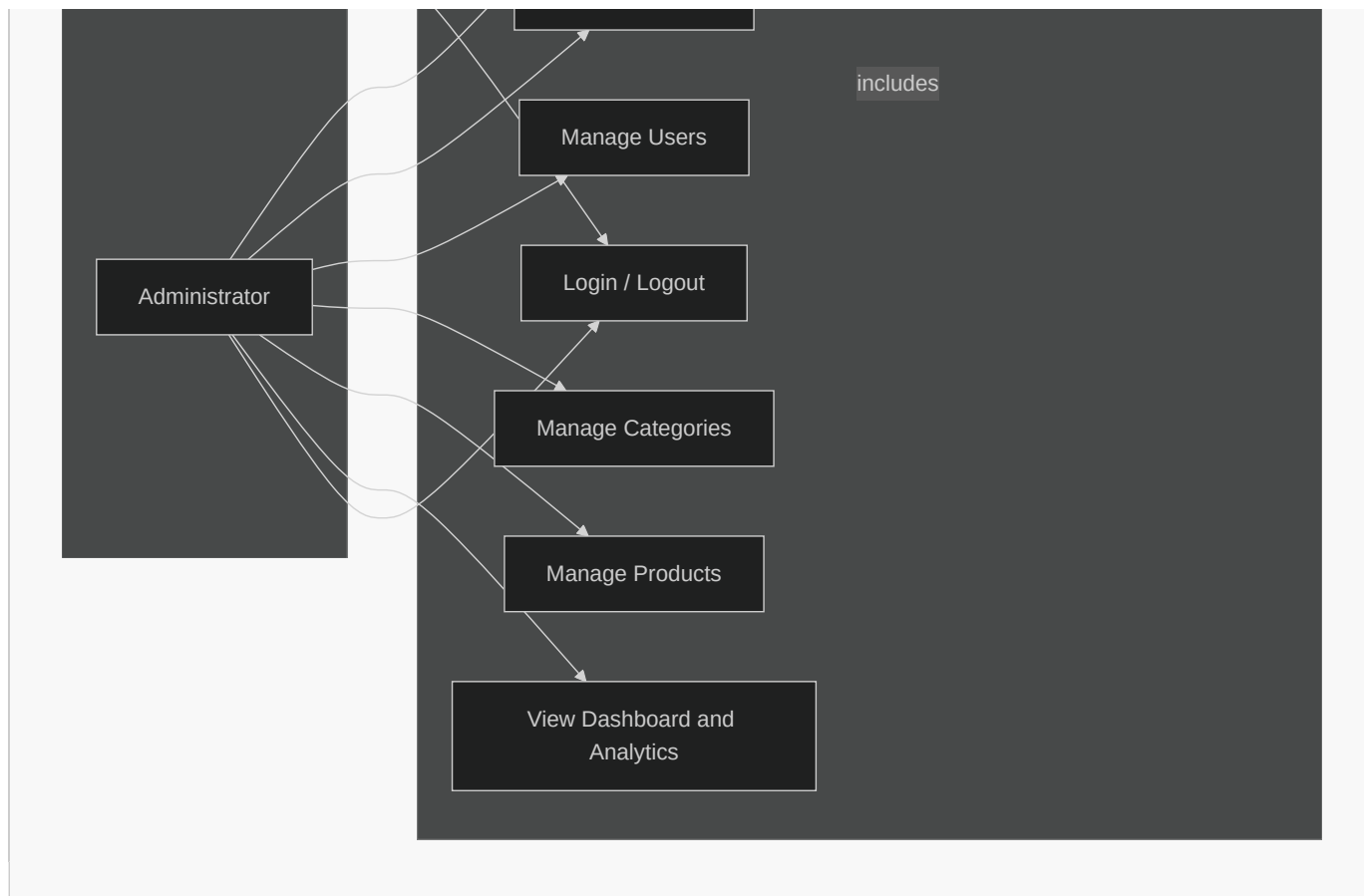
- Customer** — An authenticated user who can browse products, manage a shopping cart and wishlist, place orders via Cash on Delivery or Stripe credit card processing, apply promotional codes, submit product reviews, and interact with the AI-powered product assistant. Customers have access to their order history and profile settings.

- **Administrator** — A privileged user assigned the **Admin** role via ASP.NET Core Identity. Administrators have full access to the dashboard for viewing sales analytics and key performance indicators, managing the product catalog (CRUD operations on products and categories), overseeing user accounts (activation/deactivation and deletion), processing order status transitions, and monitoring inventory levels.

The following use case diagram captures the functional scope of the system from the perspective of each actor:



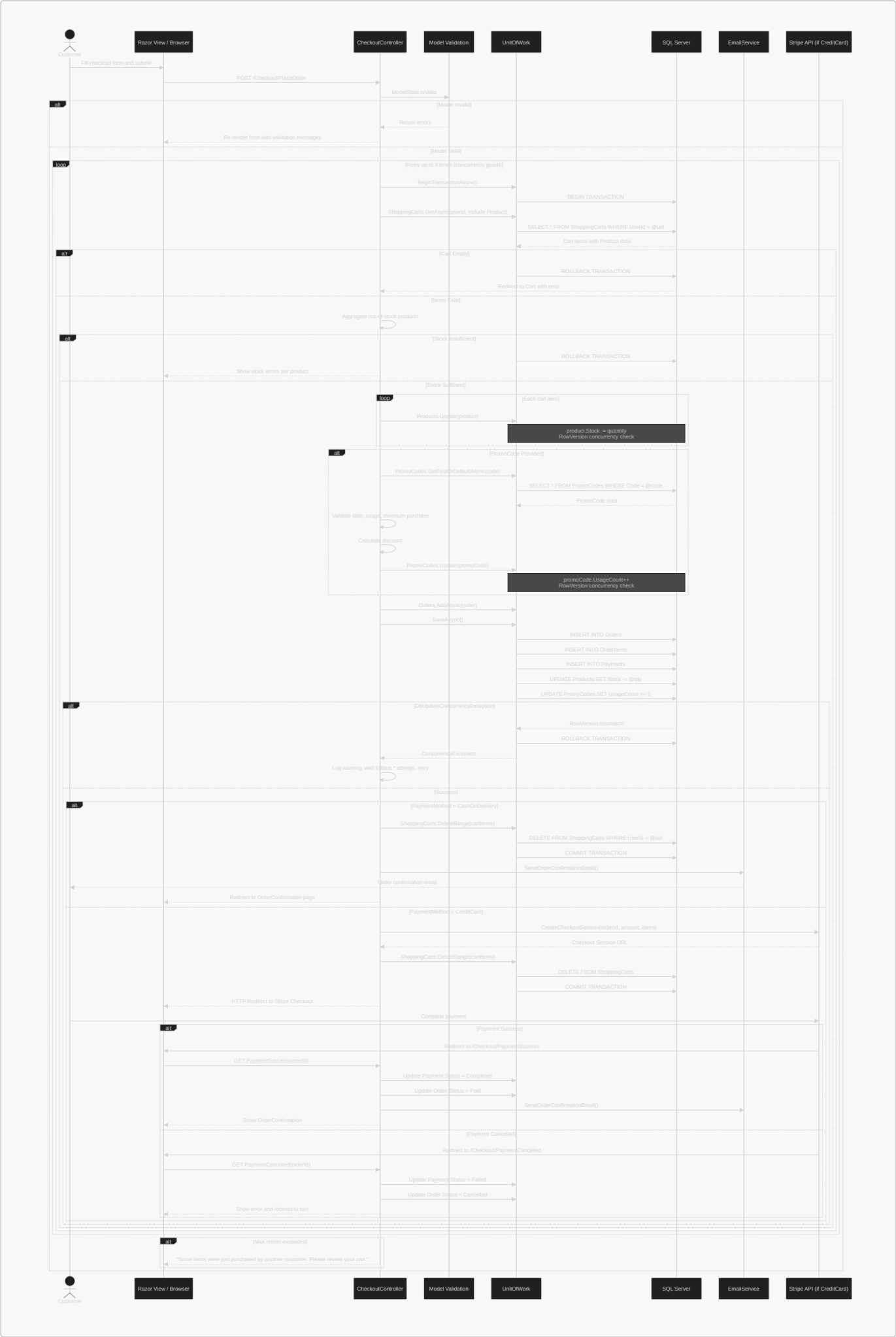




The diagram uses **include** relationships to show that browsing products inherently involves filtering and viewing details, and **extend** relationships to indicate that promo code application is an optional extension of the checkout flow. The authentication use cases (register, login, password reset) are shared across both actors, though administrators log in through the same Identity pipeline to access the admin area.

7.3 Checkout Sequence Diagram

The checkout process represents the most **architecturally critical transaction** in the system. It must atomically validate inventory levels, deduct stock quantities, apply promotional discounts, create order and payment records, and clear the user's cart — all while handling **concurrent access** from multiple shoppers. The system employs a **database-level transaction** (**BeginTransactionAsync**) wrapped in a **retry loop** of up to three attempts to resolve optimistic concurrency conflicts detected via the **[Timestamp] RowVersion** columns on **Product**, **ProductVariant**, and **PromoCode** entities. The following sequence diagram traces the exact message flow for a Cash on Delivery (COD) order, which represents the simplest payment path while still exercising the full transaction pipeline:

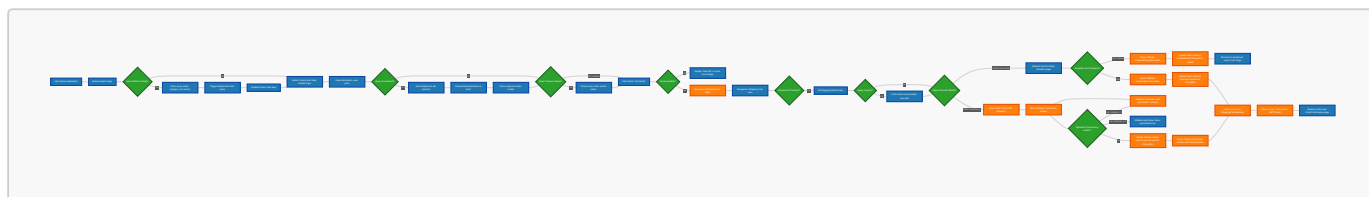


The sequence diagram highlights several architectural decisions:

1. **Transaction boundary** — The `BEGIN TRANSACTION` and `COMMIT/ROLLBACK` operations bracket all write operations, ensuring atomicity. If any `UPDATE`, `INSERT`, or `DELETE` fails, the entire operation rolls back.
2. **Stock validation before writes** — All out-of-stock products are identified and reported to the user before any stock deduction occurs, preventing partial failures.
3. **Concurrency retry loop** — The `DbUpdateConcurrencyException` triggers a full rollback, a logarithmic wait ($100\text{ms} \times \text{attempt number}$), and a retry that re-reads fresh data from the database. This resolves conflicts where two users purchase the last unit of the same product simultaneously.
4. **Payment method branching** — The COD path completes the transaction immediately and sends a confirmation email. The Stripe path delegates payment to the external gateway and handles the callback via two dedicated endpoints (`PaymentSuccess` and `PaymentCancelled`), which update the order and payment statuses accordingly.
5. **Email delivery** — The confirmation email is sent outside the transaction boundary (after commit) and is wrapped in a try/catch so that a transient email failure does not invalidate the completed order.

7.4 Overall Program Flowchart

The flowchart below maps the entire program execution path, illustrating the visual layout navigation, debounced AJAX searches, OpenAI/Gemini modal fetches, variant validation checks, payment branching (Stripe vs Cash on Delivery), transactional concurrency loop, and order completion processes:



8. Conclusion & References

8.1 Conclusion

The development of the Ataba E-commerce platform successfully demonstrates the viability of building a custom, full-stack, self-hosted online commerce application tailored to local and regional market requirements. By using a monolithic ASP.NET Core 10.0 MVC architecture combined with Microsoft SQL Server, the project provides a highly cohesive system that balances performance, ease of deployment, and ease of maintenance for small-to-medium enterprises.

Throughout the project lifecycle, several core milestones were reached and validated:

1. **Enterprise Data Access:** The implementation of the Repository and Unit of Work patterns abstracted data access behind clean interfaces, ensuring proper separation of concerns and database independence.
2. **Concurrency Safety:** By introducing SQL Server row versioning (`[[Timestamp] RowVersion]`) and EF Core optimistic concurrency tokens, the platform successfully mitigated stock overselling and promo code race conditions under high concurrent demand.
3. **Responsive Frontend & Modern UX:** Integrating design tokens, cookie-based theme storage (avoiding Flash of Unstyled Content), and debounced AJAX filters created a desktop and mobile UX that rivals global proprietary platforms.
4. **Secure Payment Processing:** The dual checkout pipeline supported both Cash on Delivery (COD) and Credit Card payments via the Stripe Checkout Session API, catering to low card-penetration markets while offering secure digital payment options.
5. **AI-Driven Customer Experience:** The context-aware AI assistant, built using Google Gemini API with robust rate-limit retries and exponential backoff, successfully demonstrated how generative AI can be securely and effectively integrated into consumer-facing platforms.

8.1.1 Future Work

While the current platform is fully functional and production-ready, several areas are identified for future enhancements:

- **Advanced Search Indexing:** Transitioning from database-level SQL `LIKE` queries to a dedicated search cluster like Elasticsearch to support fuzzy matching, auto-suggestions, and high-performance faceted searches.
- **Distributed Cache Integration:** Migrating from built-in in-memory caching to a distributed Redis cache, enabling the system to scale horizontally across multiple web nodes.
- **Automated Testing Suite:** Implementing a testing project containing unit tests for core services (such as `GeminiService` and `StripePaymentService`) and integration tests for controller workflows.
- **Mobile Native Applications:** Developing native mobile wrappers (using Flutter or React Native) communicating with the backend's ASP.NET Core Web APIs to capture the mobile-first customer base.

8.2 References

1. **Microsoft Corporation.** (2025). *ASP.NET Core MVC Documentation: Overview of MVC architecture*. Retrieved from <https://learn.microsoft.com/aspnet/core/mvc>.
2. **Microsoft Corporation.** (2025). *Entity Framework Core Documentation: Handling Concurrency Conflicts*. Retrieved from <https://learn.microsoft.com/ef/core/saving/concurrency>.
3. **Stripe Inc.** (2026). *Stripe Checkout API Reference: Creating checkout sessions*. Retrieved from <https://stripe.com/docs/api/checkout/sessions>.
4. **Google Cloud.** (2026). *Gemini API Documentation: Developer Guides & SDK References*. Retrieved from <https://ai.google.dev/gemini-api/docs>.
5. **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional. (Details regarding the Repository and Unit of Work patterns).

6. **Martin, R. C.** (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
(Guidelines on separation of concerns and interface boundaries).
7. **Fielding, R., & Reschke, J.** (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. RFC 7231, Internet Engineering Task Force (IETF).
8. **Barth, A.** (2011). *HTTP State Management Mechanism (Cookies)*. RFC 6265, Internet Engineering Task Force (IETF).